

# 기획서 규칙

완성된 기획 - 검은색 글씨

미완성 기획 - 파란색 글씨

수정 예정 & 미확정 기획 - 핑크색 글씨

새로 생긴 기획 - 빨간색 글씨

필요한 클래스 & 필요한 이벤트는 예시 사항 꼭 지켜야 할 필요는  
없음

## 제작 이유와 개요

왜 이 게임을 기획하는가?

요즘 나오고 유행하고 사람들이 즐겨워하는 게임들이 함께 협동하고 멀티 플레이를 하는 것을 선호한다, **NPC**나 **AI**로 대체하기 어려운 플레이어 행동을 기반으로 재미가 발생한다

플레이어 간 커뮤니케이션을 핵심 콘텐츠로 설계하고 시스템을 기획하여 플레이어 간 상호작용을 중심으로 한 게임을 구현하는 것이 목표

따라서 본 프로젝트는 협력, 배신, 정보, 음성 소통이 실제 플레이 경험에 어떤 영향을 주는지 검증하는 것을 핵심 목표로 삼는다

왜 이 게임을 제작하는가?

플레이어 간 관계 변화에서 발생하기 때문에 멀티플레이 환경이 필수적이다

서버 게임 제작 미완성에 대한 걱정은 제작 범위를 극단적으로 줄여 완성 가능성을 확보한다

최소인원 4인

맵 1개

역할 2개

음성 채팅 사용

매치메이킹 없음, 초대 코드로만 초대 가능

게임명

- **Trust No One**

장르

- 1인칭, 멀티플레이, 사회적 추리, 협동, 생존

플랫폼

- **PC**

게임 한 줄 소개

- 모든 플레이어가 각자의 목표를 가진 채 제한된 시간 동안 생존하고, 행동과 정보를 바탕으로 진실을 추리하며 탈출을 노리는 사회적 추리 게임

게임 소개

- 플레이어들은 폐쇄된 시설에 갇힌 생존자가 되어 제한된 시간 동안 다양한 미션을 수행하며 탈출을 준비한다  
그러나 생존자 중에는 다른 플레이어를 제거하려는 살인마가 숨어 있으며, 모든 시민 역시 자신만의 개인 목표를 가지고 있기 때문에 항상 협력하려 하지 않는다  
누군가는 회복 아이템을 숨기고,  
누군가는 미션을 일부러 하지 않으며,  
누군가는 거짓 정보를 흘린다  
이러한 행동들은 모두 시민일 수도 있고 살인마일 수도 있기 때문에  
플레이어는 단순히 말을 믿는 것이 아니라 행동과 정보를 종합하여 진실을 추리해야한다  
게임 후반에는 탈출 가능한 인원이 제한된 탈출구가 등장하며,  
남은 플레이어들은 새로운 탈출구를 찾거나 끝까지 살아남기 위해 서로를 이용하거나 배신하게 된다

핵심 컨셉



- 살인마를 찾는 게임이 아닌 모두가 거짓말을 하고 그 속에서 시민과 범인을 추리하여 탈출해야하는 게임이다

#### 핵심 재미

- 회의 시간이 따로 없고 행동과 상황만으로 자연스럽게 이어지는 추리  
단체 미션이 아닌 개인 미션으로 인한 거짓말  
탈출 인원 제한으로 시민 사이에서도 배신이 발생  
중립의 존재  
“모든 플레이어가 각자의 목표를 위해 거짓말할 수 있기 때문에 누구도 완전히 믿을 수 없는 상태에서 행동과 정보를 조합해 진실을 추리하는 것”

#### 우선적으로 제작해야하는 시스템들

| 우선 순위 | 시스템         | 이유                            |
|-------|-------------|-------------------------------|
| 0     | 기본 시스템      | 모든 시스템의 기반                    |
| 0-1   | 이동 시스템      | 모든 행동의 시작                     |
| 0-2   | 스태미너 시스템    | 이동, 추격, 도주의 기반                |
| 0-3   | 상호작용 시스템    | 미션, 아이템, 문 등 공통 사용            |
| 0-4   | 애니메이션 시스템   | 모든 행동의 기반                     |
| 0-5   | 플레이어 상태 시스템 | Idle, Run, Dead, Interact     |
| 1     | 역할 시스템      | 시민, 살인마, 중립의 행동과 목표가 추리를 만들어냄 |
| 2     | 미션 시스템      | 플레이어가 흩어지고 행동할 이유가 됨          |
| 3     | 추리 시스템      | 게임의 핵심 재미                     |
| 4     | 신뢰도, 의심 시스템 | 행동이 곧 정보                      |
| 5     | 살인 시스템      | 긴장감                           |
| 6     | 탈출 시스템      | 승부를 결정하는 장치                   |
| 7     | 중립 역할 시스템   | 긴장감과 의심을 더욱 강화                |
| 8     | 아이템 / 제작    | 전략을 다양하게 만듦                   |
| 9     | 거짓말 탐지 / 증거 | 후반부 정보 강화                     |

#### 상위 유사게임

- Lockdown protocol
- First Class Trouble
- Goose Goose Duck

하위 유사게임

- Dale & Dawson Stationery Supplies

키 지정

|              |       |
|--------------|-------|
| 기본 이동        | WASD  |
| 점프           | Space |
| 달리기          | Shift |
| 상호작용         | F     |
| 인벤토리 슬롯      | 1234  |
| 마이크 음소거, 활성화 | V     |
| 미정이          | Q     |
| 미정이          | E     |
| 미정이          | R     |
| 설정, 메뉴       | ESC   |
| 지도, 작업 목록    | M     |
| 공격, 물건 집기    | ML    |
| 물건 던지기/내려놓기  | MR    |

# 인터페이스 문서(API 문서)

살인

Kill(Player A, Player B)

입력

공격자

피격자

출력

Success

Fail

이벤트

OnPlayerDead()

아이템

PickupItem()

Dropltem()

UseItem()

DestroyItem()

탈출

EnterExit()

LeaveExit()

ExitComplete()

# 튜토리얼 시스템

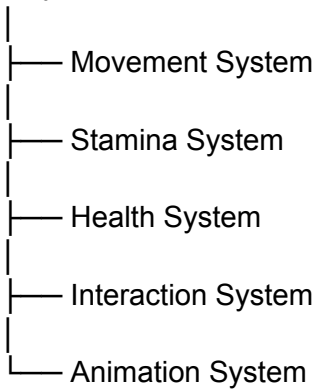




# 클라 시스템

플레이어 구조

Player



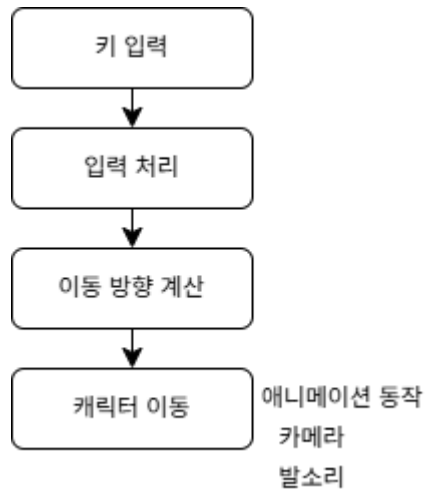
|        |       |          |
|--------|-------|----------|
| 시스템    | 서버 판정 | 클라이언트    |
| 이동     | 위치 검증 | 이동 애니메이션 |
| 스태미너   | 서버    | UI 변화    |
| 살인     | 서버 승인 | 이펙트      |
| 아이템 획득 | 서버    | UI       |
| 미션 완료  | 서버    | 진행도 표시   |
| 탈출     | 서버    | 연출       |
| 사망     | 서버    | 애니메이션    |
| 증거 생성  | 서버    | 표시       |

# 플레이어이동시스템

## 시스템 목적

- 플레이어가 맵을 자유롭게 이동하여 미션을 수행하고, 다른 플레이어를 관찰하며, 추리와 생존 활동을 할 수 있도록 한다
- 이동은 모든 행동의 기반이 되는 시스템이며, 모든 플레이어가 동일한 조건에서 이동해야 한다

## 시스템 구조



## 이동 방식

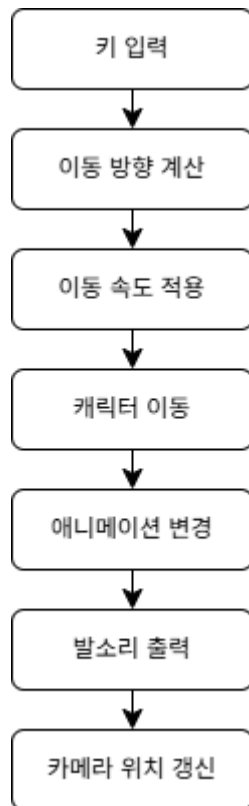
| 항목        | 내용    |
|-----------|-------|
| 이동 방식     | WASD  |
| 달리기       | Shift |
| 점프        | Space |
| 시점        | 마우스   |
| 상호작용 중 이동 | 불가능   |
| 공격 중 이동   | 제한    |
| 사망 상태     | 불가능   |

## 입력 처리

| 입력    | 기능   |
|-------|------|
| W     | 전진   |
| A     | 좌측이동 |
| S     | 후진   |
| D     | 우측이동 |
| Shift | 달리기  |

|       |    |
|-------|----|
| Space | 점프 |
|-------|----|

이동 처리 순서



입력 처리

- WASD 입력 감지
- Shift 입력 감지
- Space 입력 감지

이동

- 현재 이동 방향 계산
- 이동 속도 적용
- 달리기 속도 적용
- 캐릭터 이동

애니메이션

| 상태   | 애니메이션 |
|------|-------|
| Idle | 대기    |
| Walk | 걷기    |
| Run  | 달리기   |
| Jump | 점프    |
| Dead | 사망    |

## 카메라

- 플레이어를 따라 이동
- 플레이어 회전에 맞춰 시점 변경
- 부드러운 카메라 이동

## 발소리

- 걷기 -> 발소리 출력
- 달리기 -> 달리기 발소리 출력
- 정지 -> 발소리 종료

| 상태   | 출력         |
|------|------------|
| Walk | Walk Sound |
| Run  | Run Sound  |
| Idle | 없음         |

## 이동 제한

| 상태     | 이동 가능 |
|--------|-------|
| 기본     | O     |
| 미션 수행  | X     |
| 탐지기 사용 | X     |
| 제작     | X     |
| 사망     | X     |
| 관전자    | 자유 이동 |

## 예외 처리

- 이동 중 사망
  - 이동 즉시 정지
  - 사망 애니메이션 재생
- 이동 중 상호작용
  - 이동 입력 무시
- 이동 중 제작
  - 이동 불가
- 이동 중 점프 불가 상태
  - 점프 입력 무시

## 서버 / 클라 분리

| 클라    | 서버    |
|-------|-------|
| 입력 처리 | 이동 검증 |

|              |            |
|--------------|------------|
| 캐릭터 이동 연출    | 최종 위치 판정   |
| 애니메이션 재생     | 충돌 검사      |
| 카메라 제어       | 속도 검증      |
| 발소리 재생       | 위치 동기화     |
| 상태에 따른 이동 제한 | 플레이어 상태 관리 |

# 체력시스템



## 시스템 목적

- 체력 시스템은 플레이어 생존 상태를 관리하는 핵심 시스템
- 플레이어는 피해를 입거나 회복 아이템을 사용하면서 체력이 변화하며, 체력이 0이 되면 사망 상태로 전환 된다
- 스테미너 사용이 0이되면 체력이 초당 10깍인다
- 체력은 모든 플레이어가 동일한 규칙을 적용받으며, 서버가 최종 값을 관리한다

## 시스템 역할

- 체력 UI 표시
- 체력 변화 애니메이션
- 피격 연출
- 회복 연출
- 저체력 경고 연출
- 사망 연출
- 플레이어 입력 제한 반영

## 시스템 구조

### 기본 표시

| 상태      | UI        |
|---------|-----------|
| 체력 100% | 전체 표시     |
| 체력 감소   | 체력바 감소 연출 |
| 체력 회복   | 체력바 증가 연출 |
| 체력 0    | 체력바 비움    |

### 피격 연출

- 화면 가장자리 붉은 효과
- 화면 흔들림
- 피격 사운드 재생
- 체력바 감소 애니메이션

### 회복 연출

- 체력바 증가 애니메이션
- 회복 이펙트 출력
- 회복 효과음 재생

### 저체력 경고

- 화면 가장자리 붉은 효과
- 심장 박동 소리 재생
- 화면 채도 감소
- 주변 플레이어 음성 작게 출력
- 화면 가운데 경고 표시

### 사망 연출

- 사망 애니메이션 재생
- 체력 UI 비활성화
- 피격 효과 종료
- 회복 효과 종료
- 입력 제한 적용
- 관전자 화면으로 전환

#### 상태별 UI

| 상태 | 체력 표시 | 피격 효과 | 회복 효과 |
|----|-------|-------|-------|
| 생존 | O     | O     | O     |
| 사망 | O     | O     | X     |
| 관전 | X     | X     | X     |
| 탈출 | X     | X     | X     |

#### UI 구성

##### 좌측 하단

- 체력바
- 체력 수치

##### 화면 중앙

- 회복 효과

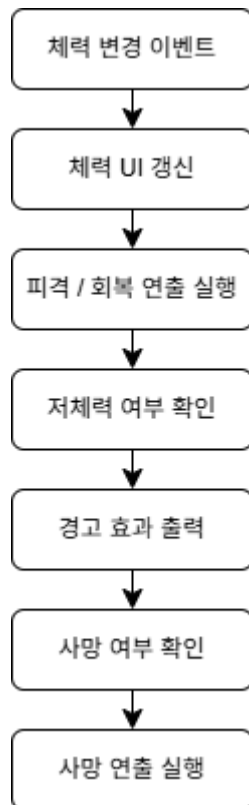
##### 화면 가장자리

- 저체력, 피격 붉은 효과

##### 오디오

- 피격 사운드
- 회복 사운드
- 심장 박동 소리

#### 처리 순서



#### 예외 처리

- 연속 피격 -> 효과 중첩 없이 갱신
- 연속 회복 -> 체력 UI만 갱신
- 피격 중 회복 -> 가장 최근 체력 값 기준으로 UI 갱신
- 사망 상태에서 체력 변경 이벤트 수신 -> UI 변경 없음
- 저체력 상태에서 회복 -> 저체력 효과 즉시 해제
- 관전자 전환 -> 체력 UI 숨김
- 탈출 완료 -> 체력 UI 비활성화

#### 서버 / 클라 분리

| 클라          | 서버         |
|-------------|------------|
| 체력 UI 표시    | 체력 계산      |
| 피격 연출       | 피해 적용      |
| 회복 연출       | 회복 처리      |
| 저체력 경고 효과   | 생존 여부 판정   |
| 사망 애니메이션    | 사망 판정      |
| 화면 효과 및 사운드 | 최종 체력 값 관리 |

사망,시체,관전자 시스템

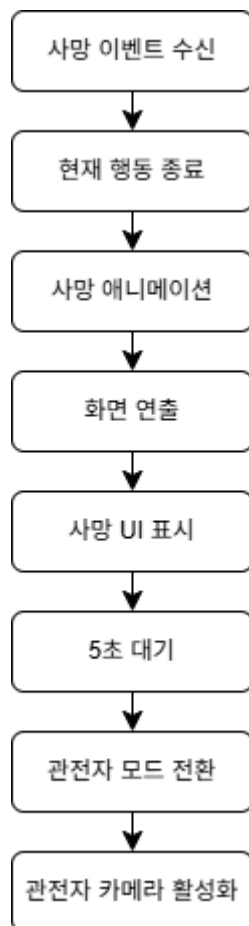
### 시스템 목적

- 사망 시스템은 플레이어의 생존 여부를 관리하고, 사망한 플레이어를 관전자 상태로 전환하여 게임의 진행을 방해하지 않으면서도 게임을 계속 관람할 수 있도록 하는 시스템이다
- 사망 판정은 반드시 서버에서 수행하며, 모든 클라이언트는 서버의 결과를 동기화한다

### 시스템 역할

- 사망 연출 출력
- 사망 UI 표시
- 아이템 처리
- 플레이어 입력 제한
- 관전자 모드 전환
- 관전자 카메라 제어
- 관전자 UI 표시
- 결과 화면 전환

### 시스템 구조

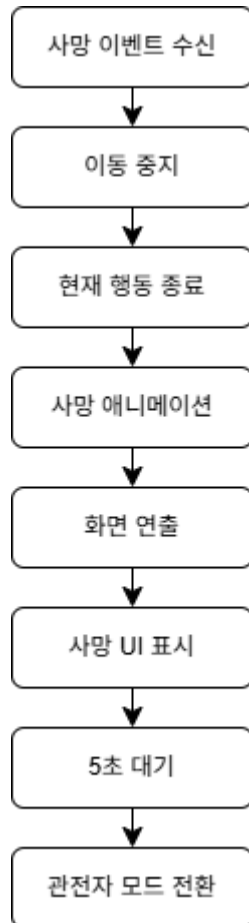


### 사망 연출

- 이동 즉시 정지
- 현재 애니메이션 종료
- 사망 애니메이션 재생

- 사망 효과음 재생
- 화면 암전 효과 출력
- 입력 비활성화

#### 사망 처리 순서



#### 플레이어 상태

| 상태  | 이동 | 상호작용 | 채팅         | 관전 |
|-----|----|------|------------|----|
| 생존  | O  | O    | O          | X  |
| 사망  | X  | X    | X          | X  |
| 관전자 | O  | X    | O(관전자 채팅만) | O  |
| 탈출  | X  | X    | X          | O  |

#### 시체 시스템

- 사망 시 시체가 생성된다

#### 시체는

- 이동 불가
- 삭제 불가
- 근처에 들고있던 아이템 드랍
- 다른 플레이어 발견 가능

#### 아이템 처리

- 사망 시 아이템 전부 바닥에 드랍

#### 증거 생성

- 사망 시 혈흔

#### 관전자 시스템

- 목적
  - 사망한 플레이어가 게임 진행을 관람할 수 있도록 한다
  - 단, 생존자에게 정보를 전달하지 못하도록 생존자에게는 관전자가 보이지 않는다

#### 관전자 카메라

- 자유 관전
  - 맵 자유 이동
  - 플레이어 출동 없음
  - 미니맵 사용 가능
- 플레이어 관전
  - 관전 버튼으로 랜덤 플레이어 시점으로 전환
  - 좌우 버튼으로 관전 대상 변경
  - 현재 관전 대상 표시

#### 관전자 UI

- 관전하기 버튼(자유 상태일때만 표시)
- 화면 닫기 버튼(관전 상태일때만 표시)
- 현재 관찰 : **PlayerA**(카메라 관전 중일 때만 표시)
- 이전 플레이어(카메라 관전 중일 때만 표시, 좌측 변경 버튼)
- 다음 플레이어(카메라 관전 중일 때만 표시, 우측 변경 버튼)
- 관전 채팅(관전자 전용 음성채팅)

#### 관전자 기능

| 기능      | 가능 여부                  |
|---------|------------------------|
| 플레이어 변경 | O                      |
| 자유 이동   | O(이동과 관전 카메라 둘다 사용 가능) |
| UI 보기   | O                      |
| 미니맵     | O                      |
| 채팅      | 관전자 전용                 |
| 핑 찍기    | X                      |
| 아이템 사용  | X                      |

#### 게임 종료

- 관전자 카메라 종료
- 결과 화면 출력
- 입력 비활성화

#### 예외 처리

- 사망 연출 중 입력 -> 입력 무시
- 관전 대상이 사망 -> 다음 생존 플레이어로 자동 변경
- 관전 가능한 플레이어 없음 -> 결과 화면으로
- 게임 종료 직전 사망 -> 사망 연출 후 결과 화면 표시
- 결과 화면 진입 -> 관전자 UI 비활성화
- 자유 관전 중 플레이어 선택 -> 선택한 플레이어 시점으로 즉시 전환
- 플레이어 관전 해제 -> 자유 관전 모드로 복귀

#### 서버 / 클라 분리

| 클라          | 서버        |
|-------------|-----------|
| 사망 애니메이션    | 사망 판정     |
| 화면 암전 및 효과음 | 사망 위치 관리  |
| 입력 제한       | 시체 생성     |
| 관전자 카메라     | 아이템 드롭 처리 |
| 관전자 UI      | 증거 생성     |
| 플레이어 시점 전환  | 관전자 상태 관리 |
| 결과 화면 표시    | 게임 종료 판정  |



# 스태미너 시스템

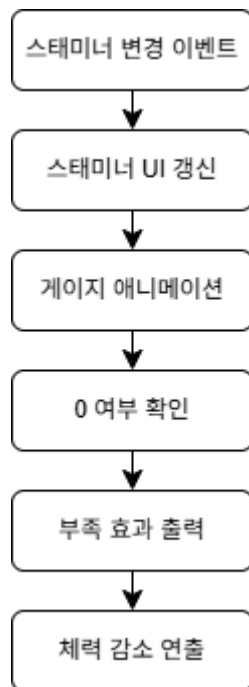
#### 시스템 목적

- 플레이어의 행동을 제한하여 무한 질주를 방지하고, 이동 경로와 행동 선택에 전략성을 부여한다

#### 시스템 역할

- 스테미너 UI 표시
- 스테미너 변화 애니메이션
- 스테미너 부족 연출
- 체력 감소 연출
- 이동 애니메이션 반영

#### 시스템 구조



#### UI 구성

| 항목    | 내용            |
|-------|---------------|
| 위치    | 체력바 아래        |
| 형태    | 노란색 게이지       |
| 표시 방식 | 현재 스테미너 비율 표시 |
| 최대값   | 100% 기준       |

#### 스테미너 표시

| 상태 | UI           |
|----|--------------|
| 최대 | 전체 표시        |
| 감소 | 게이지 감소 애니메이션 |

|    |              |
|----|--------------|
| 회복 | 게이지 증가 애니메이션 |
| 0  | 빈 게이지 표시     |

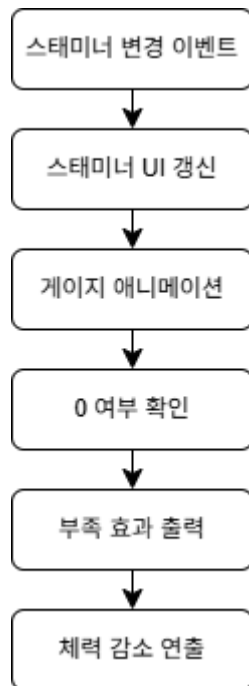
#### 스태미너 변화 연출

- 스태미너 감소
  - 게이지 감소 애니메이션
  - 달리기 또는 점프 애니메이션 유지
- 스태미너 회복
  - 게이지 증가 애니메이션
  - 자연스럽게 게이지 복원

#### 스태미너 부족 연출

- 화면 가장자리 노란색 효과
- 경고 아이콘 표시
- 경고 효과음 재생
- 체력 감소 시 피격 연출 재생

#### 처리 순서



#### 상태별 UI

| 상태     | 게이지 표시 | 경고 효과 |
|--------|--------|-------|
| 일반     | O      | X     |
| 감소 중   | O      | X     |
| 회복 중   | O      | X     |
| 스태미너 0 | O      | O     |

|    |   |   |
|----|---|---|
| 사망 | X | X |
|----|---|---|

예외 처리

연속 달리기 -> 스테미너 UI 지속 갱신

점프 후 스테미너 0 -> 부족 효과 및 체력 감소 연출 표시

회복 중 다시 소비 -> 회복 애니메이션 중단 후 감소 애니메이션 실행

스테미너 0 상태 유지 -> 부족 효과 유지 및 체력 감소

사망 -> 스테미너 UI 비활성화

게임 종료 -> UI 비활성화

서버 / 클라 분리

| 클라              | 서버              |
|-----------------|-----------------|
| 스테미너 게이지 표시     | 스테미너 값 계산       |
| 게이지 감소 회복 애니메이션 | 스테미너 소비 및 회복 처리 |
| 부족 경고 효과        | 체력 감소 판정        |
| 체력 감소 연출        | 달리기 가능 여부 판정    |
| 이동 애니메이션 반영     | 사망 판정           |

# 사운드시스템



# 음성 시스템

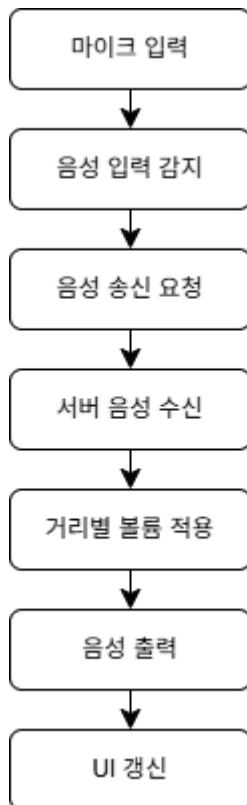
### 시스템 목적

- 음성 시스템은 플레이어 간 실시간 음성 채팅을 관리하는 시스템이다
- 클라이언트는 플레이어의 마이크 입력을 처리하고, 서버로부터 전달받은 음성을 재생하며, 음성 상태를 **UI**를 통해 표시한다

### 시스템 역할

- 마이크 입력 처리
- 음성 송신 시작 및 종료
- 음성 재생
- 거리별 볼륨 적용
- 음소거 기능
- 음성 상태 **UI** 표시
- 채널 변경 **UI** 반영

### 시스템 구조



### 음성 입력 방식

| 항목       | 내용           |
|----------|--------------|
| 방식       | 음성 인식        |
| 음소거      | V 키          |
| 출력       | 실시간 음성       |
| 음성 상태 표시 | 말하는 중 아이콘 표시 |



#### 클라이언트 처리

| 기능     | 내용              |
|--------|-----------------|
| 마이크 입력 | 음성 데이터 수집       |
| 음성 재생  | 서버에게 전달받은 음성 출력 |
| 볼륨 조절  | 거리 비례 볼륨 적용     |
| 음소거    | 마이크 송신 차단       |
| UI 표시  | 말하는 플레이어 표시     |

#### 음성 처리 순서

| 순서 | 처리          |
|----|-------------|
| 1  | 마이크 입력 감지   |
| 2  | 음성 송신 시작    |
| 3  | 서버로부터 음성 수신 |
| 4  | 거리별 볼륨 적용   |
| 5  | 음성 출력       |
| 6  | 음성 UI 갱신    |

#### 플레이어 상태별 음성

| 상태     | 마이크 입력    | 음성 재생          |
|--------|-----------|----------------|
| 생존     | O(생존자 채널) | O(생존자 채널)      |
| 사망(대기) | X         | X              |
| 관전     | O(관전자 채널) | O(관전자, 생존자 채널) |
| 탈출     | X         | O(생존자 채널)      |

#### 거리 판정

| 거리    | 볼륨   |
|-------|------|
| 0~3m  | 100% |
| 3~7m  | 70%  |
| 7~12  | 40%  |
| 12~15 | 10%  |

|        |        |
|--------|--------|
| 15m 이상 | 들리지 않음 |
|--------|--------|

#### 음소거 기능

- V키를 눌러 자신의 마이크를 음소거 / 해제한다
- 음소거 상태에서는 음성이 송신되지 않는다
- UI에 음소거 아이콘을 표시한다

#### 음성 UI

- 플레이어 머리위
  - 말하는 중 아이콘 표시
- 화면 좌측
  - 현재 말하는 플레이어 목록
  - 음소거 아이콘 표시
- 설정 메뉴
  - 전체 음량
  - 음성 채팅 음량
  - 마이크 음량
  - 입력 장치 선택
  - 출력 장치 선택
- 채널 표시
  -

| 상태  | UI        |
|-----|-----------|
| 생존  | 생존자 채널 표시 |
| 관전자 | 관전자 채널 표시 |
| 탈출  | 관전자 채널 표시 |

#### 상태 변화

| 상황     | 처리           |
|--------|--------------|
| 사망     | 음성 송신 종료     |
| 관전자 전환 | 관전자 채널 UI 표시 |
| 탈출     | 생존자 채널 UI 제거 |
| 게임 종료  | 음성 UI 비활성화   |

#### 필요한 클래스

- VoiceInputController
- VoiceOutputController
- VoiceUIController
- VoiceVolumeController
- VoiceSettingController
- VoiceIndicatorController

#### 필요한 이벤트

- OnVoiceStart
- OnVoiceStop
- OnVoiceReceived
- OnMute
- OnUnmute
- OnVoiceChannelChanged
- OnGameEnd

#### 예외처리

음소거 상태 -> 음성 송신 중지 및 음소거 아이콘 표시

마이크 미연결 -> 경고 메시지 출력(플레이어 화면 가운데 출력)

출력 장치 없음 -> 음성 재생 중지 및 안내 표시

음성 수신 중 대상이 범위 밖으로 이동 -> 볼륨을 점진적으로 감소 후 재생 종료

플레이어 사망 -> 음성 송신 종료 및 UI 갱신

게임 종료 -> 모든 음성 재생 및 UI 종료

로비로 이동 -> 거리 비례 음성 채널 복구(로비 음성 채널)

#### 서버 / 클라 분리

| 클라                | 서버           |
|-------------------|--------------|
| 마이크 입력            | 음성 송신 허용 여부  |
| 음성 재생             | 음성 수신 대상     |
| 거리별 볼륨 적용         | 거리 계산        |
| 말하는 플레이어 UI       | 음성 채널 관리     |
| 음소거 UI            | 플레이어 상태 확인   |
| 음성 설정(입출력 장치, 볼륨) | 게임 종료시 음성 종료 |

# 게임진행시스템

## 시스템 목적

- 서버에서 전달되는 게임 진행 상태를 클라이언트에 반영하는 시스템이다.
- 게임 시작부터 종료까지 현재 게임 상태와 남은 게임 시간을 플레이어에게 표시한다.
- 역할 및 미션 배정이 완료되면 게임 진행 화면을 표시한다.
- 미션, 사망, 탈출 등의 상태 변화를 전달받아 플레이어 화면에 반영한다.
- 게임 종료 시스템에서 전달되는 게임 종료 상태에 따라 플레이어의 행동을 제한하고 결과 화면으로 전환한다.

클라이언트는 게임 진행 상태를 판정하지 않으며, 서버에서 전달된 데이터를 표시하고 이에 맞는 **UI**와 화면 상태만 변경한다.

## 2. 시스템 역할

- 게임 시작 화면 표시
- 게임 상태 표시
- 게임 진행 시간 표시
- 게임 시작 연출
- 역할 배정 완료 상태 반영
- 미션 배정 완료 상태 반영
- 게임 진행 중 **UI** 관리
- 플레이어 상태 변화 반영
- 탈출 상태 반영
- 게임 종료 상태 반영
- 게임 종료 연출
- 결과 화면 전환

## 3. 게임 상태

서버의 `GameState`를 전달받아 클라이언트 화면을 변경한다.

| 상태       | 클라이언트 처리                      |
|----------|-------------------------------|
| Waiting  | 대기 화면 표시                      |
| Starting | 게임 시작 준비 화면 표시                |
| Playing  | 게임 플레이 화면 표시                  |
| Ending   | 게임 플레이 <b>UI</b> 비활성화 및 종료 연출 |

|        |               |
|--------|---------------|
| Result | 결과 화면 표시      |
| Closed | 세션 종료 및 로비 이동 |

## 상태 흐름

```
Waiting
  ↓
Starting
  ↓
Playing
  ↓
Ending
  ↓
Result
  ↓
```

Closed

## 4. 게임 시작 처리

게임 진행 시스템으로부터 `Starting` 상태를 전달받으면 게임 시작 준비를 진행한다.

### 처리 순서

```
서버 GameState = Starting
  ↓
게임 시작 연출 출력
  ↓
역할 배정 완료 대기
  ↓
미션 배정 완료 대기
  ↓
서버 GameState = Playing
  ↓
```

게임 플레이 UI 활성화

클라이언트는 역할이나 미션을 직접 배정하지 않고, 서버에서 배정된 결과를 전달받아 UI에 표시한다.

## 5. 게임 시작 UI

게임 시작 시 다음 정보를 표시한다.

| UI       | 내용       |
|----------|----------|
| 게임 시작 알림 | 게임 시작 안내 |

|        |                  |
|--------|------------------|
| 역할 안내  | 자신에게 배정된 역할 표시   |
| 미션 안내  | 자신에게 배정된 미션 표시   |
| 게임 타이머 | 남은 게임 시간         |
| 게임 HUD | 게임 진행에 필요한 기본 UI |

## 예시

게임 시작  
↓  
당신의 역할은...  
↓  
[ 시민 ]  
↓  
개인 미션 확인  
↓  
개인 행동 목표 확인  
↓

게임 시작

※ 실제 역할 공개 범위와 미션 공개 범위는 역할/미션 시스템의 규칙을 따른다.

## 6. 게임 시간 표시

서버에서 전달받은 게임 시간을 기반으로 남은 시간을 표시한다.

### 표시 정보

남은 시간

14:59

- 서버 시간을 기준으로 표시한다.
- 클라이언트 로컬 시간은 게임 종료 판정에 사용하지 않는다.
- 서버와 클라이언트의 시간 차이가 발생할 경우 서버 값을 기준으로 보정한다.
- 시간이 0이 되면 게임 종료 시스템에서 전달한 상태에 따라 종료 처리를 진행한다.

## 7. 게임 진행 UI

게임 플레이 중 화면에 현재 게임 진행에 필요한 정보를 표시한다.

### 기본 UI

- 남은 게임 시간
- 체력
- 스테미나
- 미션 진행도
- 개인 미션
- 개인 행동 목표
- 인벤토리
- 상호작용 UI
- 음성 상태
- 플레이어 상태

각 UI의 실제 데이터는 해당 시스템에서 전달받는다.

예:

MissionManager



미션 진행도



GameProgress UI

게임 진행 시스템은 미션 자체를 관리하지 않고, 게임 전체 진행 상태에 필요한 정보만 표시한다.

## 8. 플레이어 상태 반영

서버에서 플레이어 상태가 변경되면 클라이언트의 상태를 변경한다.

| 서버 상태   | 클라이언트 처리          |
|---------|-------------------|
| Alive   | 일반 게임 화면          |
| Dead    | 사망 연출 → 관전자 화면    |
| Escaped | 탈출 연출 → 생존자 관전 화면 |
| Ending  | 게임 플레이 제한         |
| Result  | 결과 화면             |

## 9. 사망 상태 반영

DeathManager에서 사망 결과를 전달받는다.



## 처리 순서

서버 사망 판정

↓  
클라이언트 사망 이벤트 수신

↓  
사망 연출

↓  
플레이어 조작 제한

↓

관전자 화면 전환

게임 진행 시스템은 사망을 판정하지 않으며, 서버에서 전달된 사망 상태를 화면에 반영한다.

## 10. 탈출 상태 반영

`EscapeManager`에서 탈출 성공 결과를 전달받는다.

### 처리 순서

탈출 성공

↓  
`Escape` 상태 수신

↓  
탈출 연출 출력

↓  
플레이어 조작 제한

↓

생존자 관전 화면 전환

### 탈출 상태 UI

- 탈출 성공 알림
- 플레이어 조작 비활성화
- 생존자 관전 화면 활성화
- 자신의 게임 플레이 UI 비활성화

탈출 가능 여부 및 탈출 성공 여부는 클라이언트에서 판단하지 않는다.

## 11. 게임 종료 상태

게임 진행 시스템 또는 게임 종료 시스템에서 `Ending` 상태를 전달받으면 게임 플레이를 종료한다.

### 처리

`GameState = Ending`

↓  
 게임 플레이 UI 비활성화  
 ↓  
 이동 제한  
 ↓  
 공격 제한  
 ↓  
 상호작용 제한  
 ↓  
 게임 종료 연출  
 ↓

Result 상태 전환

## 12. 게임 종료 중 행동 제한

| 행동     | 가능 여부         |
|--------|---------------|
| 이동     | X             |
| 공격     | X             |
| 미션 수행  | X             |
| 상호작용   | X             |
| 아이템 사용 | X             |
| 탈출     | X             |
| 음성     | 시스템 정책에 따라 종료 |
| 결과 확인  | O             |

클라이언트에서 먼저 입력을 막지만, 서버에서도 종료 상태의 요청을 최종적으로 거부한다.

## 13. 결과 화면 전환

게임 종료 시스템에서 `Result` 상태를 전달받으면 결과 화면을 활성화한다.

처리 순서

GameEndManager  
 ↓

```
GameState = Result
↓
SyncGameResult()
↓
ResultManager
↓
```

결과 화면 출력

## 결과 화면 표시 정보

- 승리 / 패배
- 자신의 역할
- 생존 / 사망 / 탈출 상태
- 공동 미션 결과
- 개인 미션 결과
- 개인 행동 목표 결과
- 게임 종료 원인

## 14. 게임 종료 연출

게임 종료 시 서버에서 전달받은 `GameEndReason`에 따라 적절한 연출을 출력한다.

| 종료 원인             | 표시            |
|-------------------|---------------|
| CitizenMission    | 시민 미션 달성      |
| CitizenEscape     | 시민 탈출         |
| KillerWin         | 살인마 승리        |
| TimeLimit         | 제한 시간 종료      |
| NoWinnerAvailable | 승리 가능 플레이어 없음 |
| ForceEnd          | 게임 강제 종료      |

## 15. Network Variable

클라이언트는 서버에서 동기화된 값을 읽어 UI 및 게임 상태를 변경한다.

### 게임 상태

- `GameState`

- `GameTime`
- `RemainingTime`
- `IsGameStarted`
- `IsGameEnding`

## 플레이어 상태

- `PlayerGameState`
- `IsAlive`
- `IsEscaped`

## 결과

게임 종료 시스템에서 전달:

- `GameEndReason`
- `WinnerTeam`
- `WinnerPlayers`
- `IsWinner`

---

## 16. RPC / 서버 이벤트

### 서버 → 클라이언트

- `StartGame()`
- `SyncGameState()`
- `SyncGameTime()`
- `NotifyPlayerStateChanged()`
- `NotifyGameEnding()`
- `ShowResult()`

### 클라이언트 → 서버

게임 진행 시스템 자체에서 별도의 판정용 **RPC**는 최소화한다.

필요한 경우:

- `RequestGameStateSync()`

정도로 구성한다.

게임 종료 요청이나 승리 판정은 클라이언트가 직접 수행하지 않는다.

---

## 17. 클라이언트 클래스

## 핵심 클래스

- `GameProgressUIManager`
- `GameStateUIController`
- `GameTimerUI`
- `GameStartUI`
- `GameProgressUI`

## 연동 클래스

- `MissionUIManager`
- `EscapeUIManager`
- `DeathUIManager`
- `SpectatorManager`
- `GameEndManager`
- `ResultManager`

## 18. 예외 처리

| 상황                                      | 클라이언트 처리               |
|-----------------------------------------|------------------------|
| 서버와 게임 상태 불일치                           | 서버 상태로 갱신              |
| 게임 시작 전 플레이어 조작                         | 게임 플레이 입력 제한           |
| 게임 종료 후 입력                              | 게임 플레이 입력 무시           |
| 게임 시간 동기화 실패                            | 마지막 서버 시간 유지 후 재동기화    |
| 게임 중 <b>Disconnect</b>                  | 연결 종료 <b>UI</b> 표시     |
| 재접속                                     | 서버의 현재 게임 상태를 받아 화면 복구 |
| 탈출/사망 상태에서 일반 게임 <b>UI</b> 입력           | 입력 제한                  |
| 결과 데이터 수신 실패                            | 재수신 요청 또는 대기           |
| 중복 게임 종료 이벤트                            | 최초 종료 상태 유지            |
| 이미 <b>Result</b> 상태에서 <b>Playing</b> 수신 | 서버 상태 확인 후 정상 상태로 동기화  |

## 19. 서버 / 클라이언트 역할 분리

| 클라이언트       | 서버          |
|-------------|-------------|
| 게임 시작 UI 표시 | 게임 시작 판정    |
| 게임 상태 표시    | 게임 상태 관리    |
| 게임 시간 표시    | 게임 시간 관리    |
| 역할 배정 결과 표시 | 역할 배정       |
| 미션 배정 결과 표시 | 미션 배정       |
| 플레이어 상태 표시  | 플레이어 상태 관리  |
| 탈출 상태 표시    | 탈출 결과 관리    |
| 사망 상태 표시    | 사망 결과 관리    |
| 게임 종료 연출    | 게임 종료 조건 검사 |
| 결과 화면 표시    | 최종 승패 판정    |
| 입력 제한       | 행동 요청 최종 검증 |

# 게임종료시스템

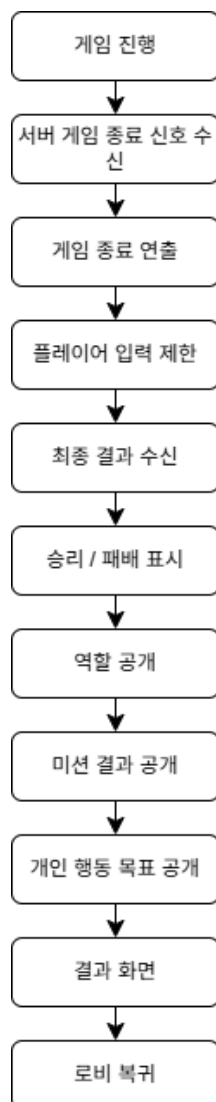
### 시스템 목적

- 게임 종료 시스템은 서버로부터 전달받은 게임 종료 상태와 최종 승패 결과를 클라이언트에 표시하고, 게임 종료에 따른 **UI** 및 연출을 처리하는 시스템
- 클라이언트는 승패를 직접 판정하지 않으며, 서버가 확정된 결과를 수신하여 화면에 반영한다

### 시스템 역할

- 게임 종료 상태 표시
- 게임 종료 연출
- 플레이어 행동 제한
- 승리 / 패배 **UI** 표시
- 탈출 승리 **UI** 표시
- 역할 공개
- 미션 결과 공개
- 개인 행동 목표 결과 공개
- 최종 결과 화면 출력
- 로비 복귀처리

### 시스템 구조





## 게임 상태별 클라이언트 처리

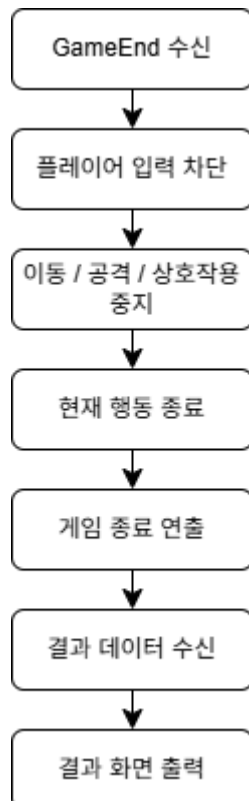
| 상태      | 클라이언트         |
|---------|---------------|
| Waiting | 대기 UI         |
| Playing | 일반 게임 UI      |
| Ending  | 입력 제한 및 종료 연출 |
| Result  | 결과 화면         |
| Closed  | 로비 이동         |

## 서버 결과 수신

- 클라이언트는 서버로부터 다음 정보를 전달받는다
  - 게임 종료 여부
  - 게임 종료 원인
  - 자신의 승리 여부
  - 다른 플레이어의 승리 여부
  - 역할
  - 탈출 여부
  - 공동 미션 결과
  - 개인 미션 결과
  - 개인 행동 목표 결과

## 게임 종료 연출

- 서버에서 **GameEnd** 이벤트를 수신하면 게임 플레이를 종료한다



## 플레이어 입력 제한

| 행동       | 가능 여부 |
|----------|-------|
| 이동       | X     |
| 달리기      | X     |
| 점프       | X     |
| 공격       | X     |
| 상호작용     | X     |
| 아이템 사용   | X     |
| 미션 수행    | X     |
| 탈출       | X     |
| 결과 화면 조작 | O     |

## 게임 종료 원인 표시

| 종료 원인             | 표시 예시                 |
|-------------------|-----------------------|
| CitizenMssion     | 시민 승리                 |
| CitizenEscape     | 탈출 성공                 |
| KillerWin         | 살인마 승리                |
| TimeLimit         | 제한 시간 종료              |
| NoWinnerAvailable | 더이상 승리 가능한 플레이어가 없습니다 |
| ForceEnd          | 게임 종료                 |

## 승리 / 패배 화면

- 자신의 결과를 표시한다

### 승리(GameClear)

- 승리했습니다

#### 승리 조건

공동 미션(완료)

개인 미션(완료)

### 탈출 승리(Escaped)

- 탈출에 성공했습니다

당신은 생존에 성공했습니다

### 패배(GameOver)

- 패배했습니다

승리 조건을 달성하지 못했습니다

#### 역할 공개

| 플레이어     | 역할  | 결과 |
|----------|-----|----|
| Player A | 시민  | 승리 |
| Player B | 시민  | 패배 |
| Player C | 살인마 | 패배 |
| Player D | 사만  | 승리 |

#### 미션 결과 공개

##### 공동 미션

- 공동 미션

발전기 수리 (완료)  
 보안 시스템 복구 (완료)  
 서버 재부팅 (완료)  
 통신 장비 복구 (미완료)

- 개인 미션

##### Player A

발전기 2회 수리  
 성공

##### Player B

의료 키드 사용  
 실패

#### 개인 행동 목표 공개

| 플레이어     | 개인 행동 목표            | 결과 |
|----------|---------------------|----|
| Player A | 아무에게도 회복 아이템을 주지 않기 | 성공 |
| Player B | 한 번도 달리지 않기         | 실패 |
| Player C | 항상 다른 사람과 함께 이동하기   | 성공 |

#### 결과 화면 구성

##### 상단

- Game Result

중앙

- 개인 승리 / 패배
- 역할
- 승리 방식

하단

- 공동 미션 결과
- 개인 미션 결과
- 개인 행동 목표 결과

추가 버튼

- 상세 결과
- 로비로 돌아가기

결과 상세 화면

[플레이어 결과]

**Player A**

시민

탈출 성공

개인 미션

발전기 수리 2회

성공

개인 행동 목표

회복 아이템을 주지 않기

성공

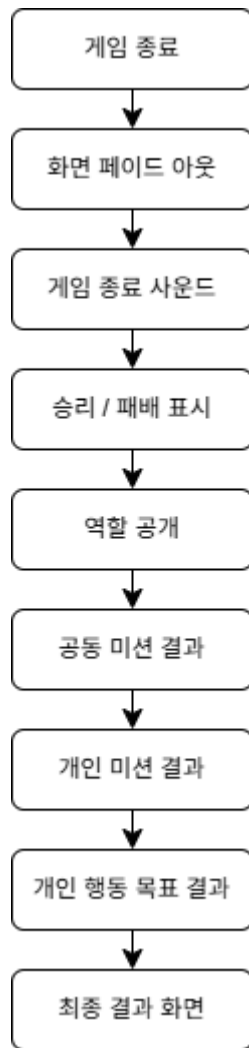
탈출 플레이어 처리

클라이언트는

- 게임 플레이 입력 차단
- 일반 **HUD** 제거
- 생존자 관전 화면 전환
- 탈출 상태 표시
- 결과 확정 대기

를 처리함

결과 공개 연출



#### Network Variable

- GameState
- IsGameEnded
- GameEndReason
- IsWinner
- WinnerTeam
- IsEscaped

#### RPC / Event

- GameEnd()
- SyncGameResult()
- RevealRole()
- RevealMission()
- RevealPersonalGoal()
- ShowResult()
- ReturnToLobby()

#### 필요한 클래스

GameEndUIManager

- └─ GameEndController
- └─ ResultUIManager
- └─ RoleRevealUI
- └─ MissionResultUI
- └─ PersonalGoalResultUI
- └─ GameEndEffectController

#### 예외처리

게임 종료 중복 수신 -> 최초 종료 처리만 실행

게임 종료 후 입력 -> 입력 무시

결과 데이터 수신 지연 -> 결과 로딩 UI 표시

결과 데이터 누락 -> 서버에 결과 재요청

**Disconnect** 후 재접속 -> 서버의 최종 결과를 다시 수신

이미 탈출하여 게임을 나간 플레이어 -> 탈출 결과 유지

게임 종료 직전 사망 -> 서버가 확정된 상태를 표시

게임 종료 직전 탈출 -> 서버가 확정된 상태를 표시

결과 화면에서 뒤로가기 -> 게임 플레이로 돌아갈 수 없음

서버 연결 종료 -> 재접속 또는 연결 종료 안내

#### 서버 / 클라이언트 역할 분리

| 클라이언트       | 서버             |
|-------------|----------------|
| 종료 UI 표시    | 종료 전 판정        |
| 승리 / 패배 연출  | 승리 / 패배 판정     |
| 역할 공개 UI    | 역할 데이터 관리      |
| 미션 결과 UI    | 미션 결과 확정       |
| 개인 행동 목표 UI | 행동 목표 성공 여부 판정 |
| 결과 화면       | 최종 결과 데이터 생성   |
| 로비 이동 UI    | 게임 세션 종료       |

# 탈출시스템

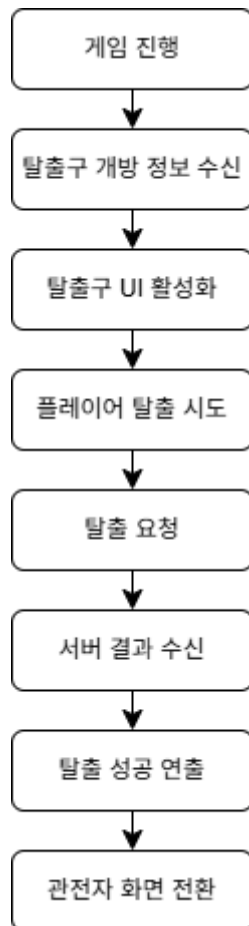
#### 시스템 목적

- 탈출 시스템은 게임 후반부에 탈출구를 개방하여 시민에게 또 다른 승리 수단을 제공하는 시스템
- 클라이언트는 탈출구 상태, 탈출 가능 여부, 탈출 성공 결과를 화면에 표시하며, 서버의 판정 결과를 반영한다

#### 시스템 역할

- 탈출구 UI 표시
- 탈출 가능 여부 표시
- 탈출 상호작용 처리
- 탈출 연출 재생
- 탈출 플레이어 UI 변경
- 관전자 화면 전환
- 게임 종료 UI 출력

#### 시스템 구조



#### 클라이언트 처리

| 기능      | 내용           |
|---------|--------------|
| 탈출구 표시  | 탈출구 활성화 및 안내 |
| 탈출 상호작용 | 상호작용 키 입력 처리 |



|          |                |
|----------|----------------|
| 탈출 애니메이션 | 탈출 성공 연출       |
| UI 변경    | 탈출 완료 UI 표시    |
| 관전자 화면   | 생존자 관전 화면으로 전환 |

#### 탈출구 표시

- 탈출구 오브젝트 활성화
- 탈출구 이펙트 출력
- 미니맵 아이콘 표시
- 화면 알림 출력

#### 탈출 가능 여부

|          |              |
|----------|--------------|
| 조건       | UI           |
| 탈출 가능    | 상호작용 안내 표시   |
| 탈출 불가    | 안내 메시지 표시    |
| 탈출 인원 초과 | 탈출 불가 메시지 표시 |

#### 탈출 처리 순서

|    |           |
|----|-----------|
| 순서 | 처리        |
| 1  | 탈출구 접근    |
| 2  | 상호작용 입력   |
| 3  | 탈출 요청     |
| 4  | 서버 결과 수신  |
| 5  | 탈출 성공 연출  |
| 6  | HUD 변경    |
| 7  | 관전자 화면 전환 |

#### 탈출 성공 연출

- 화면 페이드 아웃
- 탈출 성공 메시지 출력
- 캐릭터 입력 비활성화
- 탈출 HUD 표시
- 관전자 화면으로 전환

#### 탈출 플레이어 UI

- 표시
  - 탈출 성공
  - 승리
  - 현재 관전 중인 플레이어

- 관전 대상 변경 안내
- 숨김
  - 공동 미션 UI
  - 개인 미션 UI
  - 상호작용 UI
  - 아이템 UI

#### 관전자 화면

- 생존자만 관전 가능
- 자유 이동 불가
- 플레이어 변경 가능
- 관전자 전용 UI 표시

#### 상태 변화

| 상태    | UI     |
|-------|--------|
| 생존    | 일반 HUD |
| 탈출    | 탈출 HUD |
| 게임 종료 | 결과 화면  |

#### UI 구성

- 화면 알림
  - 탈출구 개방
  - 탈출 성공
  - 탈출 실패
- 미니맵
  - 탈출구 위치 표시
  - 탈출구 아이콘 표시
- HUD
  - 탈출 가능 여부
  - 탈출 남은 인원

#### 필요한 클래스

- EscapeUIController
- EscapeInteractionController
- EscapeAnimationController
- EscapeHUDController
- EscapeNotificationController
- EscapeSpectatorController

#### 필요한 이벤트

- OnEscapeOpened
- OnEscapeRequested
- OnEscapeSuccess
- OnEscapeFailed

- OnEscapeCapacityChanged
- OnEnterEscapeSpectator

예외 처리

탈출구 미개방 -> 상호작용 UI 표시하지 않음

탈출 인원 초과 -> “탈출 가능한 인원이 없습니다” 메시지 출력(플레이어 화면에)

탈출 실패 -> 실패 안내 메시지 출력(플레이어 화면에 콘솔창X)

탈출 성공 후 입력 -> 모든 플레이 입력 비활성화

탈출 후 재접속 -> 탈출 HUD 및 관전자 화면 복원

게임 종료 -> 결과 화면으로 전환

서버 / 클라 분리

| 클라          | 서버          |
|-------------|-------------|
| 탈출구 UI 표시   | 탈출구 개방      |
| 상호작용 입력     | 탈출 가능 여부 판정 |
| 탈출 애니메이션    | 탈출 성공 판정    |
| 탈출 HUD 표시   | 탈출 가능 인원 관리 |
| 관전자 화면 전환   | 플레이어 상태 변경  |
| 알림 및 이펙트 출력 | 게임 종료 조건 검사 |

# 역할시스템

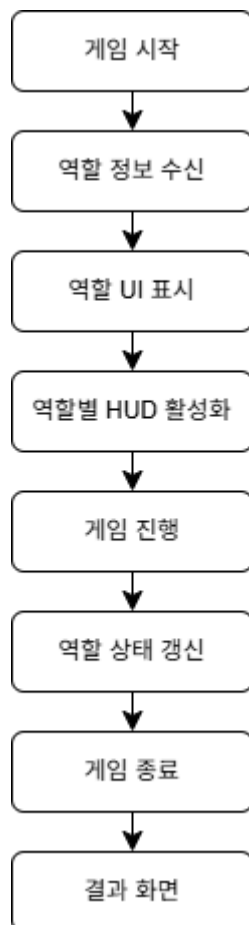
### 시스템 목적

- 역할 시스템은 게임 시작 시 플레이어에게 역할을 부여하고, 각 역할의 승리 조건을 관리하는 시스템이다
- 클라이언트는 역할 정보를 바탕으로 UI를 표시하며, 역할별 행동 가능 여부를 사용자에게 안내한다

### 시스템 역할

- 역할 UI 표시
- 역할 설명 출력
- 역할별 HUD 변경
- 역할 상태 표시
- 역할 변경 연출
- 게임 종료 결과 표시

### 시스템 구조



### 역할 종류

| 역할  | 설명                             |
|-----|--------------------------------|
| 시민  | 공동 미션 및 개인 미션 수행, 또는 탈출을 통해 승리 |
| 살인마 | 시민 제거하여 승리                     |

## 역할 UI

- 시민
  - 역할 이름
  - 역할 설명
  - 공동 미션 안내
  - 개인 미션 안내
  - 개인 행동 목표 안내
- 살인마
  - 역할 이름
  - 역할 설명
  - 살인마 미션 안내
  - 살인 가능 조건 안내

## 역할 표시

| 상황    | 표시        |
|-------|-----------|
| 게임 시작 | 역할 화면 출력  |
| 게임 진행 | 역할 아이콘 표시 |
| 게임 종료 | 역할 공개     |

## 역할별 HUD

### 시민

- 공동 미션 & 개인 미션 진행도
- 개인 미션
- 개인 행동 목표
- 탈출 가능 여부

### 살인마

- 시민 미션 진행도
- 살인마 미션
- 살인 가능 여부
- 살인 쿨타임

## 역할 상태

| 상태 | UI        |
|----|-----------|
| 생존 | 일반 HUD    |
| 사망 | 관전자 HUD   |
| 탈출 | 탈출 완료 HUD |

## 게임 진행 중 UI

### 시민

- 공동 미션 & 개인 미션 진행도
- 개인 행동 목표
- 탈출 안내

#### 살인마

- 시민 미션 진행도
- 살인마 가능 상태
- 쿨타임 표시

#### 결과 화면(게임 종료시)

- 자신의 역할
- 승리 / 패배
- 공동 미션 & 개인 미션 결과
- 개인 행동 결과

#### 처리 순서

| 순서 | 처리               |
|----|------------------|
| 1  | 역할 정보 수신         |
| 2  | 역할 UI 출력         |
| 3  | 역할별 HUD 활성화      |
| 4  | 역할 상태 변경 시 UI 갱신 |
| 5  | 게임 종료 시 결과 화면 출력 |

#### 필요한 클래스

- RoleUIController
- RoleHUDController
- RoleInfoController
- ResultUIController
- PlayerStatusUI
- MissionUIController

#### 필요한 이벤트

- OnRoleAssigned
- OnRoleUpdated
- OnPlayerDead
- OnPlayerEscaped
- OnRoleChanged
- OnGameEnd
- OnResultReveal

#### 예외처리

역할 정보 수신 전 -> 역할 UI 비표시

사망 -> 관전자 HUD로 변경

탈출 -> 탈출 완료 UI 표시

게임 종료 -> 결과 화면 출력

역할 정보 재수신 -> UI 갱신

재접속 -> 현재 역할 및 상태 UI 복원

클라 / 서버 분리

| 클라            | 서버       |
|---------------|----------|
| 역할 UI 표시      | 역할 배정    |
| 역할 설명 표시      | 역할 정보 관리 |
| 역할별 HUD 변경    | 승리 조건 판정 |
| 역할 상태 UI 갱신   | 패배 조건 판정 |
| 결과 화면 표시      | 게임 종료 판정 |
| 미션 및 탈출 정보 표시 | 탈출 여부 판정 |



# 살인시스템

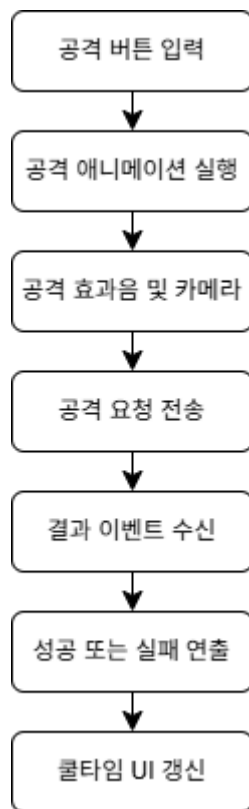
#### 시스템 목적

- 범인이 특정 조건을 만족한 후 시민을 공격하여 제거하는 시스템
- 살인은 서버가 최종 판정을 담당하여 모든 클라이언트는 결과만 전달받는다

#### 시스템 역할

- 공격 입력 처리
- 공격 애니메이션 재생
- 공격 효과음 출력
- 카메라 연출
- 쿨타임 UI 표시
- 공격 가능 여부 표시

#### 시스템 구조



#### 공격 입력

| 입력    | 기능    |
|-------|-------|
| 공격 버튼 | 공격 시도 |

#### 공격 연출

- 공격 애니메이션 재생
- 공격 효과음 재생
- 카메라 흔들림
- 공격 모션 종료까지 입력 제한

#### 공격 성공 연출

- 공격 성공 효과음
- 피격 효과 출력
- 대상 사망 애니메이션 재생
- 화면 연출 출력
- 공격 쿨타임 UI 시작

#### 공격 실패 연출

- 실패 효과음
- 실패 UI 표시
- 공격 애니메이션 종료

#### 공격 가능 표시

| 상태    | 표시               |
|-------|------------------|
| 공격 가능 | 공격 버튼 활성화        |
| 쿨타임   | 버튼 비활성화 및 쿨타임 표시 |
| 공격 불가 | 버튼 비활성화          |

#### 쿨타임 UI

- 공격 성공 후 쿨타임 게이지 표시
- 남은 시간을 원형 또는 막대 형태로 표시
- 쿨타임 종료 시 공격 가능 상태로 변경

#### 처리 순서



## UI 구성

- 공격 버튼
- 쿨타임 게이지
- 공격 가능 표시
- 실패 메시지

## 예외 처리

- 공격 애니메이션 중 추가 입력 -> 입력 무시
- 쿨타임 중 공격 입력 -> 입력 무시 및 쿨타임 UI 유지
- 공격 실패(헛스윙) -> 아무런 일도 일어나지 않음
- 대상 사망 연출 중 -> 추가 공격 연출 실행 안함
- 게임 종료 -> 공격 UI 비활성화
- 플레이어 사망 -> 공격 UI 비활성화

## 서버 / 클라 분리

| 클라           | 서버          |
|--------------|-------------|
| 공격 입력 처리     | 살인 가능 여부 판정 |
| 공격 애니메이션     | 공격 거리 검사    |
| 공격 효과음       | 대상 생존 여부 확인 |
| 카메라 흔들림      | 살인 성공/실패 판정 |
| 공격 성공/실패 연출  | 사망 처리       |
| 쿨타임 UI 표시    | 쿨타임 관리      |
| 공격 버튼 활성화 표시 | 증거 및 시체 생성  |

## 칼 획득

- 플레이어가 바닥에 떨어진 칼의 상호작용 범위에 들어오면 칼 획득 UI를 표시한다
- 칼과 플레이어 거리 확인
- 서버에서 동기화된 **IsKnifeTaken** 상태 확인
- 획득 가능한 경우 상호작용 UI 표시
- 플레이어가 E 입력
- **RequestPickupKnife()** 호출

## 칼 획득 연출

- 바닥의 칼 오브젝트 제거
- 플레이어 손에 칼 모델 표시
- 칼 소지 모습으로 변환
- 획득 사운드 출력
- 상호작용 UI 제거
- **HasKnife = true** 반영

## 칼 소지 상태

- 칼 모델 활성화

- 손 **Bone**에 칼 부착
- 칼 소지 애니메이션 적용
- 칼 사용 가능 **UI** 표시
- 시민도 칼을 소지할 수 있으므로 칼을 들고 있다고 해서 공격 버튼이 자동으로 활성화 되면 안된다(살인마 확킬 미션 후 열리는 공격 버튼)
- 서버에서 전달받은 **Cankill** 상태에 따라 공격 가능 여부를 표시한다

#### 공격 입력

- 좌클릭
- ↓
- 공격 가능 상태 확인
- ↓
- **RequestKill()**
- ↓
- 공격 애니메이션 시작
- ↓
- 서버 결과 대기
- 클라이언트에서는 공격 입력 직후 살인 성공을 확정하지 않는다

#### 공격 애니메이션

- 애니메이션 흐름
  - **KnifeHold**
  - ↓
  - **KillStart**
  - ↓
  - **KnifeStab**
  - ↓
  - **KillEnd**
  - ↓
  - **KnifeHold**

#### 찌르기 연출

- 공격자
  - 찌르기 애니메이션과 함께
  - 칼 휘두름 / 찌르기 효과
  - 칼 이동 효과
  - 공격 효과음
  - 카메라 흔들림
  - 공격 모션
- 피격자
  - 서버에서 **KillSuccess()**를 전달받은 경우
  - 피격 애니메이션
  - 피격 효과
  - 혈흔 효과
  - 피격 사운드
  - 사망 애니메이션

#### 살인 성공 연출

- **KillSuccess()**
- ↓
- 피격 효과
- ↓
- 피격 사운드
- ↓
- **Death Animation**
- ↓
- 카메라 연출
- ↓
- 시체 상태
- ↓
- **Blood / Evidence** 표시
- ↓
- 살인 쿨타임 **UI** 시작

#### 공격자 화면

- 찢르기 애니메이션 완료
- 카메라 흔들림
- 피 효과
- 살인 성공 사운드
- 쿨타임 **UI** 활성화

#### 피해자 화면

- 피격
- 화면 흔들림
- 사망 애니메이션
- 사망 상태 전환
- 사망 후 관전자 화면 전환

#### 주변 플레이어

- 살인/피격 애니메이션 동기화
- 시체 생성
- 관련 사운드 출력

#### 살인 실패 연출

- 서버에서 **KillFailed()**가 전달되면 살인 실패 연출을 출력한다

#### 연출

- 공격 애니메이션 종료
- 대상에게 피해 적용 없음
- 사망 처리 없음
- 시체 생성 없음

#### 카메라 연출

##### 공격 성공

- 짧은 카메라 흔들림
- 공격 방향 중심으로 시점 유지
- 과도한 화면 전환은 사용하지 않음

## 피격

- 짧은 화면 흔들림
- 피격 방향에 딸느 시각 효과

## 사망

- 사망 애니메이션에 맞춰 카메라 연출
- 사망 후 관전자 카메라로 전환

## 사운드

- 공격
  - 칼 꺼내는 소리
  - 찌르기 효과음
- 피격
  - 피격음
- 사망
  - 사망 효과음
- 실패
  - 공격 실패 효과음

# 로비/방시시스템



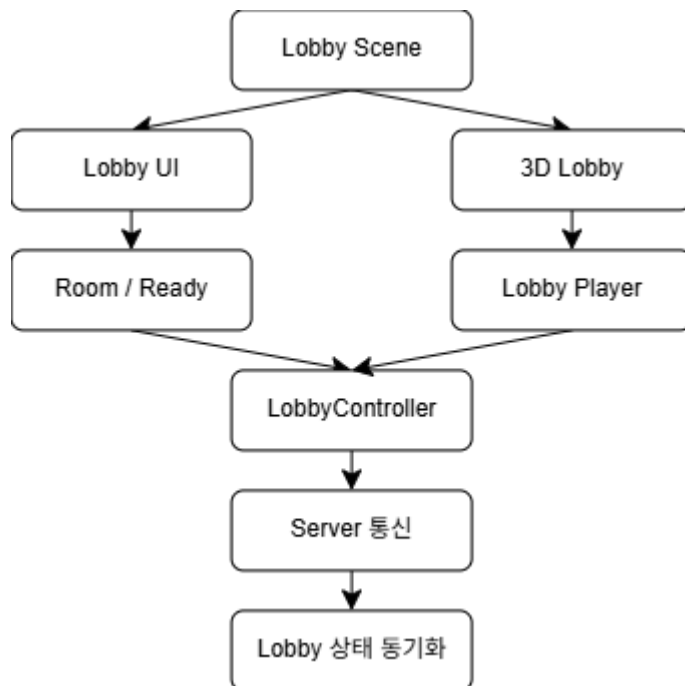
### 시스템 목적

- 로비 시스템은 플레이어가 게임 시작 전에 방을 생성하거나 다른 방에 참가하고, 함께 플레이할 플레이어를 확인한 뒤 게임을 시작할 수 있도록 하는 클라이언트 시스템이다
- 클라이언트는 로비 **UI**, 버튼 입력, 플레이어 목록 표시, 준비 상태 표시 등의 화면 처리를 담당한다
- 방 생성 가능 여부, 방 참가 가능 여부, 방장 권한, 게임 시작 가능 여부 등 최종 판정은 서버가 담당한다

### 시스템 역할

- 로비 씬 관리
- **3D** 로비 환경 출력
- 로비 플레이어 **Spawn**
- 로비 플레이어 이동
- 다른 플레이어 캐릭터 표시
- 플레이어 닉네임 표시
- 방 목록 출력
- 방 생성 **UI**
- 방 참가 **UI**
- 플레이어 목록 표시
- 방장 표시
- 플레이어 머리 위 **Ready** 상태 표시
- 게임 설정 표시
- 게임 시작 버튼 처리
- 서버 로비 상태 동기화
- 게임 씬 전환
- 로비 나가기 처리

### 시스템 구조



## 로비 진입 흐름



## 3D 로비

- 게임 시작 전 별도의 3D 공간을 제공한다

### 이동

| 행동     | 입력    |
|--------|-------|
| 이동     | WASD  |
| 달리기    | Shift |
| 점프     | Space |
| 카메라 회전 | Mouse |
| 상호작용   | F     |

## 로비 플레이어

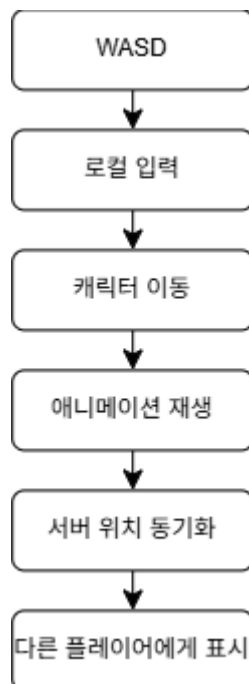
- 방에 참가한 플레이어는 **3D** 캐릭터로 생성된다
- 각 플레이어 캐릭터 위에는
- 닉네임
  - 방장 표시
  - **Ready** 상태
  - 음성 사용 상태

#### 플레이어 Spawn

- 방에 입장하면 서버에서 전달받은 **Spawn** 위치를 기준으로 캐릭터를 생성한다



#### 로비 이동



#### 클라이언트 처리

- 입력
- 캐릭터 이동 표현
- 이동 애니메이션

- 달리기 애니메이션
- 점프 애니메이션
- 발소리
- 카메라

로비 플레이어 애니메이션

- 상태에 따라 애니메이션을 변경한다

| 상태    | 애니메이션           |
|-------|-----------------|
| 대기    | Idle            |
| 걸기    | Walk            |
| 달리기   | Run             |
| 점프    | Jump            |
| Ready | Idle / Ready 표시 |
| 게임 시작 | Start 연출        |

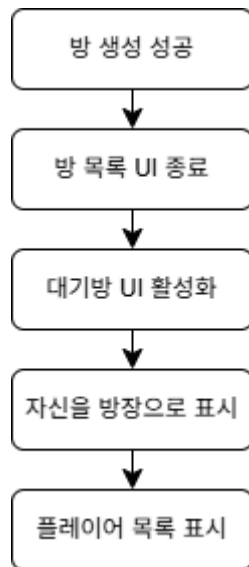
방 생성 처리



방 생성 성공

서버에선 **CreateRoomSuccess()** 를 받으면

- 방장도 일반 플레이어와 동일하게 **Ready**를 눌러야함



#### 방 참가

- 방 목록에서 원하는 방을 선택하거나 초대받은 방에 참가한다



#### 참가 실패

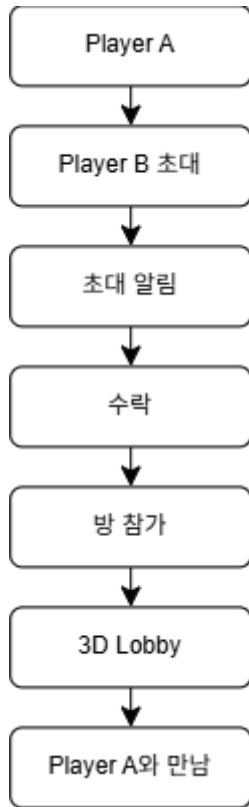
- 방이 가득 참
- 방이 존재하지 않음
- 게임 시작됨
- 접속 오류

#### 실패 메시지

| 서버 결과        | UI             |
|--------------|----------------|
| RoomFull     | 방이 가득 찼습니다     |
| RoomNotFound | 존재하지 않는 방입니다   |
| GameStarted  | 이미 게임이 시작되었습니다 |

#### 초대 시스템 연동

- 초대 시스템에서 초대받아 방에 참가한 경우에도 동일하게 **3D** 로비로 진입한다
- 초대 시스템 자체는 별도 시스템으로 관리하고, 로비 시스템은 초대 성공 후 해당 방에 진입하는 부분만 처리한다



#### 로비 음성

- 로비에서는 같은 방의 플레이어끼리 음성으로 대화할 수 있다

#### 클라이언트 처리

- 마이크 입력
- 음성 데이터 송신
- 다른 플레이어 음성 재생
- 거리 기반 볼륨
- 음성 아이콘
- 음소거 UI
- 게임 본편의 음성 채널과는 별도의 로비 음성 채널로 구분한다

#### 로비 음성 거리

- 이를 통해 플레이어가 실제로 가까이 모여 대화하는 느낌을 줄 수 있다

| 거리   | 볼륨   |
|------|------|
| 0~3m | 100% |
| 3~7m | 70%  |

|        |     |
|--------|-----|
| 7~12m  | 40% |
| 12~15m | 10% |
| 15m 이상 | 0%  |

- 계단 식으로 적어놨지만 자연스럽게 볼륨이 줄어듦과 커지는 느낌을 원함

#### 음성 상태 표시

- 플레이어가 말하고 있을 경우 캐릭터 위에 음성 표시를 출력한다
- 클라이언트는 서버에서 전달 받은 음성 상태를 UI로 표시한다

#### Ready 시스템

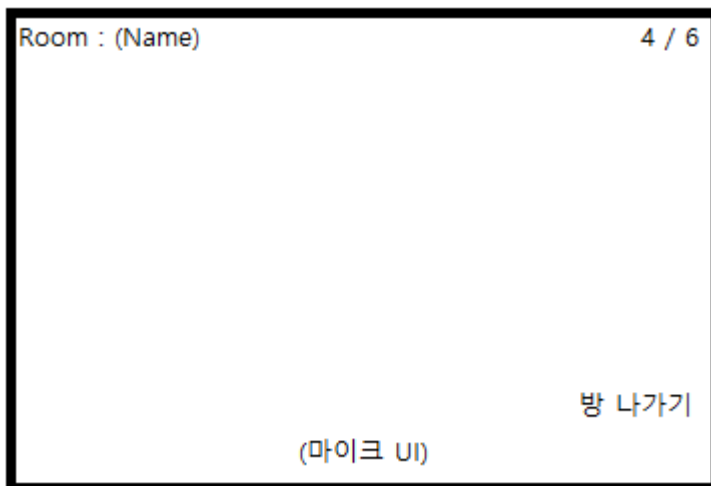
- 플레이어는 로비에서 **Ready** 버튼을 눌러 준비 상태를 변경한다
- **[Ready]** -> 서버 요청 -> 서버 승인 -> Ready 상태 -> 캐릭터 위 Ready 표시

#### Ready 상태에서도

- 이동 가능
- 점프 가능
- 음성 가능
- 다른 플레이어와 소통 가능
- **Ready**는 게임 시작 준비 상태일 뿐 플레이어의 조작을 제한하지 않는다

#### 로비 UI

- ESC로 상세 로비 UI를 열 수 있다



#### 상세 로비 UI

- 방장은 추가로 **[게임 시작]** 버튼을 표시한다



#### 방장 표시

- 방장은 캐릭터와 UI에서 구분한다
- Player A
- 방장이 변경되면 서버의 정보를 받아 즉시 UI를 갱신한다

#### 게임 시작

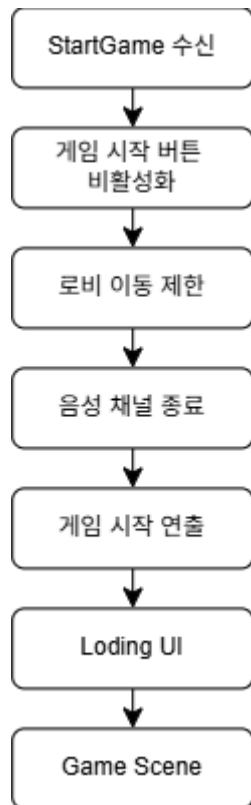
- 방장은 로비 안에서 게임 시작 버튼을 사용할 수 있다
- UI 방식 : [게임 시작]

#### 게임 시작 조건

- 클라이언트는 서버에서 전달받은 상태를 준으로 게임 시작 버튼을 활성화한다
- 예) 현재 인원 : 6, 준비 완료 : 6  
[게임 시작]
- 현재 인원 : 5, 준비 완료 : 4  
[게임 시작 불가]

#### 게임 시작 연출





예)

GAME STARTING . . .

3

2

1

씬 전환

Lobby Scene -> StartGame -> Loading Scene / Loading UI -> Game Scene

로비 나가기

- 플레이어가 [방 나가기]를 선택하면 확인창을 표시한다
- 정말 방을 나가시겠습니까?  
[확인]                      [취소]
- **LeaveRoom()** -> 서버 승인 -> **Lobby Player** 제거 -> 로비 메뉴로 이동

방장 나가기

- 방장이 나가면 서버가 새로운 방장을 지정한다
  - 클라이언트는 서버에서 변경된 방장 정보를 받아
    - 기존 방장 왕관 제거
    - 새 방장 왕관 표시
    - 새 방장에게 게임 시작 버튼 표시
- 를 표시함

#### 로비 상태 동기화

- 클라이언트는 서버 데이터를 기준으로 로비를 갱신한다
- 동기화 데이터
  - RoomID
  - RoomName
  - HostPlayerID
  - PlayerList
  - PlayerPosition
  - PlayerRotation
  - PlayerState
  - IsReady
  - VoiceState
  - RoomState

#### Network Variable

- RoomID
- RoomName
- HostPlayerID
- PlayerList
- LobbyPosition
- LobbyRotation
- PlayerState
- IsReady
- IsHost
- IsTalking
- RoomState

#### RPC / Event

- CreateRoomSuccess()
- CreateRoomFailed()
- JoinRoomSuccess()
- JoinRoomFailed()
- SpawnLobbyPlayer()
- DespawnLobbyPlayer()
- UpdateLobbyPosition()
- UpdateLobbyRotation()
- SyncPlayerList()
- SyncReadyState()
- SyncHost()
- JoinLobbyVoice()
- LeaveLobbyVoice()
- RequestStartGame()
- StartGame()
- LeaveRoom()
- PlayerKicked()

#### 필요한 클래스

LobbyManager

- └─ LobbyController
- └─ LobbyPlayerController
- └─ LobbyPlayerSpawner
- └─ LobbyMovementController
- └─ LobbyVoiceController
- └─ LobbyUIManager
- └─ RoomUI
- └─ PlayerListUI
- └─ ReadyUI
- └─ HostUI
- └─ LobbyLoadingUI

예외 처리

방 생성 실패 -> 실패 메시지

방 참가 실패 -> 실패 메시지

방이 가득 참 -> 참가 불가 표시

방장이 나감 -> 새 방장 표시

플레이어 나감 -> 캐릭터 제거

플레이어 접속 끊김 -> 캐릭터 제거 / 상태 갱신

**Ready** 상태 변경 -> UI 즉시 갱신

게임 시작 실패 -> 실패 사유 표시

게임 시작 중 **Disconnect** -> 로비/연결 상태 처리

서버 연결 끊김 -> 연결 끊김 UI

음성 연결 실패 -> 음성 연결 상태 표시

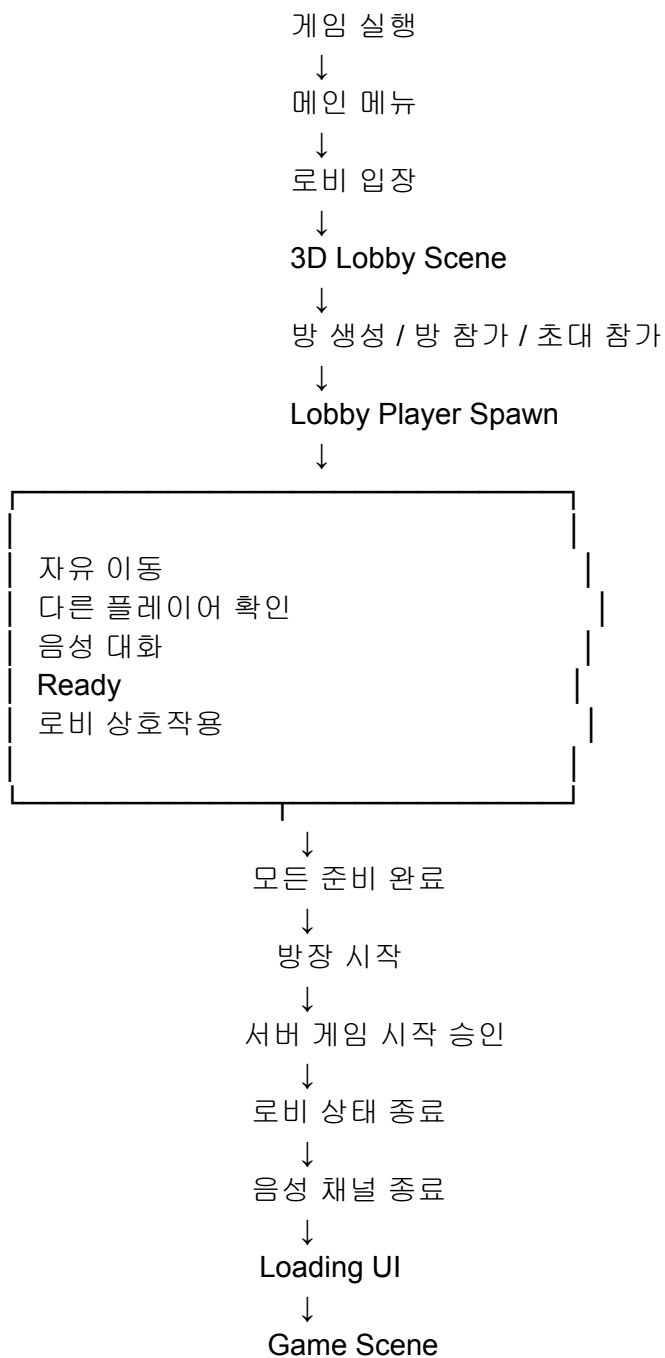
게임 시작 승인 -> 로비 입력 제한 후 씬 전환

서버 / 클라 역할 분리

| 기능          | 클라 | 서버 |
|-------------|----|----|
| 방 이름 입력     | O  |    |
| 방 생성 UI     | O  |    |
| 방 생성 판정     |    | O  |
| 방 참가 UI     | O  |    |
| 방 참가 판정     |    | O  |
| 3D 캐릭터 표시   | O  |    |
| 로비 이동 입력    | O  |    |
| 로비 위치 동기화   | O  | O  |
| Ready 버튼    | O  |    |
| Ready 상태 저장 |    | O  |
| 방장 표시       | O  | O  |

|            |   |   |
|------------|---|---|
| 음성 입력 / 재생 | O |   |
| 음성 채널 관리   |   | O |
| 게임 시작 버튼   | O |   |
| 게임 시작 판정   |   | O |
| 로딩 화면      | O |   |
| 게임 씬 전환 승인 |   | O |

전체 동작 흐름



# 초대시스템

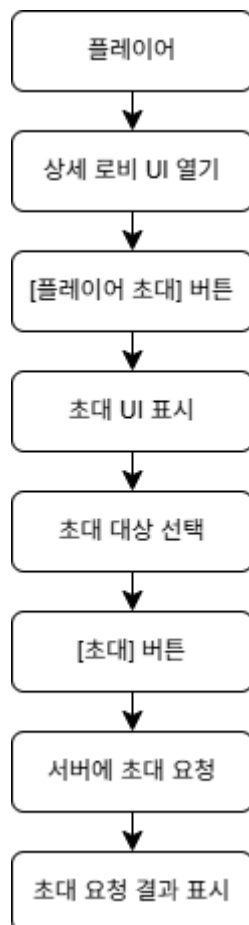
### 시스템 목적

- 초대 시스템은 플레이어가 다른 플레이어를 현재 참가 중인 방으로 초대하거나, 다른 플레이어에게 받은 초대를 확인하고 수락 또는 거절할 수 있도록 하는 클라이언트 시스템이다
- 초대 대상 선택, 초대 **UI** 출력, 초대 요청 입력, 수락 및 거절 입력, 초대 결과 표시를 담당한다

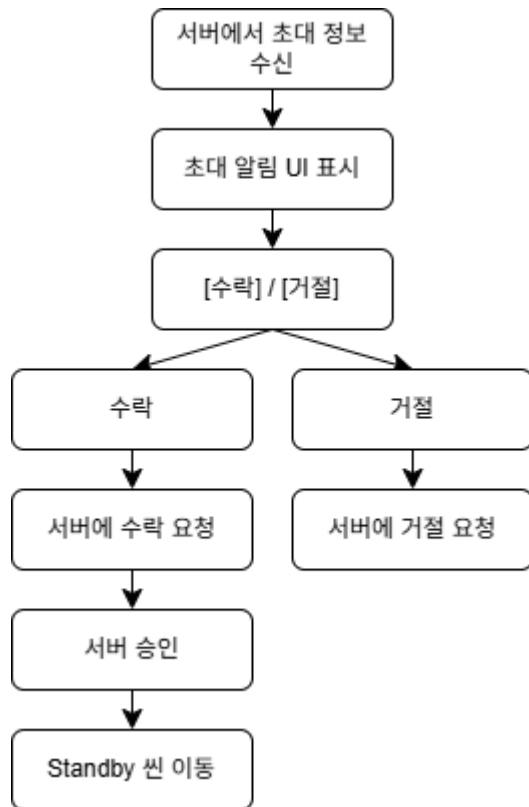
### 시스템 역할

- 초대 **UI** 표시
- 초대 가능한 플레이어 목록 표시
- 초대 대상 선택
- 초대 요청 입력 처리
- 초대 수신 **UI** 표시
- 초대 수락 / 거절 입력 처리
- 초대 상태 **UI** 갱신
- 초대 만료 시간 표시
- 초대 결과 알림 출력
- 방 참가 성공 시 **Standby** 씬 전환 처리

### 시스템 구조



- 초대 받는 플레이어의 경우



## UI 구성

### Invite UI



### 초대 목록 UI

- 플레이어가 현재 방에 있는 상태에서 **ESC**를 눌러 상세 로비 **UI**를 열 수 있다
- 상세 로비 **UI**에서
  - **[Ready]**
  - **[플레이어 초대]**
  - **[방 나가기]**
- 플레이어 초대 버튼을 선택하면 초대 목록 **UI**가 표시된다
  - **[플레이어 초대] -> InviteListPanel** 활성화

### 초대 가능한 플레이어 표시

- 표시 상태

| 플레이어 상태 | UI        |
|---------|-----------|
| 초대 가능   | 초대 버튼 활성화 |
| 이미 초대함  | 초대 대기 중   |

|         |                 |
|---------|-----------------|
| 다른 방 참가 | 초대 불가           |
| 게임 진행 중 | 초대 불가           |
| 오프라인    | 표시하지 않음 또는 비활성화 |

#### 초대 요청

- 플레이어가 목록에서 대상 플레이어의 초대 버튼을 선택한다
  - **Player B 선택 -> [초대] -> RequestInvite(Player B)**
- 클라이언트는 초대 요청을 서버에 전달한다
- 초대 요청 후 버튼 상태를 변경한다
  - **[초대] -> [초대 요청 중] -> 서버 응답 대기**
- 서버에서 초대가 성공적으로 생성되면
  - **[초대 요청 중] -> [초대 대기 중]**

#### 초대 요청 실패

- 서버에서 초대 요청이 거부되면 플레이어 화면에 실패 메시지를 표시한다
- “초대 할 수 없습니다”

| 상황            | UI메시지            |
|---------------|------------------|
| 방이 가득 참       | 방의 인원이 가득 찼습니다   |
| 게임 시작         | 게임이 이미 시작되었습니다   |
| 대상이 게임 중      | 현재 게임 중인 플레이어입니다 |
| 대상이 다른 방에 있음  | 이미 다른 방에 참가 중입니다 |
| 대상 Disconnect | 플레이어가 오프라인입니다    |

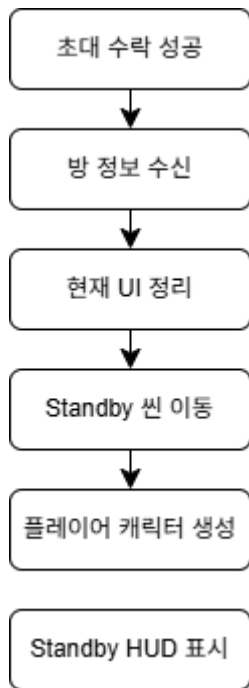
#### 초대 수신

- 다른 플레이어가 자신을 초대하면 서버에서 초대 정보를 전달받는다
- 클라이언트는 즉시 초대 수신 팝업을 표시한다
  - **Invite 수신 -> InviteReceivePopup 활성화**
- 표시 정보
  - 초대한 플레이어 이름
  - 방 이름
  - 현재 인원
  - 최대 인원
  - 남은 초대 시간

#### 초대 수락

- 플레이어가 수락 버튼을 선택한다
  - **[수락] -> 버튼 입력 잠금 -> 서버에 AcceptInvite 요청 -> 결과 대기**
- 서버가 방 참가를 승인하면





- 초대 수락으로 참가한 플레이어는 기본적으로
  - **Ready = False** 상태로 시작한다

#### 초대 수락 실패

- 초대 수락 시점에서 방 상태가 변경될 수 있다
- 서버에서 참가 실패 결과를 받으면 **Standby** 씬으로 이동하지 않고 현재 화면을 유지한다
  - 방 참가에 실패했습니다
- 예시
  - 방 최대 인원 초과
  - 방 삭제
  - 게임 시작
  - 초대 만료
  - 초대 취소

#### 초대 거절

- 플레이어가 [거절] 버튼을 선택한다
  - [거절] -> 서버에 **DeclineInvite** 요청 -> 팝업 닫기
- 서버에서 정상 처리 결과를 받으면 초대 **UI**를 완전히 종료한다

#### 초대 취소

- 초대한 플레이어는 자신이 보낸 초대를 취소할 수 있다
  - 초대 대기중 -> [초대 취소] -> 서버에 **CancellInvite** 요청 -> 결과 수신 -> 초대 상태 **UI** 갱신
- 상태 변경
  - 초대 대기 중 -> 초대 버튼 활성화

#### 초대 만료

- 초대에는 제한 시간이 존재한다
- 클라이언트는 서버에서 전달받은 만료 시간을 기준으로 남은 시간을 **UI**에 표시한다

- 예) 초대 만료까지 2분
  - 시간이 종료 되면
    - 남은 시간 0 -> 초대 UI 자동 종료
- 단 클라이언트가 시간이 끝났다고 초대를 최종적으로 삭제하는 것이 아니라 서버의 만료 처리 결과를 기준으로 종료한다

#### 초대 상태 UI

- 초대 상태에 따라 UI를 변경한다

| 초대 상태     | UI 처리     |
|-----------|-----------|
| Pending   | 초대 대기 중   |
| Accepted  | 참가 처리     |
| Declined  | 초대 거절 알림  |
| Expired   | 초대 만료 알림  |
| Cancelled | 초대 취소 알림  |
| Failed    | 초대 실패 메시지 |

#### 초대 UI 입력 처리

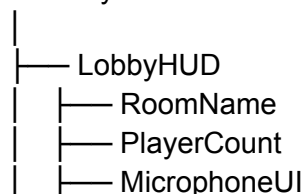
- 초대 UI가 열려 있는 동안에는 UI 입력을 우선 처리한다
  - InvitePanel 활성화 -> 마우스 커서 표시 -> 캐릭터 이동 입력 제한 -> 카메라 회전 제한
- UI를 닫으면
  - InvitePanel 비활성화 -> 게임 입력 복귀 -> 캐릭터 이동 가능
- 단 현재 Standby 씬에서 캐릭터가 실제로 돌아다니는 구조이므로 초대 목록 UI는 ESC 상세 로비 UI 위에 표시되는 서브 패널로 구성한다
  - LobbyDatailPanel -> [플레이어 초대] -> InviteListPanel

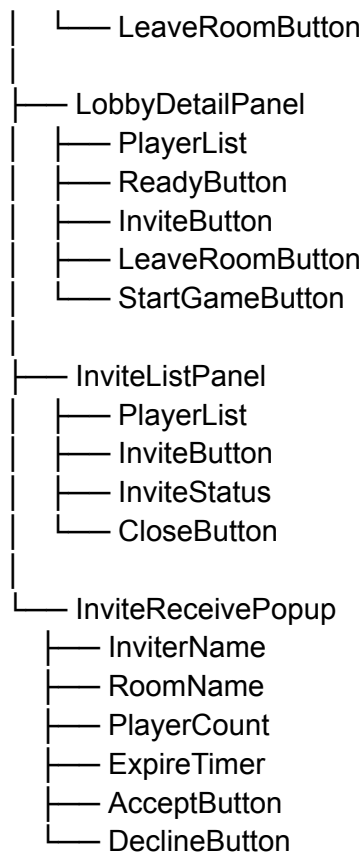
#### Standby 씬과 연결

- 전체 클라이언트 흐름은 다음과 같다
  - Lobby Scene -> 방 생성 / 방 참가 -> Standby 씬 -> 캐릭터 생성 -> Lobby HUD 표시
- 플레이 중
  - ESC -> 상세 로비 UI -> [플레이어 초대] -> InviteListPanel -> 초대 요청
- 초대를 받은 플레이어
  - 현재 Lobby Scene -> InviteReceivePopup -> 초대 수락 -> 서버 승인 -> Standby 씬 이동

#### UI 상태 구조

##### StandbyCanvas





#### 클라이언트 처리

- 초대 버튼 입력
- 플레이어 목록 표시
- 초대 요청 전달
- 초대 수신 팝업
- 수락 / 거절 입력
- 초대 상태 UI 갱신
- 남은 시간 표시
- 실패 메시지 출력
- Standby 썸 전환

#### 이벤트 흐름

OnInviteButtonClicked



RequestInvite



OnInviteRequested



OnInviteReceived



OnInviteAccepted

또는

OnInviteDeclined

또는

OnInviteExpired

↓  
OnInviteClosed

필요한 클래스

InviteUIManager

- └ InviteListUI
- └ InviteReceiveUI
- └ InviteStatusUI
- └ InviteNotificationUI

InviteClientManager

- └ InviteRequestHandler
- └ InviteResponseHandler
- └ InviteSceneHandler

예외 처리

- 초대 중 동일 플레이어 다시 선택 -> 버튼 비활성화
- 초대 대상이 초대 불가 상태 -> 버튼 비활성화
- 초대 수락 중 중복 클릭 -> 버튼 입력 잠금
- 초대 수락 중 서버 응답 지연 -> 로딩 표시
- 초대가 만료됨 -> 팝업 종료 및 안내
- 초대자가 초대 취소 -> 팝업 종료
- 방이 삭제됨 -> 참가 실패 메시지
- 게임 시작으로 초대 취소 -> 팝업 종료
- 수락 시 방이 가득 참 -> 참가 실패 메시지
- 초대 수신 중 다른 초대 도착 -> 초대 목록 또는 우선순위에 따라 표시
- Standby 썸 이동 중 중복 요청 -> 썸 전환 중 추가 입력 차단

서버 / 클라 역할 분리

| 기능            | 서버 | 클라     |
|---------------|----|--------|
| 초대 요청         | O  | 요청 전송  |
| 초대 가능 여부 확인   | O  | UI상 표시 |
| 방 존재 여부 확인    | O  | X      |
| 방장 여부 확인      | O  | X      |
| 방 최대 인원 확인    | O  | X      |
| 대상 플레이어 상태 확인 | O  | 표시     |
| 게임 진행 여부 확인   | O  | 표시     |
| 초대 정보 생성      | O  | X      |
| 초대 정보 저장      | O  | X      |
| 초대 대상에게 전달    | O  | 수신     |

|               |             |          |
|---------------|-------------|----------|
| 초대 수신 팝업      | X           | O        |
| 초대 수락         | 최종 판정       | 입력 전달    |
| 초대 거절         | 처리          | 입력 전달    |
| 실제 방 참가 처리    | O           | 결과 수신    |
| Standby 씬 이동  | 참가 승인 정보 전달 | O        |
| 초대 취소         | 처리          | 요청       |
| 초대 만료 판정      | O           | 남은 시간 표시 |
| 초대 상태 관리      | O           | UI 갱신    |
| 중복 초대 검사      | O           | 버튼 제한    |
| 초대 성공 / 실패 알림 | 결과 전달       | UI 표시    |

#### 초대 코드 생성

- 방이 생성될 때 서버에서 초대 코드를 생성한다

#### 방 생성 조건

- 모든 방은 고유한 초대 코드를 가진다
- 이미 사용 중인 코드는 중복되지 않아야한다
- 초대 코드는 서버에서 생성한다
- 클라이언트가 임의로 코드를 생성할 수 없다
- 방 생성 -> Room ID : Room34 -> Invite Code : qwe45h6b

#### 초대 코드 구성

- 초대 코드는 플레이어가 직접 전달하기 쉽게 문자열로 구성한다
- 초대코드는 알파벳, 숫자를 섞어 8자리로 만든다
- 조건

|  |  |
|--|--|
|  |  |
|  |  |
|  |  |
|  |  |
|  |  |
|  |  |
|  |  |

-

# 미니맵, 지도시스템

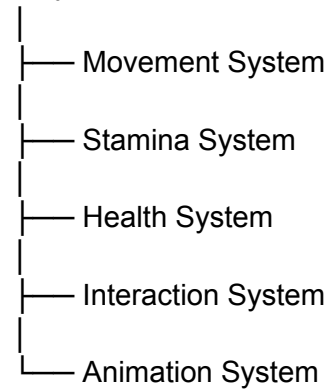


# 서버 시스템



## 플레이어 구조

### Player



| 시스템    | 서버 판정 | 클라이언트    |
|--------|-------|----------|
| 이동     | 위치 검증 | 이동 애니메이션 |
| 스태미너   | 서버    | UI 변화    |
| 살인     | 서버 승인 | 이펙트      |
| 아이템 획득 | 서버    | UI       |
| 미션 완료  | 서버    | 진행도 표시   |
| 탈출     | 서버    | 연출       |
| 사망     | 서버    | 애니메이션    |
| 증거 생성  | 서버    | 표시       |

- 룸 생성
- 방 입장
- 플레이어 관리
- 역할 배정
- 플레이어 Spawn
- RPC
- Network Object
- 변수 동기화
- 게임 시작
- 게임 종료
- 승리 판정
- 재접속
- 플레이어 나감
- Ping 처리

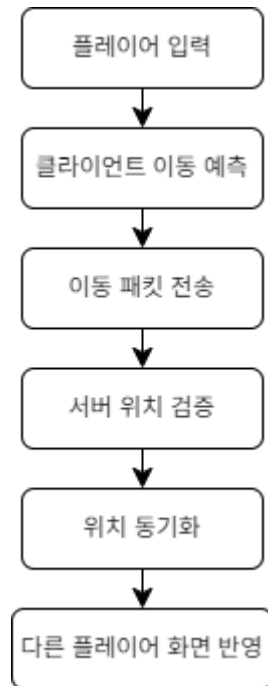
살인 -> 서버 승인 -> 전체에게 전파

# 플레이어 이동 시스템

#### 시스템 목적

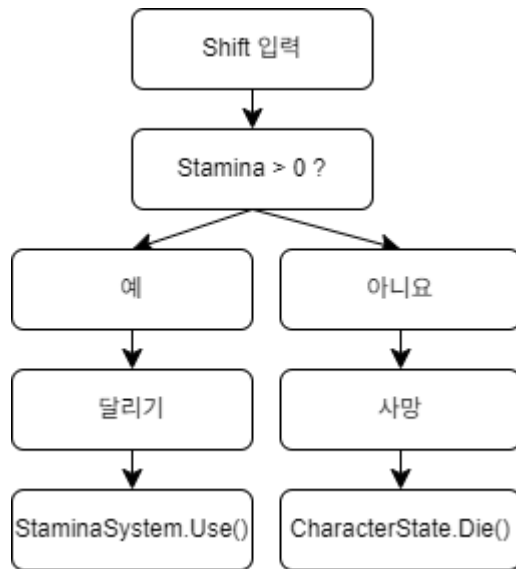
- 플레이어가 맵을 자유롭게 이동하여 미션을 수행하고, 다른 플레이어를 관찰하며, 추리와 생존 활동을 할 수 있도록 한다
- 이동은 모든 행동의 기반이 되는 시스템이며, 모든 플레이어가 동일한 조건에서 이동해야 한다

#### 시스템 구조



#### 이동 방식

| 항목        | 내용    |
|-----------|-------|
| 이동 방식     | WASD  |
| 달리기       | Shift |
| 점프        | Space |
| 시점        | 마우스   |
| 상호작용 중 이동 | 불가능   |
| 공격 중 이동   | 제한    |
| 사망 상태     | 불가능   |



#### 서버 판정

- Player Position
- Player Rotation
- Move Speed
- Current State
- Alive 여부

#### 이동 처리 순서

1. 클라이언트 입력
2. 서버 이동 요청
3. 속도 검사
4. 벽 충돌 검사
5. 최종 위치 승인
6. 전체 플레이어 동기화

#### 클라이언트 처리

- 입력 처리
- 이동 애니메이션
- 발소리
- 카메라 이동

#### 이동 제한

| 상태     | 이동 가능 |
|--------|-------|
| 기본     | O     |
| 미션 수행  | X     |
| 탐지기 사용 | X     |
| 제작     | X     |
| 사망     | X     |

|     |       |
|-----|-------|
| 관전자 | 자유 이동 |
|-----|-------|

예외 처리

높은 Ping

클라이언트는 -> Presicion 적용 -> 서버 위치 수신 -> 보정

벽 통과

서버 충돌 검사 -> 실패 -> 이전 위치 복귀

속도 핵

허용 속도 초과 -> 이동 거부 -> 로그 기록

순간이동

허용 거리 초과 -> 동기화 거부

이동 중 사망

즉시 이동 정지 -> 사망 애니메이션

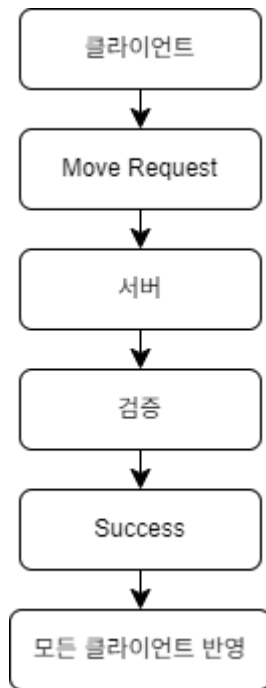
이동 중 탈출

탈출 우선 -> 이동 입력 무시

동기화 주기

|           |       |
|-----------|-------|
| 데이터       | 주기    |
| Position  | 20Hz  |
| Rotation  | 20Hz  |
| Animation | Event |
| Footstep  | Event |

서버 Authority



#### 필요한 클래스

- PlayerController
- NetworkPlayer
- MovementManager
- MovementValidator
- CharacterState
- AnimationController

#### 필요한 RPC

- RequestMove()
- UpdatePosition()
- SyncRotation()
- Teleport()
- StopMove()

#### 필요한 Network Variable

- Position
- Rotation
- MoveState
- Alive
- CurrentSpeed

#### 서버 / 클라 분리

|           |          |
|-----------|----------|
| 클라        | 서버       |
| 입력 처리     | 이동 검증    |
| 캐릭터 이동 연출 | 최종 위치 판정 |

|              |            |
|--------------|------------|
| 애니메이션 재생     | 충돌 검사      |
| 카메라 제어       | 속도 검증      |
| 발소리 재생       | 위치 동기화     |
| 상태에 따른 이동 제한 | 플레이어 상태 관리 |

# 체력 시스템



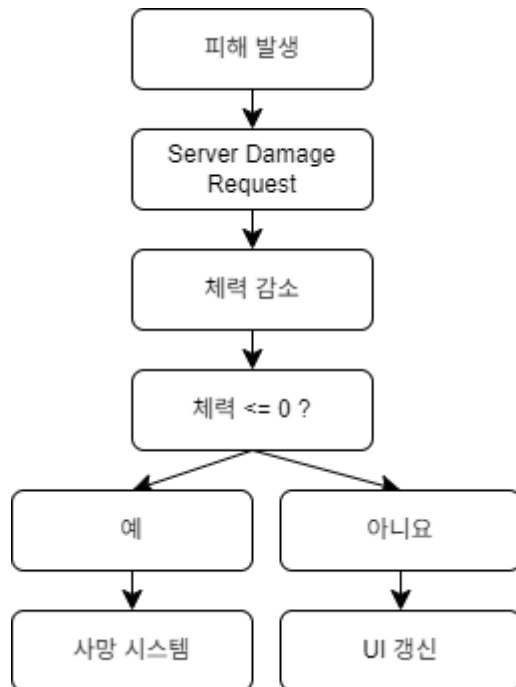
### 시스템 목적

- 체력 시스템은 플레이어 생존 상태를 관리하는 핵심 시스템
- 플레이어는 피해를 입거나 회복 아이템을 사용하면서 체력이 변화하며, 체력이 0이 되면 사망 상태로 전환 된다
- 스테미너 사용이 0이되면 체력이 초당 10깍인다
- 체력은 모든 플레이어가 동일한 규칙을 적용받으며, 서버가 최종 값을 관리한다

### 시스템 역할

- 플레이어 현재 체력 관리
- 피해 계산
- 회복 계산
- 사망 판정
- 체력 UI 동기화
- 회복 아이템 사용 처리

### 시스템 구조



### 기본 능력치

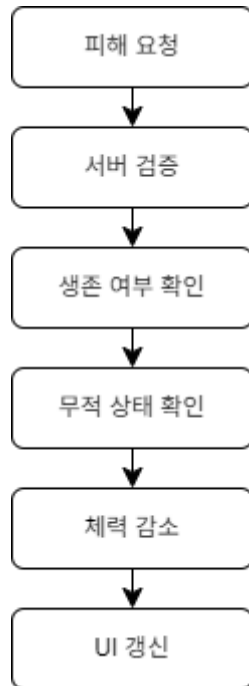
| 항목       | 값(예시) |
|----------|-------|
| 최대 체력    | 100   |
| 시작 체력    | 100   |
| 최소 체력    | 0     |
| 최대 체력 제한 | 100   |

### 체력 변경 방법

| 행동 | 체력 변화 |
|----|-------|
|----|-------|

|           |      |
|-----------|------|
| 살인 성공     | 즉시 0 |
| 회복 아이템 사용 | +회복량 |
| 무기 공격     | -피해량 |

#### 피해 처리



#### 회복 처리

- 회복은 회복 아이템을 통해서만 이루어진다
- 회복 후 체력은 최대 체력을 초과할 수 없다

현재 체력 =  $\text{Min}(\text{현재 체력} + \text{회복량} \leq \text{최대 체력})$

#### 사망 조건

- 체력  $\leq 0$
- 살인 시스템에서 즉사 판정

#### 사망 시

- 이동 불가
- 상호작용 불가
- 아이템 사용 불가
- 관전자 시스템으로 전환 -> 관전자 상태로는 자유 이동 가능

#### 서버 판정

##### 서버가 관리

- 현재 체력
- 최대 체력
- 생존 여부
- 피해 계산
- 회복 계산
- 사망 여부

클라이언트가 관리

- 체력바 UI
- 피격 효과
- 화면 흔들림
- 피격 사운드
- 회복 이펙트

상태 변화

| 상태    | 체력 변경 가능 |
|-------|----------|
| 생존    | O        |
| 사망    | X        |
| 관전자   | X        |
| 탈출 완료 | X        |

UI

플레이어 좌측 하단

체력이 낮을 수록

- 화면 가장자리 붉은 효과
- 심장 박동 소리 증가
- 화면 가운데 경고 표시
- 주변 플레이어 음성 작게 들림(거의 안들림)

Network Variable

- CurrentHealth
- MaxHealth
- IsAlive

RPC

- RequestDamage()
- RequestHeal()
- SyncHealth()
- PlayerDead()

필요한 클래스

- HealthSystem
- DamageManager
- HealManager
- PlayerStatus
- HUDManager

예외처리

- 동시에 회복과 피해 -> 서버 처리 순서 기준 적용 -> 최종 체력 계산
- 최대 체력 초과 회복 -> 120 -> 100으로 제한
- 이미 사망한 플레이어 회복 -> 회복 실패 -> 아이템 사용 취소

- 체력이 1인데 더 큰 피해 -> 1 -> -100 -> 0 -> 사망
- Ping 차이 -> 클라이언트 80HP -> 서버 60HP -> 서버값 적용
- Disconnect -> 피해 계산 중 종료 -> 서버가 최종 체력 저장 -> 재접속 시 복원

#### 서버 / 클라 분리

| 클라          | 서버         |
|-------------|------------|
| 체력 UI 표시    | 체력 계산      |
| 피격 연출       | 피해 적용      |
| 회복 연출       | 회복 처리      |
| 저체력 경고 효과   | 생존 여부 판정   |
| 사망 애니메이션    | 사망 판정      |
| 화면 효과 및 사운드 | 최종 체력 값 관리 |

사망, 시체, 관전자 시스템

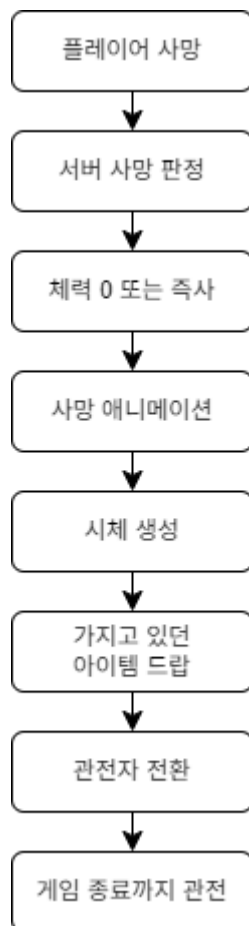
### 시스템 목적

- 사망 시스템은 플레이어의 생존 여부를 관리하고, 사망한 플레이어를 관전자 상태로 전환하여 게임의 진행을 방해하지 않으면서도 게임을 계속 관람할 수 있도록 하는 시스템이다
- 사망 판정은 반드시 서버에서 수행하며, 모든 클라이언트는 서버의 결과를 동기화한다

### 시스템 역할

- 플레이어 사망 판정
- 시체 생성
- 아이템 처리
- 증거 생성
- 플레이어 입력 제한
- 관전자 상태 전환
- 관전자 시점 변경
- 게임 종료 시 결과 화면 전환

### 시스템 구조



### 사망 조건

- 살인마 즉사 공격 성공
- 무기에 계속 맞아 체력이 0이 됨
- 달리기, 점프를 계속하여 피가 깎여 체력이 0이 됨

#### 사망 처리 순서

| 순서 | 처리       |
|----|----------|
| 1  | 서버 사망 판정 |
| 2  | 이동 중지    |
| 3  | 현재 행동 취소 |
| 4  | 사망 애니메이션 |
| 5  | 체력 0     |
| 6  | 시체 생성    |
| 7  | 아이템 드랍   |
| 8  | 관전자 전환   |

#### 서버 판정

##### 서버가 관리

- 생존 여부
- 사망 위치
- 사망 원인
- 사망 시간
- 시체 생성
- 아이템 드랍 생성
- 관전자 여부

##### 클라이언트가 관리

- 사망 애니메이션
- 효과음
- 화면 암전
- UI 변경
- 관전자 카메라

#### 플레이어 상태 변화

| 상태  | 이동 | 상호작용 | 채팅         | 관전 |
|-----|----|------|------------|----|
| 생존  | O  | O    | O          | X  |
| 사망  | X  | X    | X          | X  |
| 관전자 | O  | X    | O(관전자 채팅만) | O  |
| 탈출  | X  | X    | X          | O  |

#### 시체 시스템

- 사망 시 시체가 생성된다

시체는

- 이동 불가
- 삭제 불가
- 근처에 들고있던 아이템 드랍
- 다른 플레이어 발견 가능

#### 아이템 처리

- 사망 시 아이템 전부 바닥에 드랍

#### 증거 생성

- 사망 시 혈흔, 시체 생성 여부를 서버가 결정한다

#### 관전자 시스템

- 목적
  - 사망한 플레이어가 게임 진행을 관람할 수 있도록 한다
  - 단, 생존자에게 정보를 전달하지 못하도록 생존자에게는 관전자가 보이지 않는다

#### 관전자 전환

- 사망 -> 5초대기 -> 관전자 모드

#### 관전자 카메라

- 플레이어 관찰 버튼을 눌러 다른 플레이어 시점으로 관찰 가능
- 관전 가능한 대상
  - 살아있는 시민, 살인마
- 좌우 키로 변경

#### 관전자 UI

- 관찰하기(자유 상태일때만 표시)
- 화면 닫기 버튼(관전 상태일때만 표시)
- 현재 관찰 : **PlayerA**(카메라 관찰 중일 때만 표시)
- <- 다른 플레이어 ->(카메라 관찰 중일 때만 표시)

#### 관전자 기능

| 기능      | 가능 여부                  |
|---------|------------------------|
| 플레이어 변경 | O                      |
| 자유 이동   | O(이동과 관전 카메라 둘다 사용 가능) |
| UI 보기   | O                      |
| 미니맵     | O                      |
| 채팅      | 관전자 전용                 |
| 핑 찍기    | X                      |
| 아이템 사용  | X                      |

#### 채팅 제한



- 생존자와 관전자는 채팅이 분리된다
- 

| 듣는 사람 | 말하는 사람 | 들림 여부     |
|-------|--------|-----------|
| 생존자   | 생존자    | O (거리 비례) |
| 생존자   | 사망자    | X         |
| 사망자   | 생존자    | O (거리 비례) |
| 사망자   | 사망자    | O (거리 비례) |

게임 종료 시 관전자도 결과 화면으로 이동한다

서버 판정

- 관전자 여부
- 관전 대상
- 생존자 목록
- 게임 종료

Network Variable

- IsAlive
- IsSpectator
- CurrentSpectateTarget

RPC

- PlayerDead()
- SpawnBody()
- SpawnEvidence()
- EnterSpectator()
- ChangeSpectator()
- GameEnd()

필요한 클래스

- DeathManager
- SpectatorManager
- BodyManager
- EvidenceManager
- CameraManager

예외 처리

- 사망과 탈출이 동시에 발생 -> 서버 시간 기준 먼저 승인된 이벤트 적용
- 관전 대상이 사망 -> 자동으로 다른 생존자 관전
- 생존자가 한 명도 없음 -> 결과 화면으로 이동
- **Disconnect** -> 사망 상태 저장 -> 재접속 -> 관전자 복귀
- 시체 위에서 사망 -> 겹치지 않도록 자동 위치 보정
- 시체가 문 중간에 생성 -> 가장 가까운 빈 위치에 생성
- 게임 종료 직전 사망 -> 사망 처리 후 즉시 결과 화면

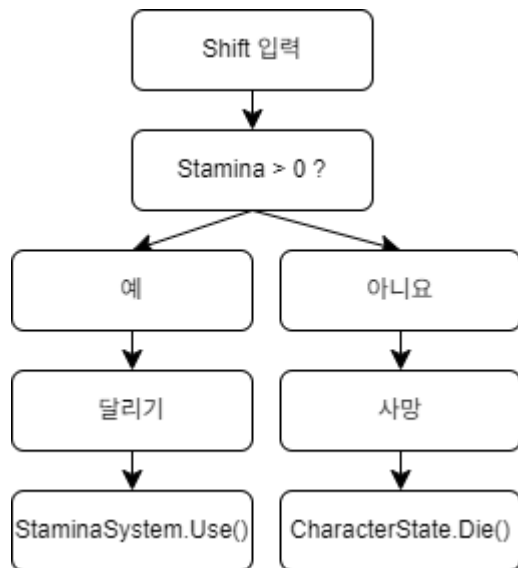
서버 / 클라 분리

| 클라          | 서버        |
|-------------|-----------|
| 사망 애니메이션    | 사망 판정     |
| 화면 암전 및 효과음 | 사망 위치 관리  |
| 입력 제한       | 시체 생성     |
| 관전자 카메라     | 아이템 드롭 처리 |
| 관전자 UI      | 증거 생성     |
| 플레이어 시점 전환  | 관전자 상태 관리 |
| 결과 화면 표시    | 게임 종료 판정  |

# 스태미너 시스템

#### 시스템 목적

- 플레이어의 행동을 제한하여 무한 질주를 방지하고, 이동 경로와 행동 선택에 전략성을 부여한다



#### 사용하는 시스템

| 시스템 | 사용 여부 |
|-----|-------|
| 달리기 | O     |
| 점프  | O     |
| 살인  | X     |
| 제작  | X     |

#### 최대 스테미너

100

#### 소비량

| 행동  | 소비량   |
|-----|-------|
| 달리기 | 초당 10 |
| 점프  | 초당 15 |

#### 회복

| 상태 | 회복량   |
|----|-------|
| 걷기 | 초당 5  |
| 정지 | 초당 10 |
| 사망 | 회복 안함 |

회복 시작 조건

스태미너 소비 후 2초 동안 새로운 소비 없음 -> 회복 시작

회복 중

걸기 -> 천천히 회복

정지 -> 빠르게 회복

스태미너 0 -> 체력 10 감소

UI

체력바 아래

노란색 게이지

서버 판정

| 항목        | 서버 | 클라이언트     |
|-----------|----|-----------|
| 스태미너 값    | O  | UI 표시     |
| 소비        | O  | 게이지 감소 연출 |
| 체력 감소     | O  | 체력 감소 연출  |
| 회복        | O  | 게이지 증가 연출 |
| 달리기 가능 여부 | O  | 애니메이션     |
| 사망 여부     | O  | 애니메이션     |

예외 처리

달리기 중 0 -> 체력이 깎임

점프 직후 0 -> 체력이 깎임

회복 중 다시 달리기 -> 회복 중간 -> 다시 소비

서버와 클라이언트 값 차이 - 서버 값 우선

서버 / 클라 분리

| 클라              | 서버              |
|-----------------|-----------------|
| 스태미너 게이지 표시     | 스태미너 값 계산       |
| 게이지 감소 회복 애니메이션 | 스태미너 소비 및 회복 처리 |
| 부족 경고 효과        | 체력 감소 판정        |
| 체력 감소 연출        | 달리기 가능 여부 판정    |
| 이동 애니메이션 반영     | 사망 판정           |

# 사운드 시스템



음성시스템



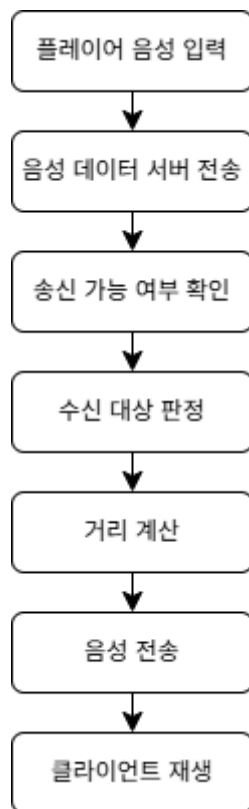
### 시스템 목적

- 음성 시스템은 플레이어 간 실시간 음성 채팅을 관리하는 시스템이다
- 서버는 플레이어의 상태와 거리 정보를 기반으로 음성 송수신 가능 여부를 판정하며, 생존자와 관전자 간의 정보 공유를 제한하여 게임의 공정성을 유지한다

### 시스템 역할

- 음성 송수신 대상 판정
- 거리 기반 음성 범위 계산
- 플레이어 상태 확인
- 음성 채널 관리
- 관전자 채널 관리
- 음성 차단 처리
- 게임 종료시 음성 채널 종료

### 시스템 구조



### 음성 통신 방식

| 항목       | 내용        |
|----------|-----------|
| 방식       | 음성 인식(기본) |
| 음소거      | V를 눌러 음소거 |
| 음성 범위    | 거리 기반     |
| 최대 음성 거리 | 15m       |

|       |              |
|-------|--------------|
| 감쇠 방식 | 거리에 따라 볼륨 감소 |
| 장애물   | 벽을 통과하지 않음   |

#### 서버 확인 항목

| 항목           | 확인 |
|--------------|----|
| Alive 여부     | O  |
| Spectator 여부 | O  |
| 위치           | O  |
| 음성 범위        | O  |
| 음소거 여부       | O  |
| 게임 종료 여부     | O  |
| 음성 채널        | O  |

#### 음성 처리 순서

| 순서 | 처리          |
|----|-------------|
| 1  | 플레이어 음성 수신  |
| 2  | 송신 가능 여부 확인 |
| 3  | 플레이어 상태 확인  |
| 4  | 거리 계산       |
| 5  | 수신 대상 선정    |
| 6  | 음성 데이터 전달   |

#### 플레이어 상태별 음성

| 상태     | 마이크 입력    | 음성 재생          |
|--------|-----------|----------------|
| 생존     | O(생존자 채널) | O(생존자 채널)      |
| 사망(대기) | X         | X              |
| 관전     | O(관전자 채널) | O(관전자, 생존자 채널) |
| 탈출     | X         | O(생존자 채널)      |

#### 채팅 가능 여부

| 말하는 사람 | 듣는 사람 | 들림 여부    |
|--------|-------|----------|
| 생존자    | 생존자   | O(거리 비례) |

|         |         |          |
|---------|---------|----------|
| 생존자     | 관전자     | O(거리 비례) |
| 생존자     | 탈출 플레이어 | O        |
| 관전자     | 생존자     | X        |
| 관전자     | 관전자     | O(거리 비례) |
| 관전자     | 탈출 플레이어 | X        |
| 탈출 플레이어 | 생존자     | X        |
| 탈출 플레이어 | 관전자     | X        |
| 탈출 플레이어 | 탈출 플레이어 | X        |

#### 음성 채널

- 생존자 채널
  - 생존자끼리만 통신
  - 거리 비례 음성 적용
- 관전자 채널
  - 관전자끼리만 통신
  - 생존자는 들을 수 없음
  - 관전자에게 생존자 목소리는 들림

#### 거리 판정

| 거리     | 볼륨     |
|--------|--------|
| 0~3m   | 100%   |
| 3~7m   | 70%    |
| 7~12   | 40%    |
| 12~15  | 10%    |
| 15m 이상 | 들리지 않음 |

#### 음성 차단 조건

| 조건       | 결과        |
|----------|-----------|
| 최대 거리 초과 | 음성 전달 안 함 |
| 다른 채널    | 전달 안 함    |
| 게임 종료    | 음성 종료     |
| 음소거      | 전달 안 함    |

## 상태 변화

| 상황     | 처리                   |
|--------|----------------------|
| 사망     | 생존 채널 종료             |
| 관전자 전환 | 관전자 채널 참가            |
| 탈출     | 생존 채널 종료 후 관전자 채널 참가 |
| 게임 종료  | 모든 채널 종료             |

## Network Variable

- IsAlive
- IsApectator
- Position
- IsTalking
- VoiceChannel

## RPC

- RequestVoice()
- StartVoive()
- StopVoice()
- JoinVoiceChannel()
- LeaveVoiceChannel()
- SyncVoiceTaget()

## 필요한 클래스

- VoiceManager
- VoiceChannelManager
- VoiceRangeManager
- VoiceValidator
- PlayerVoiceState
- SpectatorVoiceManager

## 예외 처리

플레이어 사망 -> 생존 채널 종료 후 관전자 전환 시 관전자 채널 참가

관전자가 생존자에게 말하기 -> 전달하지 않음

생존자가 관전자 음성 듣기 -> 전달하지 않음

음성 송신 중 **Disconnect** -> 즉시 채널 제거

최대 거리 초과 -> 음성 전송 중단

게임 종료 -> 모든 음성 채널 종료

음소거 플레이어 -> 송신 및 수신 제한(설정에 따라 적용)

탈출 완료 -> 생존 채널에서 제거 후 생존자 채널 참가(듣기만 말하기X)

## 서버 / 클라 분리

|        |             |
|--------|-------------|
| 클라     | 서버          |
| 마이크 입력 | 음성 송신 허용 여부 |

|                   |              |
|-------------------|--------------|
| 음성 재생             | 음성 수신 대상     |
| 거리별 볼륨 적용         | 거리 계산        |
| 말하는 플레이어 UI       | 음성 채널 관리     |
| 음소거 UI            | 플레이어 상태 확인   |
| 음성 설정(입출력 장치, 볼륨) | 게임 종료시 음성 종료 |

# 게임 진행 시스템

# 게임 진행 시스템

## 시스템 목적

- 게임 전체의 진행 상태를 관리하는 시스템이다.
- 게임 시작부터 종료까지 현재 게임 상태와 진행 시간을 서버에서 관리한다.
- 미션, 탈출, 사망 등 다른 시스템에서 발생하는 상태 변화를 전달받아 현재 게임을 계속 진행할 수 있는지 확인한다.
- 게임 종료 조건이 충족되거나 더 이상 승리 가능한 플레이어가 존재하지 않는 경우 게임 종료 시스템에 종료를 요청한다.
- 게임 진행에 필요한 주요 상태는 서버가 최종 권한을 가진다.

## 시스템 역할

- 게임 시작 관리
- 게임 상태 관리
- 게임 진행 시간 관리
- 플레이어 게임 상태 관리
- 미션 시스템 결과 수신
- 탈출 시스템 결과 수신
- 사망 시스템 결과 수신
- 역할 시스템 상태 수신
- 승리 가능 플레이어 확인
- 게임 종료 조건 검사
- 게임 종료 시스템 호출
- 게임 진행 중 상태 변경 제한
- 게임 종료 전 마지막 상태 동기화

게임 진행 시스템은 최종 승패를 직접 확정하지 않는다.

게임 종료가 필요한 상황을 판단하고 `GameEndManager`에 종료를 요청한다.

## 게임 상태

| 상태       | 설명             |
|----------|----------------|
| Waiting  | 게임 시작 전 대기     |
| Starting | 게임 시작 준비 및 초기화 |
| Playing  | 실제 게임 진행       |

|        |                      |
|--------|----------------------|
| Ending | 게임 종료 요청 이후 종료 처리 대기 |
| Result | 결과 처리 상태             |
| Closed | 게임 세션 종료             |

## 상태 흐름

```

Waiting
↓
Starting
↓
Playing
↓
Ending
↓
Result
↓

```

Closed

## 게임 시작

### 게임 시작 조건

- 최소 플레이어 수 충족
- 모든 플레이어 준비 완료
- 역할 배정 완료
- 미션 배정 완료
- 게임에 필요한 초기 데이터 생성 완료

### 처리 순서

게임 시작 조건 확인

↓  
GameState = Starting

↓  
역할 및 미션 초기화

↓  
플레이어 상태 초기화

↓  
게임 시간 초기화

↓  
각 시스템 초기화 완료 확인

↓  
GameState = Playing

↓



게임 타이머 시작

게임 시작 이후부터 서버가 게임 시간을 관리한다.

## 게임 시간 관리

- 게임 시간은 서버 기준으로 관리한다.
- 클라이언트의 로컬 시간을 게임 판정에 사용하지 않는다.
- 게임 시작 시 서버의 게임 시작 시간을 기록한다.
- 제한 시간에 도달하면 게임 진행을 중단하고 종료 조건을 검사한다.

## 기본 설정

| 항목       | 값         |
|----------|-----------|
| 게임 제한 시간 | 15분       |
| 게임 시작    | Playing   |
| 시간 종료    | Ending 전환 |

15:00

↓

게임 진행

↓

시간 경과

↓

00:00

↓

게임 진행 중단

↓

최종 상태 확인

↓

GameEndManager 호출

## 게임 진행 상태 관리

게임 진행 시스템은 다른 시스템의 결과를 받아 현재 게임 상태를 갱신한다.

## 주요 입력

| 시스템            | 전달 정보            |
|----------------|------------------|
| MissionManager | 공동/개인 미션 진행 및 완료 |

|                  |                  |
|------------------|------------------|
| EscapeManager    | 탈출구 상태 및 플레이어 탈출 |
| DeathManager     | 플레이어 사망          |
| RoleManager      | 플레이어 역할          |
| GameTimerManager | 현재 게임 시간         |

## 게임 진행 이벤트

게임 진행 시스템은 다음과 같은 이벤트가 발생할 때마다 게임 상태를 검사한다.

- 공동 미션 완료
- 개인 미션 완료
- 플레이어 사망
- 플레이어 탈출
- 탈출 가능 인원 변경
- 게임 시간 변경
- 제한 시간 도달
- 플레이어 **Disconnect**
- 기타 승리 조건에 영향을 주는 상태 변화

이벤트 발생

↓

GameProgressManager

↓

현재 상태 갱신

↓

승리 가능 여부 검사

↓

게임 종료 필요 여부 확인

## 승리 가능 플레이어 확인

게임 진행 시스템은 현재 게임에서 아직 승리 조건을 달성할 수 있는 플레이어가 존재하는지 확인한다.

확인 대상

- 생존 시민
- 탈출 시민
- 생존 살인마

시민

다음 중 하나의 승리 가능 경로가 남아있는지 확인한다.

[미션 승리 경로]

공동 미션 완료 가능

+

개인 미션 완료 가능

+

개인 행동 목표 성공 가능

또는

[탈출 승리 경로]

탈출 가능

+

개인 행동 목표 성공 가능

단, 탈출 성공 여부 자체는 **EscapeManager**가 관리하고, 게임 진행 시스템은 전달받은 상태를 이용한다.

살인마

- 시민이 남아 있는지 확인
- 살인마가 승리할 수 있는 상태인지 확인
- 시민 탈출 여부 확인

현재 게임 규칙상 시민이 한 명이라도 탈출한 경우 살인마 승리 조건은 무효화한다.

## 게임 종료 요청 조건

게임 진행 시스템은 다음 조건에서 **GameEndManager**에 게임 종료를 요청한다.

### 1. 시민 승리 조건 충족

공동 미션 100%

↓

개인 미션 완료

↓

개인 행동 목표 성공 여부 확인

↓

시민 승리 가능

↓

**GameEndManager**

※ 최종 승패 확정은 게임 종료 시스템에서 담당한다.

### 2. 살인마 승리 조건 충족

모든 시민 사망

↓

시민의 승리 가능 경로 없음

↓

살인마 승리 가능

↓

GameEndManager

단, 시민이 탈출한 경우에는 살인마 승리 조건을 무효화한다.

### 3. 제한 시간 종료

GameTime >= 15:00

↓

게임 상태 변경

↓

추가 게임 진행 차단

↓

GameEndManager

제한 시간 종료 시 승리 조건을 최종적으로 확인한 뒤 결과를 확정한다.

### 4. 승리 가능한 플레이어가 존재하지 않음

현재 남아있는 플레이어 중 어떠한 승리 조건도 달성할 수 없는 경우 게임 종료를 요청한다.

예:

시민 2명

살인마 2명

공동 미션 미완료

개인 미션 완료 불가능

탈출 불가능

↓

시민 승리 가능 경로 없음

↓

게임 진행 불가능

↓

GameEndManager 호출

## 탈출과 게임 진행

탈출 시스템은 플레이어의 탈출을 처리하고 결과를 게임 진행 시스템에 전달한다.

EscapeManager

↓

EscapeSuccess

↓

PlayerState = Escaped

↓  
GameProgressManager  
↓

승리 가능 플레이어 재검사

## 탈출한 플레이어

- 게임에서 제외
- 이동 불가
- 미션 수행 불가
- 생존자 관전
- 음성 송신 불가

단, 탈출했다고 즉시 전체 게임을 종료하지 않는다.

예:

시민 A 탈출

↓  
A 게임에서 제외  
↓  
나머지 시민 / 살인마 계속 진행  
↓

승리 가능 조건 재검사

## 플레이어 상태

| 상태        | 게임 참여 | 승리 조건        | 이동 |
|-----------|-------|--------------|----|
| Alive     | O     | O            | O  |
| Dead      | X     | X            | X  |
| Escaped   | X     | 탈출 결과에 따라 판정 | X  |
| Spectator | X     | X            | O  |

Dead, Escaped 상태가 발생하면 게임 진행 시스템에 전달하고 전체 게임 진행 가능 여부를 다시 확인한다.

## 게임 진행 중 상태 제한

GameState = Playing이 아닌 경우 일반적인 게임 플레이 요청을 제한한다.

| 상태       | 이동 | 공격 | 미션 | 탈출 | 결과 |
|----------|----|----|----|----|----|
| Waiting  | X  | X  | X  | X  | X  |
| Starting | 제한 | X  | X  | X  | X  |
| Playing  | O  | O  | O  | O  | X  |
| Ending   | X  | X  | X  | X  | O  |
| Result   | X  | X  | X  | X  | O  |
| Closed   | X  | X  | X  | X  | X  |

실제 이동/공격/미션/탈출 가능 여부는 각 시스템에서 최종적으로 검증한다.

## 게임 종료 요청

게임 진행 시스템에서 종료 조건이 확인되면 `GameEndManager`에 종료를 요청한다.

```
CheckGameProgress()
```

```
↓
종료 조건 확인
```

```
↓
IsGameEnding 확인
```

```
↓
IsGameEnding = true
```

```
↓
GameState = Ending
```

```
↓
```

```
GameEndManager.StartGameEnd()
```

### 중복 종료 방지

- 이미 `Ending`, `Result`, `Closed` 상태라면 추가 종료 요청을 처리하지 않는다.
- 서버에서 최초 종료 요청만 유효하게 처리한다.

## Network Variable

### 게임 상태

- `GameState`
- `GameTime`
- `GameStartTime`

- RemainingTime
- IsGameStarted
- IsGameEnding

## 플레이어 진행 상태

- PlayerGameState
- IsAlive
- IsEscaped
- CanWin

WinnerPlayers, WinnerTeam, GameEndReason 등 최종 결과 데이터는

GameEndManager / ResultManager에서 관리하는 것이 적절하다.

---

## RPC / 이벤트

게임 진행 시스템은 직접적인 **RPC**보다 서버 내부 이벤트 → 상태 검사 → **GameEndManager** 호출 구조를 권장한다.

### 서버 내부 이벤트

- OnGameStarted
- OnMissionStateChanged
- OnPlayerDeath
- OnPlayerEscape
- OnPlayerDisconnected
- OnGameTimeExpired

### 서버 처리

```
OnMissionStateChanged()
OnPlayerDeath()
OnPlayerEscape()
OnGameTimeExpired()
    ↓
CheckGameProgress()
    ↓
CheckWinAvailability()
    ↓
조건 충족
    ↓
```

```
GameEndManager.StartGameEnd()
```

### 클라이언트 동기화용

- SyncGameState()
- SyncGameTime()
- NotifyGameEnding()

## 필요한 클래스

### 핵심 클래스

- GameProgressManager
- GameStateManager
- GameTimerManager
- GameProgressValidator

### 연동 클래스

- MissionManager
- EscapeManager
- DeathManager
- RoleManager
- GameEndManager

### 역할 분리

| 클래스                   | 역할                  |
|-----------------------|---------------------|
| GameProgressManager   | 전체 게임 진행 관리         |
| GameStateManager      | 게임 상태 관리            |
| GameTimerManager      | 게임 시간 관리            |
| GameProgressValidator | 승리 가능 상태 및 종료 조건 검사 |
| GameEndManager        | 최종 게임 종료 처리         |
| ResultManager         | 최종 결과 데이터 및 결과 화면   |

## 예외 처리

| 상황 | 서버 처리 |
|----|-------|
|----|-------|



|                        |                     |
|------------------------|---------------------|
| 게임 시작 전 종료 요청          | 무시                  |
| 게임 종료 후 게임 진행 요청       | 무시                  |
| 탈출과 사망 동시 발생           | 서버에서 먼저 승인된 상태 적용   |
| 미션 완료와 사망 동시 발생        | 서버 처리 순서에 따라 상태 반영  |
| 제한 시간과 탈출 동시 발생        | 서버 게임 시간 기준 판정      |
| 여러 종료 조건 동시 발생         | 모든 상태 반영 후 최종 조건 검사 |
| 중복 게임 종료 요청            | 최초 요청만 처리           |
| 플레이어 <b>Disconnect</b> | 마지막 서버 상태 유지        |
| 게임 진행 중 서버 상태 불일치      | 서버 상태를 기준으로 동기화     |
| 게임 종료 중 신규 행동 요청       | 요청 거부               |
| 게임 종료 후 신규 행동 요청       | 요청 거부               |

# 게임 종료 시스템

### 시스템 목적

- 게임 종료 시스템은 게임 진행 중 발생하는 시민 및 살인마의 승리 조건을 종합하여 게임 종료 여부를 최종 판정하는 시스템이다
- 게임 종료 조건이 충족되면 더 이상의 게임 진행을 제한하고, 모든 플레이어에게 최종 결과를 전달한다

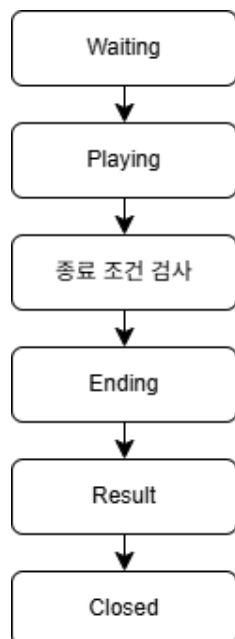
### 시스템

- 게임 종료 조건 검사
- 시민 승리 조건 검사
- 살인마 승리 조건 검사
- 탈출 승리 처리
- 제한 시간 관리
- 승리 가능 플레이어 확인
- 게임 진행 중단
- 최종 승패 판정
- 결과 데이터 저장
- 결과 화면 전환 요청
- 미션 및 개인 행동 목표 공개 요청

### 게임 상태

| 상태      | 설명               |
|---------|------------------|
| Waiting | 게임 시작 전 대기       |
| Playing | 게임 진행            |
| Ending  | 종료 조건 확인 및 결과 처리 |
| Result  | 결과 확정            |
| Closed  | 게임 세션 종료         |

게임 시작 후 기본 상태는 **Playing**으로 설정한다



#### 게임 종료 조건

- 게임 종료는 서버만 판정한다

#### 조건 1. 시민 전체 승리 조건 달성

- 시민이 공동 미션 **100%** 완료
- 개인 미션 완료

두 조건을 모두 만족하면 시민 승리 조건이 충족된다

- 공동 미션 **100%** + 개인 미션 완료 -> 시민 승리

#### 조건 2. 시민 탈출

- 탈출 성공 -> 해당 플레이어 승리 확정 -> 게임에서 제외
- 다른 플레이어의 게임은 승리 가능한 조건이 남아 있는 경우 계속 진행한다

#### 조건 3. 살인마 승리

- 모든 시민이 사망하거나 탈출 등으로 게임에 더 이상 참여할 수 없게 되어 살인마의 승리 조건이 충족된 경우 살인마 승리로 종료한다
- 현재 기획 기준으로는 시민이 한 명이라도 탈출하면 살인마는 패배하므로, 탈출 성공이 발생한 경우 살인마의 승리 조건을 즉시 무효화한다

#### 조건 4. 제한 시간 종료

- 게임 시간이 제한 시간에 도달하면 서버가 최종 승리 조건을 검사한다
- **15분** 도달 -> 게임 진행 중지 -> 승리 조건 검사 -> 최종 결과 확정
- 조건을 만족하지 못한 플레이어는 패배 처리한다

#### 조건 5. 승리 가능한 플레이어가 존재하지 않음

- 남아있는 플레이어 중 더 이상 승리 조건을 달성할 수 없는 경우 즉시 게임을 종료한다
- 시민 **2명**  
살인마 **2명**  
공동 미션 미완료  
개인 미션 완료 불가능  
탈출 가능 조건도 없음  
-> 더이상 시민이 승리할 방법 없음  
-> 게임 즉시 종료

#### 승리 판정 우선순위

- 기본 우선순위
  1. 탈출 / 사망 등 플레이어 상태 변경
  2. 미션 완료 상태 반영
  3. 승리 조건 검사
  4. 게임 종료 여부 결정
  5. 결과 확정

#### 탈출과 게임 종료

- 탈출한 플레이어가 발생했다고 해서 무조건 전체 게임을 종료하지 않는다
- 예) 시민 **4명**, 살인마 **2명**  
시민 **A** 탈출 -> **A = 승리** -> 나머지 플레이어 계속 진행

#### 시민 승리 판정

- 공동 미션 + 개인 미션

공동 미션 완료 -> 개인 미션 완료 여부 확인 -> 둘 다 만족 -> 시민 승리

| 공동 미션 | 개인 미션 | 결과 |
|-------|-------|----|
| O     | O     | 승리 |
| O     | X     | 패배 |
| X     | O     | 패배 |
| X     | X     | 패배 |

#### 사망한 시민 처리

- 사망한 시민은 게임에서 직접 행동할 수 없지만, 기획상 개인 미션은 계속 수행할 수 있도록 설계되어 있으므로 서버는 해당 플레이어의 개인 미션 상태를 유지한다
- 단체 미션은 해당 플레이어가 사망했다고 해서 별도로 실패 처리하지 않는다
- 시민 사망 -> 공동 미션 진행 참여 불가 -> 기존 공동 미션 진행도 유지 -> 개인 미션 진행 가능
- 공동 미션은 살아있는 시민들의 행동으로 완료 여부를 판단한다

#### 개인 행동 목표 처리

- 개인 행동 목표는 진영 승패와 별도로 관리한다
- 게임 종료 시 서버가 성공 여부를 최종 판정한다
- 게임 종료 -> 개인 행동 목표 검사 -> 성공/실패 -> 결과 데이터 저장

| 플레이어     | 행동 목표         | 결과 |
|----------|---------------|----|
| Player A | 회복 아이템을 주지 않기 | 성공 |
| Player B | 한 번도 달리지 않기   | 실패 |

- 개인 행동 목표 실패는 시민/살인마의 진영 승패에 영향을 주지 않는다

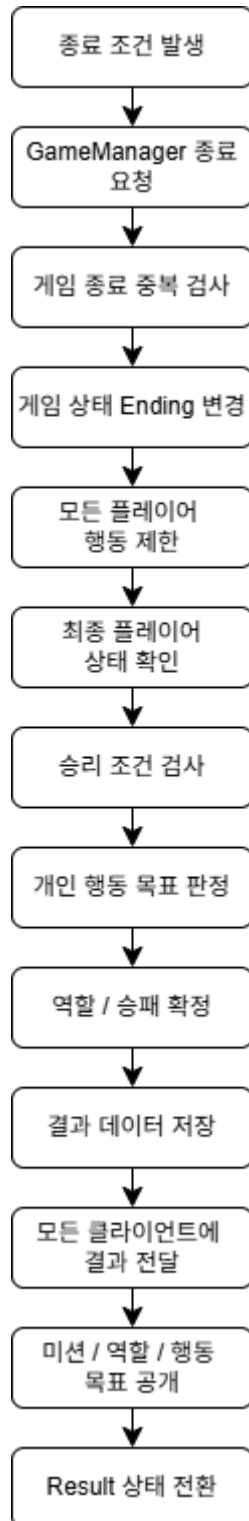
#### 결과 데이터

- 서버는 게임 종료 시 플레이어별 결과를 저장한다

| 데이터             | 내용          |
|-----------------|-------------|
| PlayerID        | 플레이어 식별자    |
| Role            | 역할          |
| IsAlive         | 생존 여부       |
| IsEscaped       | 탈출 여부       |
| TeamMission     | 공동 미션 결과    |
| PersonalMission | 개인 미션 결과    |
| PersonalGoal    | 개인 행동 목표 결과 |

|           |          |
|-----------|----------|
| Winner    | 승리 여부    |
| EndReason | 게임 종료 원인 |

게임 종료 처리 순서



게임 종료 중 행동 제한

|        |    |
|--------|----|
| 행동     | 가능 |
| 이동     | X  |
| 공격     | X  |
| 상호작용   | X  |
| 미션 수행  | X  |
| 아이템 사용 | X  |
| 탈출     | X  |
| 결과 확인  | O  |

게임 종료 원인

|                   |               |
|-------------------|---------------|
| EndReason         | 내용            |
| CitizenMission    | 시민 미션 승리      |
| CitizenEscape     | 탈출 승리         |
| KillerWin         | 살인마 승리        |
| TimeLimit         | 제한 시간 종료      |
| NoWinnerAvailable | 승리 가능 플레이어 없음 |
| ForceEnd          | 강제 종료         |

Network Variable

- GameState
- gameTime
- GameEndReason
- IsGameEnded
- WinnerTeam
- WinnerPlayers

플레이어 별

- IsWinner
- IsEscaped
- IsAlive
- PersonalMissionCompleted
- PersonalGoalCompleted

RPC

- RequestGameEnd()
- StartGameEnd()
- SyncGameState()
- SyncGameResult()

- RevealRole()
- RevealMission()
- RevealPersonalGoal()
- ShowResult()
- ReturnToLobby()

필요한 클래스

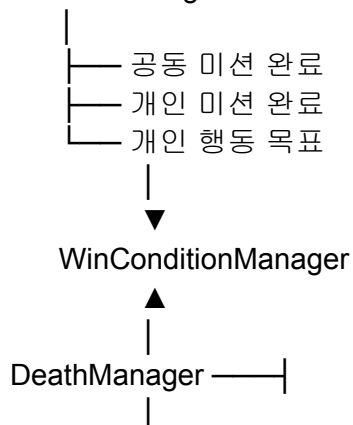
- GameStateManager
- WinConditionManager
- GameTimerManager
- ResultManager
- MissionResultManager
- GameEndManager

예외 처리

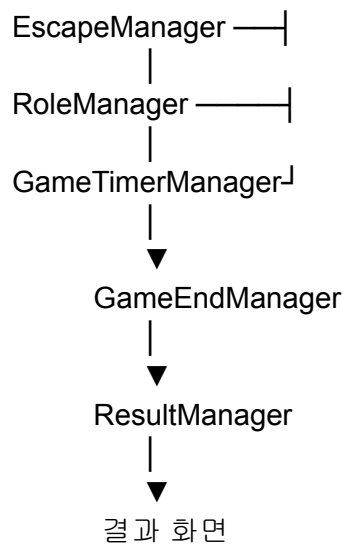
| 상황                   | 서버 처리                        |
|----------------------|------------------------------|
| 탈출과 사망 동시 발생         | 서버 시간 기준 먼저 승인된 상태 적용        |
| 미션 완료와 사망 동시 발생      | 서버가 먼저 처리한 이벤트 반영            |
| 제한 시간과 탈출 동시 발생      | 서버 시간 기준 판정                  |
| 두 플레이어가 동시에 승리 조건 달성 | 먼저 승인된 이벤트 처리 후 전체 승리 조건 재검사 |
| 게임 종료 후 RPC 수신       | 무시                           |
| 게임 종료 후 공격 요청        | 거부                           |
| 게임 종료 후 탈출 요청        | 거부                           |
| 클라이언트 Disconnect     | 서버에 저장된 상태로 결과 처리            |
| 결과 데이터 전송 실패         | 서버 결과 데이터 유지 후 재전송           |
| 중복 GameEnd 호출        | 최종 종료 처리만 적용                 |

전체 시스템 연동

MissionManager







#### 서버 / 클라이언트 역할 분리

| 클라이언트       | 서버             |
|-------------|----------------|
| 종료 UI 표시    | 종료 전 판정        |
| 승리 / 패배 연출  | 승리 / 패배 판정     |
| 역할 공개 UI    | 역할 데이터 관리      |
| 미션 결과 UI    | 미션 결과 확정       |
| 개인 행동 목표 UI | 행동 목표 성공 여부 판정 |
| 결과 화면       | 최종 결과 데이터 생성   |
| 로비 이동 UI    | 게임 세션 종료       |

# 탈출 시스템

### 시스템 목적

- 탈출 시스템은 게임 후반부에 탈출구를 개방하여 시민에게 또 다른 승리 수단을 제공하는 시스템
- 서버는 탈출구 개방, 탈출 가능 인원, 탈출 성공 여부를 최종 판단하며, 협력과 추리 결과에 따라 탈출 가능 인원을 조정하여 플레이어 간 심리전 유도한다

### 시스템 역할

- 탈출구 개방 관리
- 탈출 가능 인원 결정
- 탈출 가능 여부 판정
- 탈출 처리
- 탈출 플레이어 상태 변경
- 탈출 가능 인원 증가 처리
- 게임 종료 조건 검사

### 시스템 구조



#### 탈출 인원

| 항목    | 내용             |
|-------|----------------|
| 최소 인원 | 현재 시민 수의 최소 기준 |
| 최대 인원 | 현재 시민 수        |
| 결정 시점 | 게임 시작          |

#### 예)

| 시민 수 | 탈출 가능 인원 |
|------|----------|
| 4명   | 1~4명 랜덤  |
| 5명   | 1~5명 랜덤  |
| 6명   | 1~6명 랜덤  |

#### 탈출 인원 증가

- 공동 미션 수행, 정보 공유 등 일정 조건을 만족하면 서버가 탈출 가능 인원을 증가시킨다

#### 예)

| 조건            | 증가량 |
|---------------|-----|
| 공동 미션 일정 수 완료 | +1  |
| 추리 이벤트 성공     | +1  |
| 최대 증가         |     |

#### 탈출구 개방 조건

| 조건           | 필요 여부 |
|--------------|-------|
| 게임 시간 ⅔ 경과   | O     |
| 탈출구 개방 미션 완료 | O     |

#### 탈출 가능 조건

| 조건     | 필요 여부 |
|--------|-------|
| 시민 역할  | O     |
| 생존 상태  | O     |
| 탈출구 개방 | O     |

|             |   |
|-------------|---|
| 탈출 가능 인원 남음 | O |
| 탈출 구역 진입    | O |

#### 탈출 처리 순서

| 순서 | 처리              |
|----|-----------------|
| 1  | 탈출 구역 진입        |
| 2  | 탈출 가능 여부 검사     |
| 3  | 탈출 성공 판정        |
| 4  | 탈출 가능 인원 차감     |
| 5  | 플레이어 상태를 탈출로 변경 |
| 6  | 관전자 화면 전환       |
| 7  | 게임 종료 조건 검사     |

#### 탈출 성공

- 해당 시민 즉시 승리
- 게임 플레이 종료
- 탈출 상태로 변경
- 생존자 음성 채널 유지
- 자유 이동 불가
- 관전자 화면 전환

#### 탈출 플레이어

- 게임에 다시 참여 불가
- 자유 이동 불가
- 생존자의 화면만 관전 가능
- 관전자, 생존자 음성 채널에 참가 불가능
- 생존자에게 자신의 음성을 전달할 수 없다

#### 서버 판정

- 탈출 가능 인원
- 현재 탈출 인원
- 탈출구 개방 여부
- 탈출 미션 완료 여부
- 플레이어 탈출 여부
- 탈출 성공 여부
- 게임 종료 여부

#### 상태 변화

| 상태 | 이동 | 미션 | 음성     | 관전 |
|----|----|----|--------|----|
| 생존 | O  | O  | 생존자 채널 | X  |

|    |   |   |                    |   |
|----|---|---|--------------------|---|
| 탈출 | X | X | 송신 X / 생존자 음성 수신 O | O |
| 사망 | X | X | 관전자 채널             | O |

#### Network Variable

- EscapeOpened
- EscapeCapacity
- EscapeCount
- IsEscaped
- EscapeAvailable

#### RPC

- OpenEscape()
- RequestEscape()
- EscapeSuccess()
- EscapeFailed()
- UpdateEscapeCapacity()
- EnterEscapeSpectator()

#### 필요한 클래스

- EscapeManager
- EscapeGateManager
- EscapeCapacityManager
- EscapeValidator
- EscapeMissionManager
- EscapeResultManager

#### 예외처리

탈출 가능 인원 초과 -> 탈출 실패

탈출 직전 사망 -> 사망 처리 우선

탈출구 미개방 -> 탈출 실패

탈출 미션 미완료 -> 탈출 실패

탈출 중 Disconnect -> 서버 판정 후 상태 저장

마지막 탈출 인원 사용 -> 탈출 종료 및 추가 탈출 불가

탈출 후 재접속 -> 탈출 상태 복원

승리 가능한 플레이어가 더 이상 없음 -> 즉시 게임 종료

#### 서버 / 클라 분리

| 클라        | 서버          |
|-----------|-------------|
| 탈출구 UI 표시 | 탈출구 개방      |
| 상호작용 입력   | 탈출 가능 여부 판정 |
| 탈출 애니메이션  | 탈출 성공 판정    |
| 탈출 HUD 표시 | 탈출 가능 인원 관리 |

|             |             |
|-------------|-------------|
| 관전자 화면 전환   | 플레이어 상태 변경  |
| 알림 및 이펙트 출력 | 게임 종료 조건 검사 |

# 역할 시스템



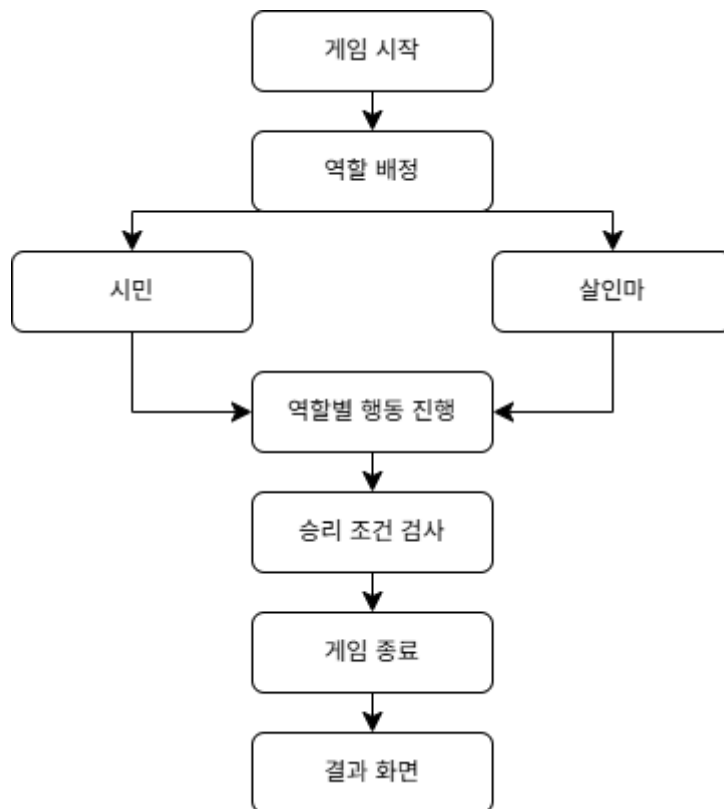
### 시스템 목적

- 역할 시스템은 게임 시작 시 플레이어에게 역할을 부여하고, 각 역할의 승리 조건을 관리하는 시스템이다
- 서버는 역할에 따른 행동 가능 여부와 승리, 패배를 최종 판정하며, 모든 클라이언트는 서버의 결과를 동기화한다

### 시스템 역할

- 역할 배정
- 역할 정보 관리
- 역할별 행동 제한
- 승리 조건 판정
- 패배 조건 판정
- 탈출 판정
- 게임 종료 판정

### 시스템 구조



### 역할 종류

| 역할  | 설명                 |
|-----|--------------------|
| 시민  | 미션 수행 또는 탈출을 통해 승리 |
| 살인마 | 시민을 제거하여 승리        |

### 시민 승리 조건

- 방법1: 승리 미션
  - 공동 미션 100% 완료

- 개인 미션 완료
- 두 조건을 모두 만족한 시민만 승리한다
- 방법2 : 탈출 승리
  - 탈출 가능한 인원은 제한된다
  - 탈출한 플레이어만 승리한다
  - 다른 시민은 계속 게임을 진행한다(누군가가 승리 조건이 있을 시에만)
  - 남은 사람들 중에 승리할 수 있는 조건이 없다면 게임은 끝난다

#### 개인 행동 목표

- 개인 행동 목표는 공동 승리 조건이 아니다
- 게임 종료 후 성공 여부로 개인의 패배 / 승리만 판단한다

| 결과 | 처리       |
|----|----------|
| 성공 | 개인 목표 성공 |
| 실패 | 개인 목표 실패 |

#### 살인마 승리 조건

- 시민 전원 사망

#### 살인마 미션

- 수행 여부와 관계없이 승패에는 영향을 주지 않는다
- 수행 시 시민을 즉사시킬 수 있는 스킬이 열린다

#### 승리 판정

- 시민

| 공동 미션 | 개인 미션 | 탈출 | 결과         |
|-------|-------|----|------------|
| O     | O     | X  | 시민 전부 승리승리 |
| O     | X     | X  | 패배         |
| X     | O     | X  | 패배         |
| X     | X     | X  | 패배         |
| 무관    | 무관    | O  | 탈출 플레이어 승리 |

#### 사망한 시민

- 공동 미션 수행 불가
- 개인 미션 수행 가능
- 게임 종료시 공통 미션과 개인 미션을 모두 만족하면 승리한다

#### 서버 판정

- 역할
- 생존 여부
- 탈출 여부
- 공통 미션 진행도

- 개인 미션 진행도
- 개인 행동 목표
- 살인마 미션
- 승리 여부
- 패배 여부
- 게임 종료

#### 역할 상태

| 상태 | 시민    | 살인마       |
|----|-------|-----------|
| 생존 | O     | O         |
| 사망 | O(관전) | O(관전)     |
| 탈출 | O(관전) | X(탈출 불가능) |

#### 게임 종료 조건

- 시민 전원 승패 확정
- 살인마 승패 확정
- 제한 시간 종료
- 더 이상 승리 가능한 플레이어가 존재하지 않음

#### Network Variable

- Role
- IsAlive
- IsEscaped
- TeamMissionProgress
- PersonalMissionProgress
- PersonalGoalProgress
- Winner
- GameState

#### RPC

- AssignRole()
- UpdateMission()
- EscapePlayer()
- UpdateWinner()
- GameEnd()
- RevealResult()

#### 필요한 클래스

- RoleManager
- MissionManager
- EscapeManager
- WinConditionManager
- GameManager
- ResultManager

#### 예외처리

시민이 공통 미션만 완료 -> 패배

시민이 개인 미션만 완료 -> 패배

시민이 공통, 개인 미션 모두 완료 -> 승리

시민이 탈출 -> 탈출한 시민만 즉시 승리

시민 사망 후 개인 미션 완료 가능 -> 개인 미션만 진행 가능(사망 전 수행했던 미션은 그대로 완료처리)

시민 사망 후 공통 미션 미완료 -> 승리 불가(살아있는 사람들이 공통 미션 전부 완료해야함)

개인 행동 목표 실패 -> 게임 승패에는 영향 없음(개인의 승패에만 영향 있음)

제한 시간 종료 -> 서버가 승리 조건을 검사하여 결과 판정

승리 가능한 플레이어가 존재하지 않음 -> 즉시 게임 종료 후 결과 화면

#### 클라 / 서버 분리

| 클라            | 서버       |
|---------------|----------|
| 역할 UI 표시      | 역할 배정    |
| 역할 설명 표시      | 역할 정보 관리 |
| 역할별 HUD 변경    | 승리 조건 판정 |
| 역할 상태 UI 갱신   | 패배 조건 판정 |
| 결과 화면 표시      | 게임 종료 판정 |
| 미션 및 탈출 정보 표시 | 탈출 여부 판정 |

# 살인 시스템

### 시스템 목적

- 범인이 특정 조건을 만족한 후 시민을 공격하여 제거하는 시스템
- 살인은 서버가 최종 판정을 담당하여 모든 클라이언트는 결과만 전달받는다

### 살인 가능 조건

- 살인마 역할
- 살인 가능 상태
- 살인 쿨타임 종료
- 살인자 미션 완료
- 대상 생존
- 공격 범위 내
- 칼 소지 상태

### 시스템 구조



#### 서버 판정, 서버 확인 항목

- 살인마 여부
- **Alive**
- **Knife**
- 거리
- 쿨타임
- **Target Alive**
- **Kill** 가능 상태
- **Kill Lock**

#### 공격 범위

- **2m**

#### 클라이언트 처리

- 공격 버튼 입력
- 공격 애니메이션
- 효과음
- 카메라 흔들림

#### 살인 성공 시

- 사망 -> 증거 생성 -> 시체 생성 -> 살인 쿨타임 시작 -> 로그 기록

#### 살인 실패

- 거리 부족
- 벽 사이
- 이미 사망
- 무적 상태
- 탈출

#### 예외 처리

- 대상이 탈출 -> 살인 실패
- 대상이 회복
  - 사망 전 -> 회복 가능
  - 사망 후 -> 회복 불가능
- 살인 중 **Disconnect** -> 서버 판정 완료 -> 사망처리
- **Ping** 차이 -> **Server Authority**
- 두 살인마 -> 동일 대상 -> 먼저 승인 -> 후순위 실패
- 증거 생성
  - 살인 성공 시 서버는 생성 여부를 판정한다
  - **Blood**
  - **Footprint**
  - **Body**
  - **Weapon Trace**

#### Network Variable

- **Alive**
- **Dead**
- **KillCooldown**

- CanKill
- CurrentTarget

#### RPC

- RequestKill()
- KillSuccess()
- KillFailed()
- SpawnBody()
- SpawnEvidence()

#### 필요한 클래스

- KillManager
- PlayerHealth
- DeathManager
- EvidenceManager
- CooldownManager

#### 서버 / 클라 분리

| 클라           | 서버          |
|--------------|-------------|
| 공격 입력 처리     | 살인 가능 여부 판정 |
| 공격 애니메이션     | 공격 거리 검사    |
| 공격 효과음       | 대상 생존 여부 확인 |
| 카메라 흔들림      | 살인 성공/실패 판정 |
| 공격 성공/실패 연출  | 사망 처리       |
| 쿨타임 UI 표시    | 쿨타임 관리      |
| 공격 버튼 활성화 표시 | 증거 및 시체 생성  |
| 칼 모델 표시      | 시체 생성       |
| 사망 애니메이션 재생  | 증거 생성       |

#### 칼 사용 조건

- 칼이 바닥에 존재
- 플레이어가 칼 획득 가능 상태
- 칼 획득 후 플레이어가 칼을 소지
- 칼이 이미 다른 플레이어에게 획득되지 않음
- 칼 사용 가능 상태

#### 칼 시스템

- 칼 생성
  - 게임 내 특정 위치 또는 조건에 따라 칼 생성
  - 칼은 월드에 실제 오브젝트로 존재
  - 모든 클라이언트에게 동일한 위치와 상태로 동기화



- 서버 확인
  - 칼 생성 여부
  - 칼 위치
  - 칼 획득 상태
  - 현재 소지자
  - 칼 사용 가능 여부
- 칼 획득
  - 플레이어가 바닥의 칼에 접근하여 상호작용하면 서버에 획득 요청을 보낸다
  - 획득 조건
    - 플레이어 **Alive**
    - 플레이어가 칼 획득 가능 상태
    - 칼이 월드에 존재
    - 칼이 다른 플레이어에게 획득되지 않음
    - 플레이어와 칼의 거리 조건 만족
- 공격 범위
  - **2m**(서버에서 플레이어와 대상의 실제 거리를 계산하여 판정한다)
  - 벽 사이 공격 불가
  - 대상 **Alive** 확인
  - 대상 탈출 상태 확인
  - 공격자 **Alive** 확인
- 살인 성공
  - 서버에서 살인 성공을 확정하면 다음 순서로 처리한다
  - **KillSuccess**
  - ↓
  - **Target HP = 0**
  - ↓
  - **Alive = false**
  - ↓
  - **Dead = true**
  - ↓
  - **DeathManager** 호출
  - ↓
  - 시체 생성
  - ↓
  - 증거 생성
  - ↓
  - 살인 쿨타임 시작
  - ↓
  - 살인 로그 기록
- 생성되는 요소
  - 시체
  - **Blood**
  - **Footprint**
  - **Weapon Trace**
- 살인 실패
  - 칼을 소지하지 않음

- 살인마가 아님
  - 살인 가능 상태가 아님
  - 살인 쿨타임 중
  - 살인자 미션 미완료
  - 공격 거리 부족
  - 벽으로 막혀 있음
  - 대상이 이미 사망
  - 대상이 탈출함
  - 공격자가 사망함
  - 무적 상태
- 예외 처리
    -

# 로비/방 시스템

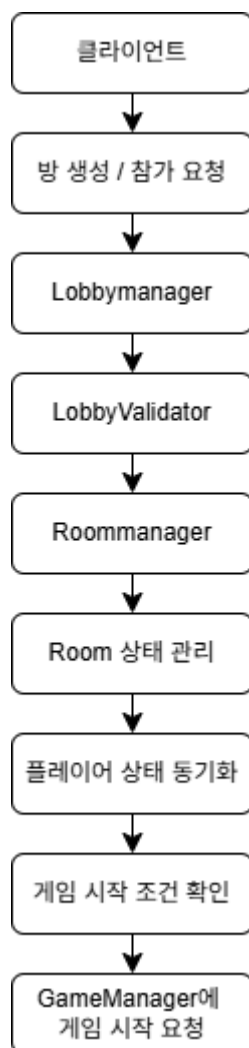
### 시스템 목적

- 로비 시스템은 플레이어가 게임 시작 전 방을 생성하고 다른 플레이어와 함께 대기하며 게임을 시작할 수 있도록 관리하는 시스템

### 시스템 역할

- 방 생성
- 방 삭제
- 방 참가
- 방 나가기
- 플레이어 목록 관리
- 방장 관리
- 준비 상태 관리
- 최대 인원 관리
- 게임 시작 가능 여부 확인
- 게임 시작 요청 처리
- 로비 상태 동기화
- 게임 씬 전환 요청
- 비정상 종료 처리

### 시스템 구조

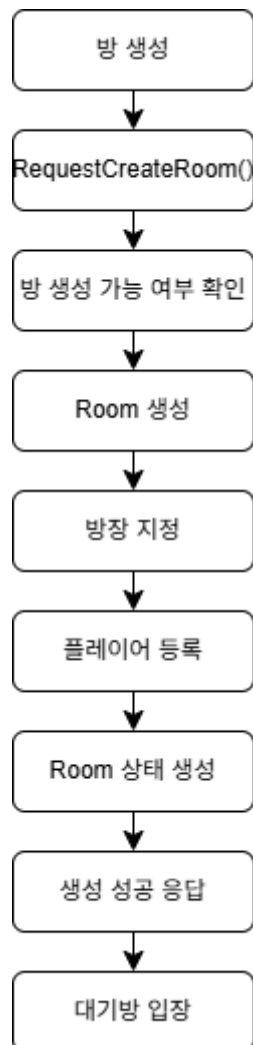


## 방 기본 설정

| 항목         | 내용           |
|------------|--------------|
| 최소 인원      | 2명           |
| 최대 인원      | 게임 설정에 따라 결정 |
| 방장         | 방 생성자        |
| 방 공개 여부    | 공개 / 비공개     |
| 준비 상태      | 플레이어별 관리     |
| 게임 시작      | 방장만 가능       |
| 방 참가       | 게임 시작 전만 가능  |
| 게임 진행 중 참가 | 불가능          |

## 방 생성

- 플레이어가 로비에서 방 만들기를 선택하면 서버에 방 생성 요청을 보낸다



#### 방 정보

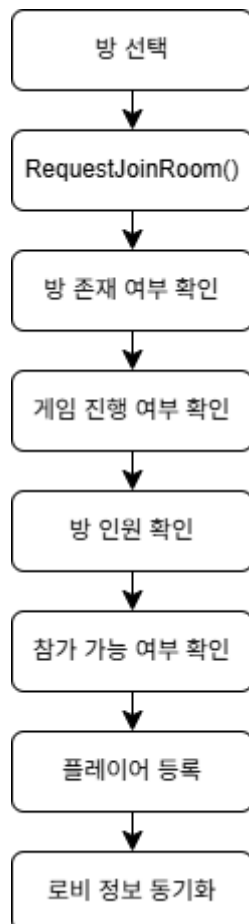
- RoomID
- RoomName
- HostPlayerID
- MaxPlayerCount
- CurrentPlayerCount
- RoomState
- IsPrivate
- PlayerList
- ReadyState
- GameSettings

#### RoomState

- Wating
- Starting
- Playing
- Ending
- Closed

#### 방 참가

- 플레이어가 방 목록에서 특정 방을 선택하면 서버에 참가 요청을 보낸다

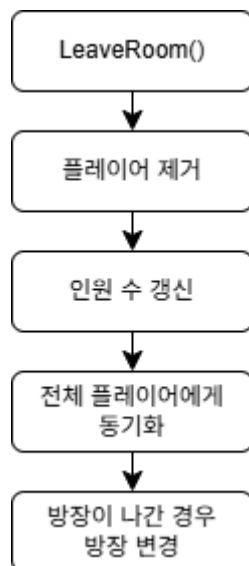


#### 방 참가 조건

| 조건             | 참가 |
|----------------|----|
| 방 존재           | O  |
| 게임 시작 전        | O  |
| 방 정원 미만        | O  |
| 플레이어가 다른 방에 없음 | O  |
| 게임 진행 중        | X  |
| 방이 가득 참        | X  |
| 방이 종료 중        | X  |

#### 방 나가기

- 플레이어가 로비에서 나가기를 선택하면 서버에서 해당 플레이어를 방에서 제거한다



#### 방장 시스템

- 방 생성자는 방장이 된다

방장은 다음 권한을 가진다

- 게임 시작
- 게임 설정 변경
- 플레이어 강제 퇴장
- 방 삭제

#### 방장 이탈

- 방장이 방을 나가면 서버가 남은 플레이어 중 한 명을 새로운 방장으로 지정한다



- 새 방장 선정 기준은 입장 순서가 가장 빠른 플레이어로 설정

플레이어 준비 상태

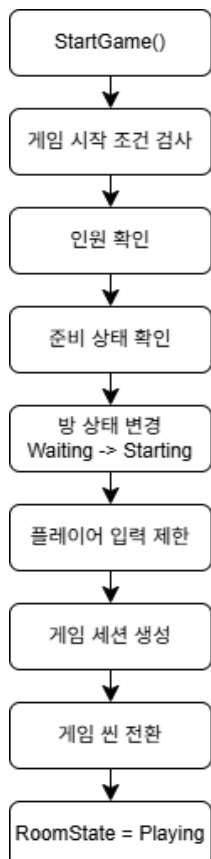
- **Not Ready -> Ready -> Not Ready**

게임 시작 조건

- 최소 인원 충족 + 모든 플레이어 **Ready** + 방장이 게임 시작 요청 = 게임 시작 가능

게임 시작

- 방장이 게임 시작 버튼을 누르면 서버에 요청한다
- 서버가 조건을 확인한 후에만 게임을 시작한다





### 게임 시작 시 로비 처리

- 게임이 시작되면 해당 방은 더 이상 일반적인 로비 참가를 허용하지 않는다
- **Playing** 상태에서는 새로운 플레이어가 해당 방에 참가할 수 없다
- **Lobby -> Starting -> Game**

### 게임 씬 관리

- 로비 서버는 게임 씬 자체의 게임 플레이를 담당하지 않는다



- 로비 서버 -> 게임을 시작해도 되는가?
- 게임 서버 -> 게임을 어떻게 진행하는가?

### 로비 상태 동기화

- 서버는 로비에 있는 모든 플레이어에게 방의 상태를 동기화한다

RoomID

RoomName

HostPlayerID

PlayerList

CurrentPlayerCount

MaxPlayerCount

ReadyState

RoomState

GameSettings

### Network Variable

- RoomID
- HostPlayerID
- CurrentPlayerCount
- MaxPlayerCount
- RoomState
- IsPrivate
- ReadyState
- LobbyPosition
- LobbyRotation
- LobbyPlayerState

- IsReady
- IsHost

#### Player 정보

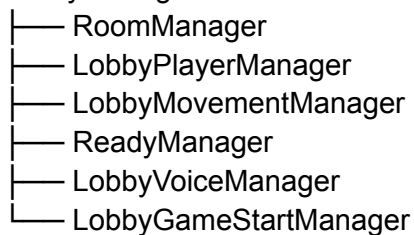
- PlayerID
- playerName
- IsReady
- IsHost

#### RPC

- RequestCreateRoom()
- CreateRoomSuccess()
- CreateRoomFailed()
- RequestJoinRoom()
- JoinRoomSuccess()
- JoinRoomFailed()
- RequestLeaveRoom()
- LeaveRoomSuccess()
- SetReady()
- SyncReadyState()
- ChangeHost()
- SyncHost()
- KickPlayer()
- PlayerKicked()
- RequestStartGame()
- StartGame()
- StartGameFailed()
- SyncRoomState()
- SyncPlayerList()
- SpawnLobbyPlayer()
- DespawnLobbyPlayer()
- UpdateLobbyPosition()
- UpdateLobbyRotation()
- JoinLobbyVoice()
- LeaveLobbyVoice()
- SyncLobbyPlayerState()
- StartGame()

#### 필요한 클래스

##### LobbyManager



#### 서버 / 클라이언트 역할 분리

- 클라이언트가 방 인원이나 게임 시작 가능 여부를 최종적으로 판단한지 않는다

|            |             |
|------------|-------------|
| 클라         | 서버          |
| 방 생성 UI    | 방 생성        |
| 방 목록 UI    | 방 삭제        |
| 방 선택       | 방 참가 판정     |
| 나가기 버튼     | 방 나가기 처리    |
| 방장 표시      | 방장 결정       |
| Ready 버튼   | 준비 상태 저장    |
| 게임 시작 버튼   | 게임 시작 조건 판정 |
| 플레이어 목록 표시 | 인원 관리       |
| 로비 UI 갱신   | 로비 상태 동기화   |
| 씬 전환 연출    | 게임 시작 승인    |

예외 처리

방이 가득 찬 경우

- JoinRoom 요청 -> CurrentPlayerCount >= MaxPlayerCount -> 참가 거부 -> RoomFull

이미 다른 방에 있는 경우

- JoinRoom 요청 -> 현재 다른 Room에 존재 -> 참가 거부

방장이 게임 시작 중 퇴장

- StartGame -> 방장 Disconnect -> 게임 시작 취소 -> 새 방장 지정 -> Wating 상태 복귀

일반 플레이어 Disconnect

- Disconnect -> 방에서 플레이어 제거 -> 인원 수 감소 -> 전체 플레이어에게 동기화

방장이 나가고 아무도 없는 경우

- Host Leave -> 남은 플레이어 없음 -> Room 삭제

게임 시작 직전 인원 감소

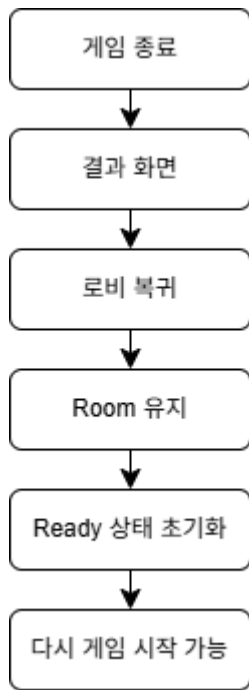
- StartGame 요청 -> 플레이어 Disconnect -> 서버 재검사 -> 최소 인원 미달 -> 게임 시작 취소

이미 게임이 시작된 방에 참가

- JoinRoom -> RoomState == Playing -> 참가 거부

방 삭제 조건

- 모든 플레이어가 퇴장
- 방장이 방을 삭제
- 서버에서 방 종료
- 비정상적으로 빈 방이 된 경우



#### 로비 위치 동기화

- 클라이언트 이동 -> 서버 위치 검증 -> 로비 플레이어 위치 저장 -> 다른 클라이언트에게 동기화

#### 다른 플레이어 화면 :

- **Player A** -> 서버에서 받은 **Position** -> **Player A** 캐릭터 이동

#### 로비 음성 채널

- **LobbyVoiceChannel**
- 같은 방인지 확인
- 로비 상태인지 확인
- 음성 송수신 가능 여부
- 방을 나가면 음성 채널 제거
- 게임 시작 시 로비 음성 채널 종료

#### 흐름

- 방 참가 -> **LobbyVoiceChannel** 참가 -> 플레이어 간 음성 통신 -> 게임 시작 -> **LobbyVoiceChannel** 종료 -> **GameVoiceChannel** 참가

#### 로비 플레이어 Spawn

- 방에 들어오면 서버가 플레이어의 로비 **Spawn** 위치를 결정
- **JoinRoom** -> **Lobby Spawn Point** 결정 -> **Network Player** 생성 -> 모든 클라이언트에 동기화

#### 플레이어가 나가면

- **Leave Room** -> **Network Player** 제거 -> **PlayerList** 갱신 -> 다른 클라이언트에 동기화

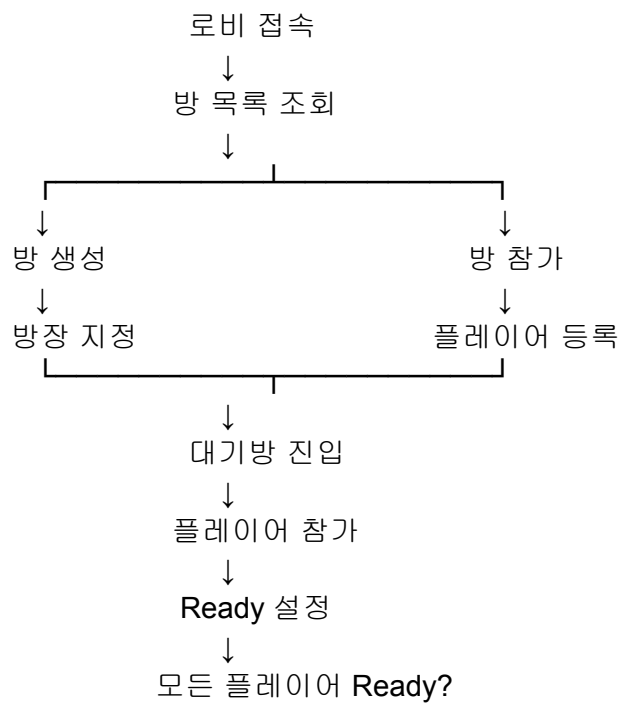
#### Ready 상태와 실제 캐릭터 상태 연결

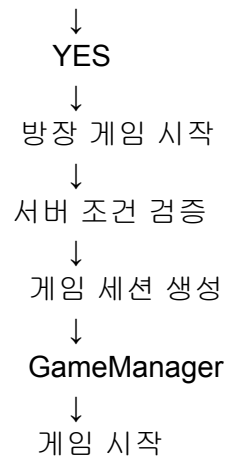
- **Ready** -> 서버 상태 변경 -> 모든 클라이언트에 동기화 -> 캐릭터 머리 위 **READY** 표시
- **Ready** 상태에서도 이동/음성은 가능

## 게임 시작 시 처리



## 전체 흐름





#### 서버 / 클라 역할 분리

| 기능          | 클라 | 서버 |
|-------------|----|----|
| 방 이름 입력     | O  |    |
| 방 생성 UI     | O  |    |
| 방 생성 판정     |    | O  |
| 방 참가 UI     | O  |    |
| 방 참가 판정     |    | O  |
| 3D 캐릭터 표시   | O  |    |
| 로비 이동 입력    | O  |    |
| 로비 위치 동기화   | O  | O  |
| Ready 버튼    | O  |    |
| Ready 상태 저장 |    | O  |
| 방장 표시       | O  | O  |
| 음성 입력 / 재생  | O  |    |
| 음성 채널 관리    |    | O  |
| 게임 시작 버튼    | O  |    |
| 게임 시작 판정    |    | O  |
| 로딩 화면       | O  |    |
| 게임 쉼 전환 승인  |    | O  |

# 초대 시스템

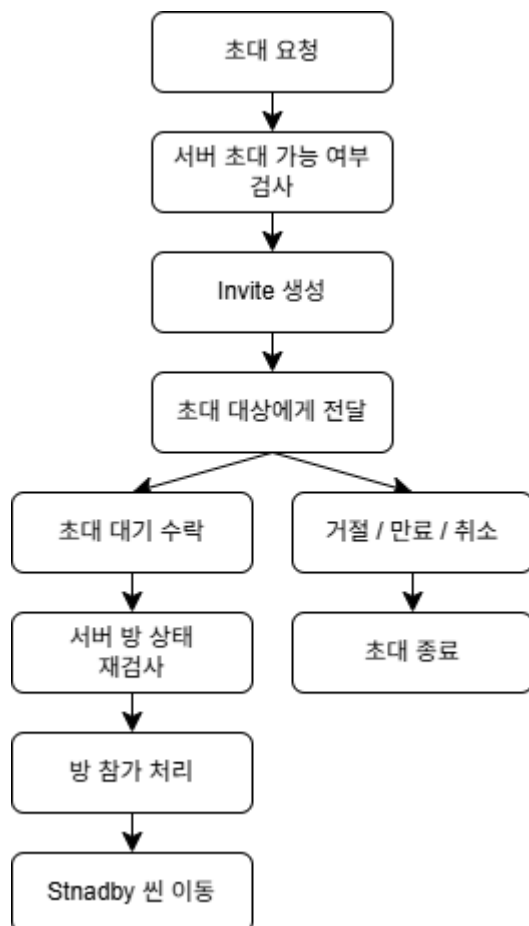
### 시스템 목적

- 초대 시스템은 플레이어가 현재 참가 중인 대기방에 다른 플레이어를 초대할 수 있도록 관리하는 시스템이다
- 서버는 초대 요청, 대상 플레이어 상태, 방 상태 및 인원 제한을 검증하고, 초대 수락 시 최종적으로 방 참가 여부를 판정한다

### 시스템 역할

- 초대 요청 검증
- 초대 가능 여부 확인
- 초대 데이터 생성
- 초대 대상에게 초대 전달
- 초대 수락 / 거절 처리
- 초대 만료 관리
- 초대 수락 시 방 상태 재검증
- 방 참가 처리
- 중복 초대 방지
- 게임 시작 시 기존 초대 종료

### 시스템 구조



### 초대 요청 처리

- 플레이어가 다른 플레이어를 초대하면 클라이언트는 서버에 초대 요청을 보낸다
- Player A -> Player B -> RequestInvite(Player B)



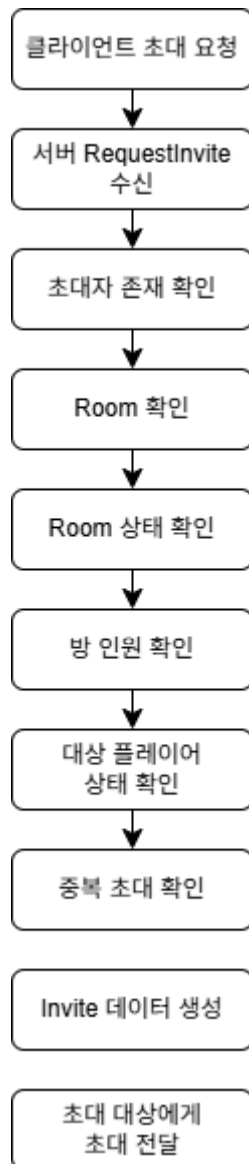
#### 서버 확인 항목

- 초대자 연결 상태
- 초대자 현재 **Room** 존재
- 방 상태
- 방 최대 인원
- 대상 플레이어 존재
- 대상 플레이어 온라인
- 대상 플레이어 게임 진행 여부
- 대상 플레이어 다른 방 참가 여부
- 중복 초대 여부

#### 초대 가능 조건

- 조건을 하나라도 만족하지 못하면 초대 요청은 거부된다
- 초대자가 방에 참가중
- 방 상태가 대기 중
- 게임 시작 전
- 현재 인원이 최대 인원 미만
- 초대 대상 온라인
- 초대 대상 게임 중 아님
- 초대 대상 다른 방에 없음
- 동일한 초대가 존재하지 않음

#### 초대 요청 처리 순서



#### 초대 데이터

| 데이터            | 설명       |
|----------------|----------|
| InviteID       | 초대 고유 ID |
| InviteID       | 초대한 플레이어 |
| TargetPlayerID | 초대 대상    |
| RoomID         | 초대 대상 방  |
| InviteState    | 초대 상태    |
| CreateTime     | 생성 시간    |
| ExpireTime     | 만료 시간    |

#### 초대 상태

- 초대 생성 시 기본 상태는 Pending

| 상태        | 설명    |
|-----------|-------|
| Pending   | 초대 대기 |
| Accepted  | 초대 수락 |
| Declined  | 초대 거절 |
| Expired   | 초대 만료 |
| Cancelled | 초대 취소 |
| Failed    | 초대 실패 |

#### 초대 수락

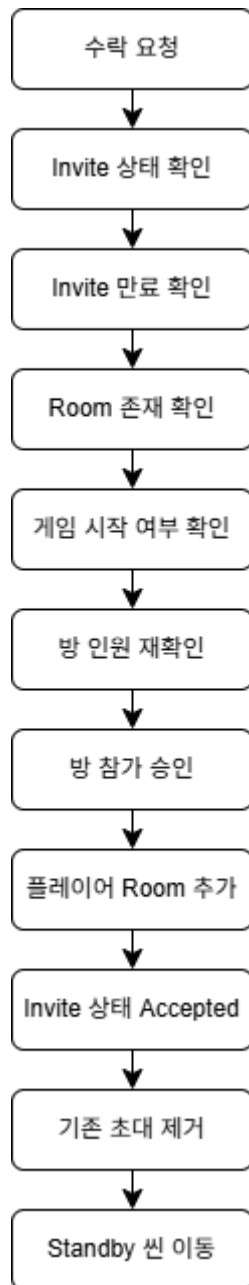
- 초대 수락 -> RequestAcceptInvite(InviteID) -> 서버 검증

#### 수락 시 서버 재검증

- 초대가 생성된 이후 방 상태가 변경되었을 수 있으므로 다시 확인한다

| 항목           | 확인      |
|--------------|---------|
| Invite 존재 여부 | O       |
| Invite 상태    | Pending |
| Invite 만료 여부 | O       |
| 대상 플레이어 일치   | O       |
| Room 존재 여부   | O       |
| Room 상태      | Waiting |
| 게임 시작 여부     | O       |
| 방 최대 인원      | O       |

#### 초대 수락 처리



방 참가 성공 후 처리

- 초대를 받아 참가한 플레이어는 기본적으로
  - **Ready = false** 로 시작한다



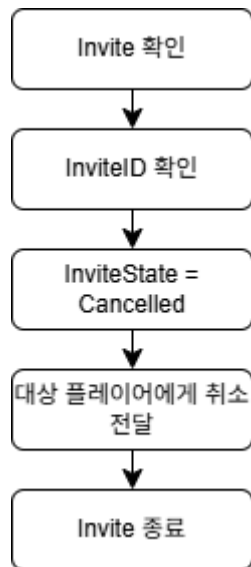
#### 초대 거절

- 초대 대상이 거절하면 서버에 거절 요청을 보낸다
- RequestDeclineInvite(InviteID)



#### 초대 취소

- 초대한 플레이어가 초대를 취소할 수 있다
- CancelInvite(InviteID)



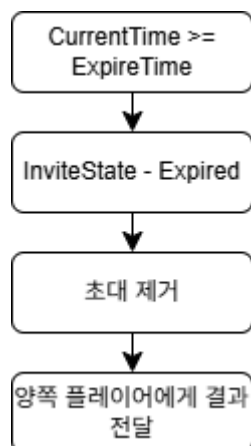
#### 초대 만료

- 초대는 일정 시간이 지나면 자동으로 만료된다(5분)



#### 서버 관리

- 서버는 초대 생성 시 ExpireTime을 저장한다



#### 게임 시작 시 처리

- 방장이 게임을 시작하면 해당 방과 관련된 모든 대기 중 초대를 종료한다



#### 초대자 방 퇴장

- 초대자가 방을 나가는 경우



#### 방 삭제

- 방에 아무도 남지 않아 방이 삭제 되는 경우



## 대상 플레이어 상태 변경

| 상황         | 처리       |
|------------|----------|
| 다른 방 참가    | 초대 취소    |
| 게임 시작      | 초대 취소    |
| Disconnect | 초대 취소    |
| 다른 초대 수락   | 기존 초대 취소 |

## 중복 초대 처리

- 동일한 플레이어에게 여러 번 초대 요청하는 것을 방지한다
- 또한 여러 방에서 동시에 초대를 받을 수는 있지만, 하나를 수락하면 나머지 초대는 자동 취소한다
- Player A -> Player B



## 서버 판정

- Invite 생성
- InviteID
- 초대 상태
- 초대 만료 시간
- 초대자
- 초대 대상
- RoomID
- 방 참가 가능 여부
- 방 최대 인원
- 게임 시작 여부
- 초대 수락 최종 판정

## RPC

- RequestInvite()
- ReceiveInvite()
- AcceptInvite()
- DeclineInvite()
- CancelInvite()
- InviteResult()



## Network Variable

- CurrentRoomID
- RoomState
- CurrentPlayerCount
- MaxPlayerCount

## 필요한 클래스

- InviteManager
  - InviteValidator
  - InviteDataManager
  - InviteResponseManager
  - InviteTimeoutManager
  - InviteCleanupManager

## 예외처리

- 초대자가 방에 없음 -> 초대 실패
- 방이 가득 참 -> 초대 실패
- 게임 시작 후 초대 -> 초대 불가
- 대상 플레이어 오프라인 -> 초대 실패
- 대상이 다른 방에 있음 -> 초대 불가
- 동일 대상 중복 초대 -> 기존 초대 유지
- 초대 만료 -> 자동 취소
- 초대자가 방 퇴장 -> 초대 취소
- 방 삭제 -> 관련 초대 전체 취소
- 초대 수락 시 방이 가득 참 -> 참가 실패
- 초대 수락 직전 게임 시작 -> 참가 실패
- 여러 초대 중 하나 수락 -> 나머지 초대 자동 취소
- 초대 대상 Disconnect -> 초대 취소
- 초대자 Disconnect -> 초대 취소
- 동시에 마지막 자리 수락 -> 서버에서 먼저 승인된 플레이어만 참가

## 초대 코드 생성

- 방이 생성될 때 서버에서 초대 코드를 생성한다

## 방 생성 조건

- 모든 방은 고유한 초대 코드를 가진다
- 이미 사용 중인 코드는 중복되지 않아야한다
- 초대 코드는 서버에서 생성한다
- 클라이언트가 임의로 코드를 생성할 수 없다
- 방 생성 -> Room ID : Room34 -> Invite Code : qwe45h6b

## 초대 코드 구성

- 초대 코드는 플레이어가 직접 전달하기 쉽게 문자열로 구성한다
- 초대코드는 알파벳, 숫자를 섞어 8자리로 만든다
- 조건

| 항목    | 내용            |
|-------|---------------|
| 생성 주체 | 서버            |
| 중복    | 불가능           |
| 대소문자  | 동일하게 처리하도록 설정 |
| 특수문자  | 사용하지 않음       |
| 숫자    | 사용            |
| 길이    | 8자            |
| 유효 방  | 하나의 방에 하나의 코드 |

#### 초대 코드와 방의 연결

- 서버는 생성된 초대 코드를 특정 방과 연결하여 관리한다
- 초대 코드를 통해 직접 플레이어를 찾는 것이 아니라
  - 초대 코드 -> 방 -> 방 찾기 순서로 처리한다

#### 초대 코드 입력 및 참가 요청

- 플레이어가 초대 코드를 입력하면 서버에 참가 요청을 전달한다
- 플레이어 코드 입력 -> **RequestJoinByCode()** -> 서버 초대 코드 검색 -> 코드 존재 여부 확인 -> 방 상태 확인 -> 인원 확인 -> 참가 가능 여부 판정

#### 초대 코드 검증

- 서버는 참가 요청을 받으면 다음 조건을 검사한다
  - 코드 존재 여부
  - 코드 유효 여부
  - 방 존재 여부
  - 방 상태
  - 방 인원 제한
  - 참가자 중복 여부
  - 게임 시작 여부
  - 참가자 상태

#### 방 참가 가능 조건

- 초대 코드 존재 -> 방 존재 -> 방 참가 가능 상태 -> 방 인원 제한 미초과 -> 게임 시작 전 -> 이미 참가한 플레이어가 아님 -> 방 참가 승인

#### 방 인원 제한

- 방에 설정된 최대 인원을 초과할 수 없다

# 미니맵, 지도 시스템

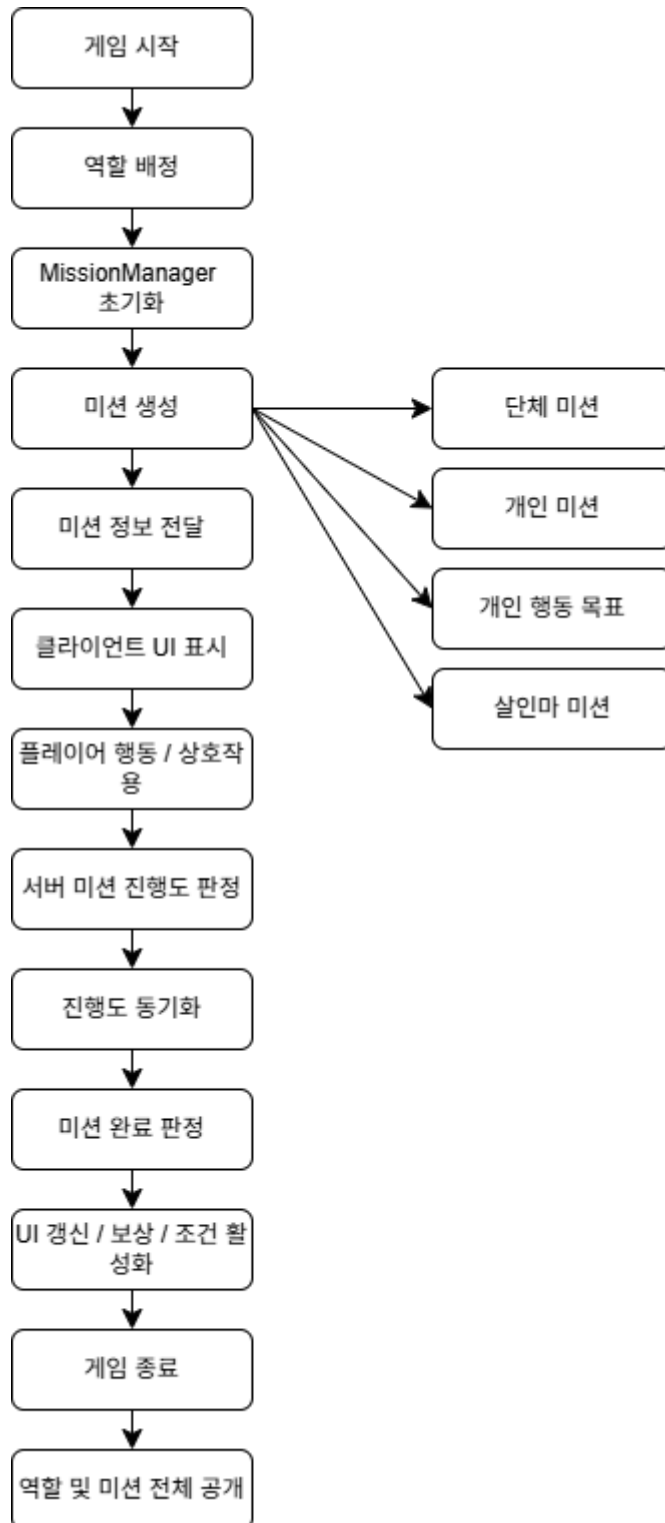


# 미션 시스템

## 시스템 목적

- 미션 시스템은 플레이어에게 다양한 행동 목표를 부여하여 게임의 진행을 유도하고, 추리와 심리전을 발생시키는 핵심 시스템이다
- 미션은 단체 미션, 개인미션, 살인마 미션으로 구분되며 플레이어는 자신의 역할에 맞는 목표를 수행한다

## 시스템 구성



## 시스템 역할

- 서버
  - 역할에 따른 미션 생성
  - 미션 **Pool** 관리
  - 랜덤 미션 선택
  - 미션 충돌 검사
  - 미션 진행도 관리
  - 미션 완료 판정
  - 단체 미션 공동 진행도 관리
  - 개인 미션 진행도 관리
  - 개인 행동 목표 성공 여부 관리
  - 살인마 미션 진행도 관리
  - 살인 가능 조건 관리
  - 거짓말 탐지기 사용 횟수 관리
  - 거짓말 탐지기 단서 생성
  - 탈출 조건 확인
  - 게임 종료 시 전체 미션 공개 정보 생성
- 클라이언트
  - 미션 **UI** 표시
  - 개인 미션 및 행동 목표 표시
  - 단체 미션 진행도 표시
  - 미션 상호작용 입력
  - 진행 바 출력
  - 미션 완료 연출
  - 효과음 재생
  - 거짓말 탐지기 **UI** 표시
  - 획득한 단서 표시
  - 게임 종료 시 전체 미션 결과 표시

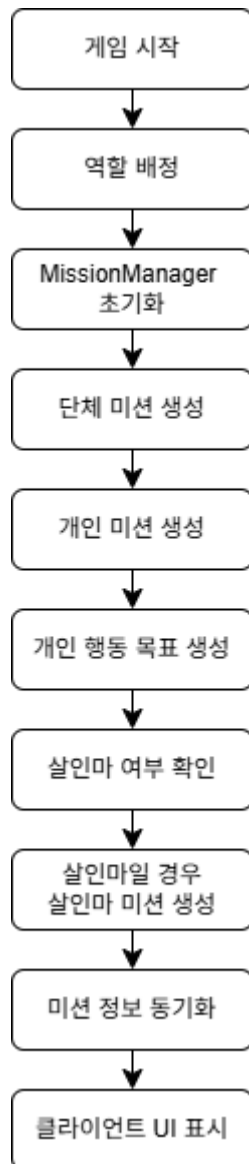
## 미션 종류

| 종류       | 대상      | 게임 영향         |
|----------|---------|---------------|
| 단체 미션    | 시민 전체   | 시민 승리 / 탈출 조건 |
| 개인 미션    | 시민 개인   | 시민 개인 승리 조건   |
| 개인 행동 목표 | 모든 플레이어 | 게임 승패와 별개     |
| 살인마 미션   | 살인마     | 살인 능력 활성화     |

## 미션 승리 조건

- 개인 행동 목표 성공 & 탈출 성공 = 개인 승리
- 개인 행동 목표 성공 & 개인 미션 완료 & 단체 미션 완료 = 시민 승리
- 개인 행동 목표를 미행했을 시 어떤 상황에 탈출을 해도 개인 패배

## 전체 미션 생성 흐름



#### 단체 미션

- 목적
  - 단체 미션은 시민 전체가 협력하여 수행하는 공동 목표이다
  - 시민들의 행동은 하나의 공동 진행도에 반영되며, 단체 미션 완료는 시민의 승리 조건 및 탈출 조건과 연결된다
- 시민 간 협력 유도
- 플레이어 이동 유도
- 특정 장소에서 플레이어 간 접촉 발생
- 살인마가 시민을 관찰하거나 공격할 기회 제공
- 게임 진행 속도 조절
- 시민 진영의 메인 목표 제공

#### 생성 방식

- 게임 시작 시 서버가 단체 미션 Pool에서 설정된 개수만큼 랜덤으로 선택한다
- TeamMission Pool
  - TM001 발전기 수리



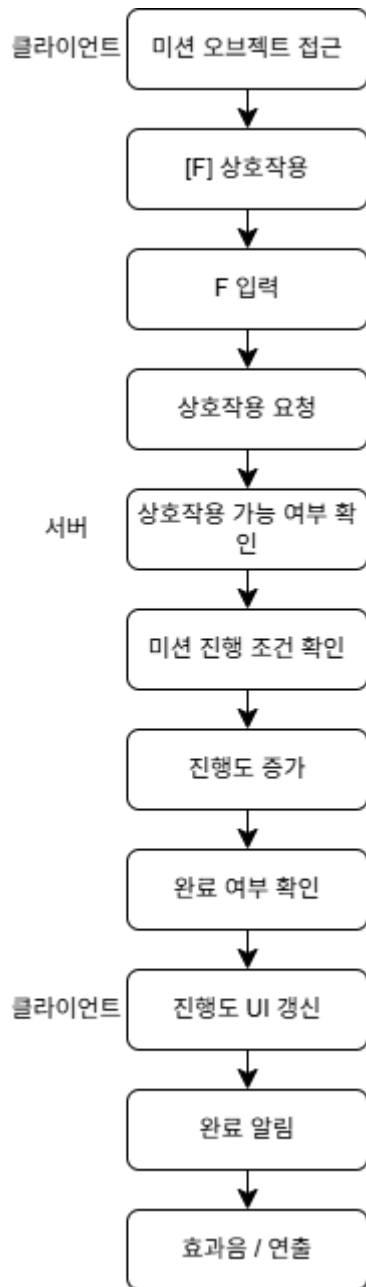
- TM002 전원 연결
- TM003 보안 시스템 복구
- TM004 서버 재부팅
- TM005 탈출구 개방
- TM006 냉각 시스템 복구
- TM007 통신 안테나 복구
- 예) Pool이 7개, 이번 게임에서 4개를 사용
  - 전체 Pool
    - 발전기 수리
    - 전원 연결
    - 보안 시스템 복구
    - 서버 재부팅
    - 탈출구 개방
    - 냉각 시스템 복구
    - 통신 안테나 복구
  - ↓
  - 랜덤 선택
    - 발전기 수리
    - 전원 연결
    - 탈출구 개방
    - 통신 안테나 복구
- 선택되지 않는 미션은 해당 게임에서 등장하지 않는다

#### 단체 미션 진행

- 각 미션은 고유한 완료 조건과 공동 진행도를 가진다
  - 예) 발전기 5대 수리
- 모든 시민의 행동이 하나의 진행도에 합산된다

| 플레이어    | 행동        | 진행도      |
|---------|-----------|----------|
| PlayerA | 발전기 1대 수리 | 1 / 5    |
| PlayerC | 발전기 1대 수리 | 2 / 5    |
| PlayerD | 발전기 2대 수리 | 4 / 5    |
| PlayerB | 발전기 1대 수리 | 5 / 5 완료 |

#### 처리 흐름



완료 처리



#### 개인 미션

- 목적
  - 개인 미션은 플레이어마다 서로 다른 목표를 제공하여 행동 경로와 플레이 패턴을 다양하게 만드는 시스템
  - 다른 플레이어는 해당 플레이어의 개인 미션을 확인할 수 없다
  - 게임 종료 후 모든 플레이어의 개인 미션이 공개된다

#### 생성 방식

- 서버는 역할 배정 후 개인 미션 **Pool**에서 플레이어별로 미션을 선택한다
- 개인 미션 **Pool**
  - PM001 발전기 수리 2회
  - PM002 의료 키트 사용
  - PM003 아이템 제작
  - PM004 의료실 방문
  - PM005 아이템 전달
  - PM006 전원 연결
  - PM007 서버 점검
  - PM008 문 개방

#### 진행 방식

- 플레이어 행동 -> 서버 조건 확인 -> 개인 미션 진행도 증가 -> 완료 여부 검사 -> 클라이언트 UI 갱신
- 예) 발전기 수리 0/2 -> 발전기 수리 성공 -> 1/2 -> 발전기 수리 성공 -> 2/2 -> 개인 미션 완료

#### 시민 승리 조건과의 관계

- 시민이 미션을 통한 승리를 얻기 위해서는 다음 조건을 모두 만족해야 한다
  - 단체 미션 완료 & 개인 미션 완료
- 두 조건을 모두 만족한 시민만 미션 승리 조건을 만족한다

#### 개인 행동 목표

- 목적
  - 개인 행동 목표는 게임의 메인 승리 조건과 직접적인 관련이 없는 숨겨진 목표이다
  - 플레이어가 평소와 다른 행동을 하도록 유도하여 다른 플레이어가 행동을 오해하거나 의심하도록 만드는 것이 목적이다
  - 예를 들어 시민이 행동 목표 때문에 이상한 행동을 하면서 살인마로 의심받을 수 있다

#### 예)

- 행동 제한
- 이동 관련
- 아이템 관련
- 추리 관련

#### 충돌 검사

- 개인 행동 목표는 개인 미션과 충돌해서는 안 된다
- 개인 미션 : 아이템 전달  
개인 행동 목표 : 누구에게도 아이템을 주지 말아라
- -> 충돌 발생

#### 처리

- 행동 목표 제외 -> 새로운 목표 선택 -> 충돌 검사 -> 문제 없음 -> 최종 지급
- 충돌 검사는 서버에서 수행한다

#### 완료 판정

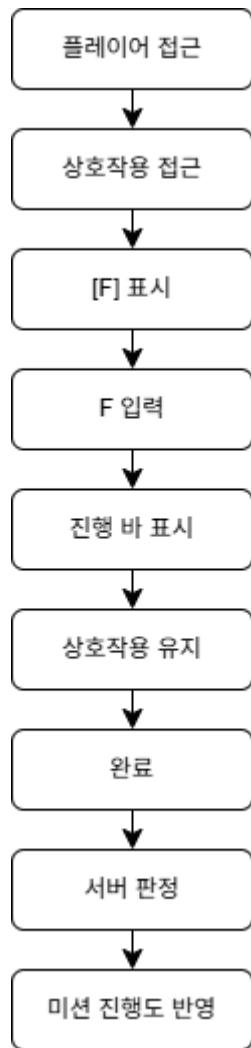
- 개인 행동 목표는 두 가지 방식으로 판정한다
- 실시간 판정
  - 조건을 만족하면 즉시 완료된다
  - 예)  
특정 플레이어와 3분 이상 함께 이동  
특정 지역 방문  
시체 최초 발견
- 게임 종료 판정
  - 게임 종료 시 전체 플레이 기록을 기준으로 판정한다
  - 예)  
한 번도 달리지 않기  
회복 아이템 끝까지 보관  
거짓말 탐지기 사용하지 않기

#### 살인마 미션

- 목적

#### 미션 상호작용

- 모든 미션 오브젝트는 기본적으로 동일한 상호작용 구조를 사용한다



#### 입력 가능 조건

| 조건           | 필요 여부 |
|--------------|-------|
| 생존 상태        | O     |
| 상호작용 가능 상태   | O     |
| 미션 오브젝트 범위 내 | O     |
| 다른 행동 중이 아님  | O     |
| 탈출 상태 아님     | O     |

#### 상호작용 취소 조건

- 상호작용 범위를 벗어남
- 입력 취소
- 피격
- 사망
- 다른 행동 수행
- 살인 애니메이션 발생

## 거짓말 탐지기 시스템

- 목적
  - 거짓말 탐지기는 범인의 정체를 직접 공개하지 않고, 플레이어의 행동 정보를 바탕으로 단서를 제공하는 추리 보조 시스템

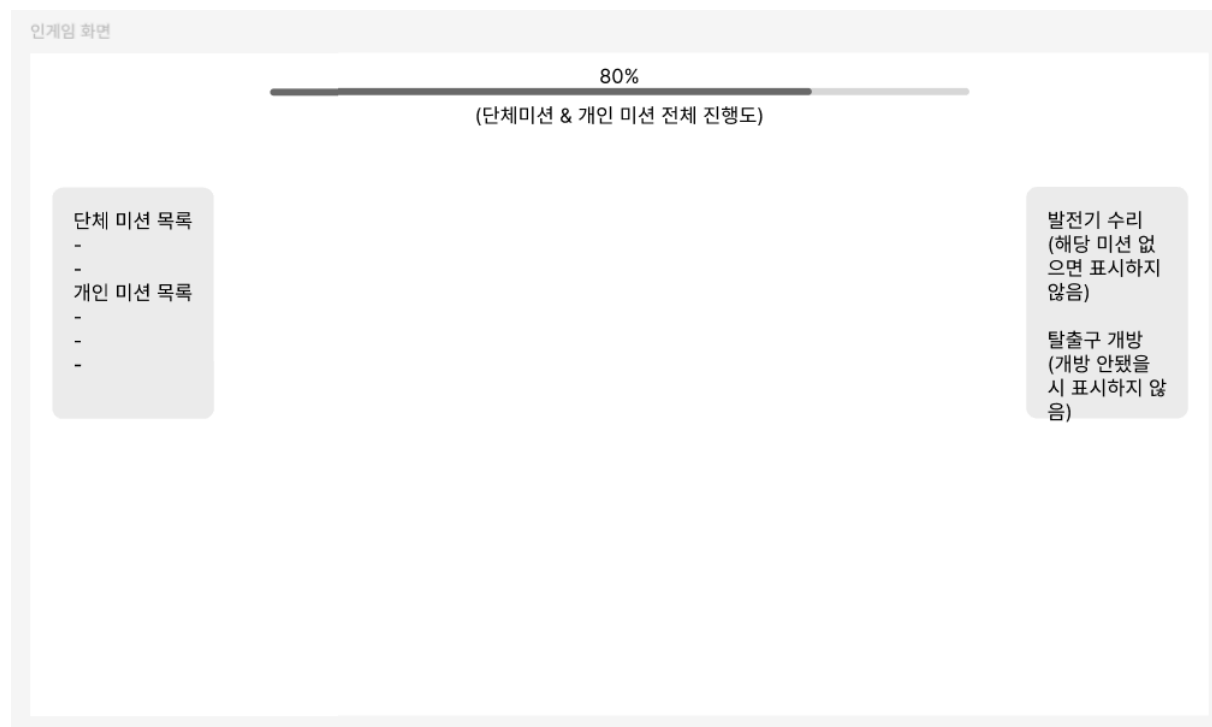
## 기본 규칙

| 항목       | 내용        |
|----------|-----------|
| 사용 횟수    | 게임당 3회    |
| 사용 대상    | 모든 플레이어   |
| 결과       | 범인의 행동 단서 |
| 직접 정체 공개 | 불가능       |

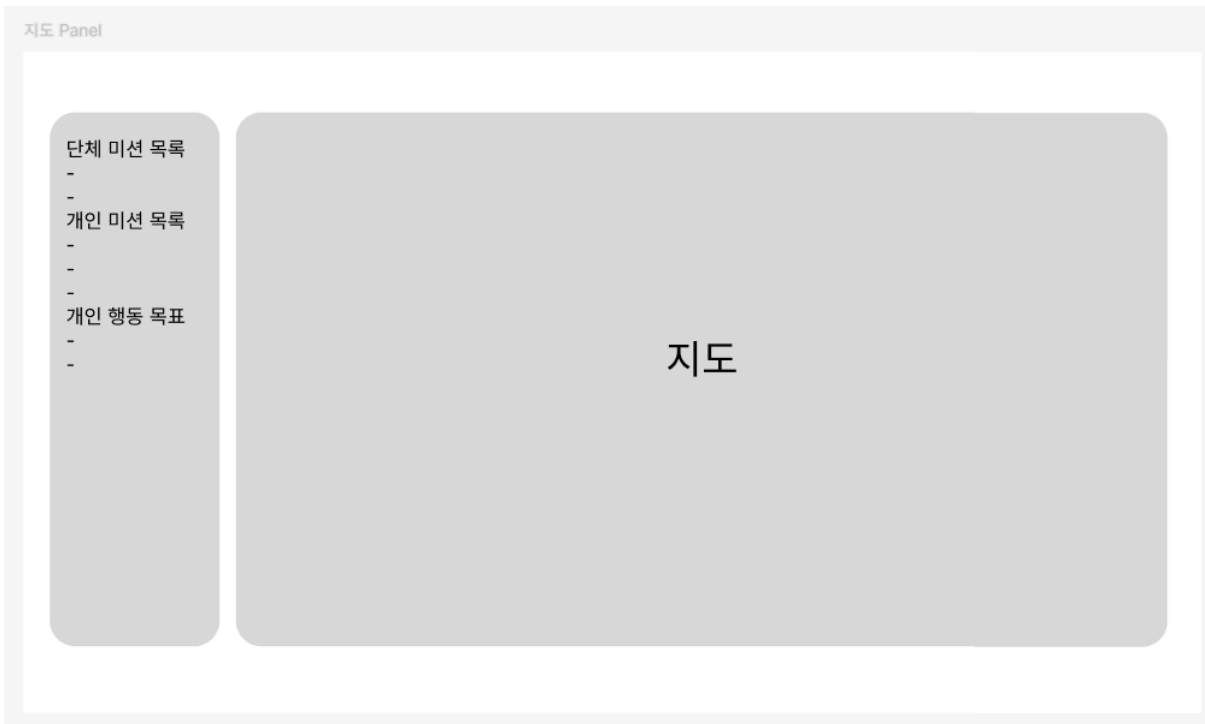
## 처리 흐름

## UI 구성

- 인게임 화면



- 지도 Panel 화면



#### 서버 / 클라이언트 역할 분리

| 기능                | 서버      | 클라이언트 |
|-------------------|---------|-------|
| 미션 <b>Pool</b> 관리 | O       | X     |
| 미션 랜덤 생성          | O       | X     |
| 단체 미션 선정          | O       | X     |
| 개인 미션 배정          | O       | 표시    |
| 행동 목표 생성          | O       | 표시    |
| 충돌 검사             | O       | X     |
| 미션 상호작용 요청        | 수신 / 검사 | 입력    |
| 상호작용 UI           | X       | O     |
| 진행 바 표시           | X       | O     |
| 미션 진행도 판정         | O       | X     |
| 완료 판정             | O       | 연출    |
| 공동 진행도 관리         | O       | 표시    |
| 개인 진행도 관리         | O       | 표시    |
| 행동 목표 성공 판정       | O       | 표시    |

|               |        |       |
|---------------|--------|-------|
| 살인 가능 상태 관리   | O      | 표시    |
| 거짓말 탐지기 사용 횟수 | O      | 표시    |
| 단서 생성         | O      | 표시    |
| 미션 완료 알림      | 결과 전달  | O     |
| 게임 종료 결과 공개   | 데이터 전달 | UI 표시 |



## 단체 미션

# 발전기 작동시키기

## 목표

시설 곳곳에 있는 발전기 3대를 모두 수리하여 시설에 전력을 공급

## 방법

- 맵 곳곳에 배치된 발전기 3대를 찾는다
- 각 발전기에 있는 상호작용 버튼을 누르면 발전기 수리가 진행된다
- 발전기 한 대의 수리가 완료되면 해당 발전기에 「수리 완료」가 표시된다
- 첫 번째 발전기가 수리 완료되는 순간 다음 발전기를 수리해야 하는 제한 시간이 시작된다
- 제한 시간은 2분이며 2분 안에 다음 발전기의 수리를 완료해야 한다
- 두 번째 발전기 수리가 완료되면 다시 2분의 제한 시간이 시작된다
- 제한 시간 안에 마지막 발전기까지 수리해야 한다
- 발전기 3대가 모두 「수리 완료」 상태가 되면 미션이 완료된다

## 진행 예시

발전기 1 수리 완료

↓

2분 이내 발전기 2 수리 완료

↓

2분 이내 발전기 3 수리 완료

↓

발전기 3대 모두 수리 완료 → 미션 성공

## 실패 조건

- 첫 번째 발전기 수리 완료 후 2분 안에 두 번째 발전기를 수리하지 못한 경우
- 두 번째 발전기 수리 완료 후 2분 안에 마지막 발전기를 수리하지 못한 경우

제한 시간 안에 다음 발전기를 수리하지 못하면 발전기 미션이 초기화되며 다시 진행해야함

## 완료 조건

발전기 1 — 수리 완료  
발전기 2 — 수리 완료  
발전기 3 — 수리 완료

## 대형 방화문 열기

### 목표

모든 단체 미션을 완료한 후 잠겨 있는 대형 방화문을 개방하여 시설에서 탈출

### 해금 조건

모든 단체 미션을 완료해야 방화문 개방을 시도할 수 있음

### 방법

- 모든 단체 미션을 완료한다
- 대형 방화문 주변 바닥에 떨어져 있는 레버를 찾는다
- 플레이어들은 떨어진 레버를 주워 대형 방화문 양쪽에 있는 레버 장착 위치까지 가져간다
- 양쪽 레버 위치에 레버를 모두 끼운다
- 모든 레버가 장착되면 30초의 제한 시간이 시작된다
- 제한 시간 안에 플레이어들이 장착된 레버를 모두 아래로 내려야 한다
- 모든 레버를 제한 시간 안에 내리는 데 성공하면 방화문의 잠금이 해제된다
- 잠금이 해제되면 대형 방화문이 열리며 다음 단계 또는 탈출이 가능해진다

### 진행 순서

모든 단체 미션 완료

↓

대형 방화문 주변에서 떨어진 레버 수집

↓

대형 방화문 양쪽 레버 위치에 장착

↓

30초 제한 시간 시작

↓

모든 레버를 아래로 내리기

↓  
대형 방화문 개방

## 완료 조건

모든 단체 미션을 완료한 상태에서 제한 시간 **30초** 안에 모든 레버를 장착하고 아래로 내리기

## 실패 조건

- 모든 단체 미션을 완료하지 않은 상태에서 방화문을 열려고 하는 경우
- 제한 시간 **30초** 안에 모든 레버를 내리지 못한 경우

실패 시 레버는 튕겨져 나옴

## 코드 순서 맞추기

### 목표

중앙 컴퓨터에 연결된 숫자 자판을 이용하여 보안 코드를 입력하고 시스템을 복구

### 방법

- 중앙 컴퓨터와 연결된 숫자 자판을 찾는다
- 숫자 자판에는 **1~9**까지의 숫자 버튼이 배치되어 있다
- 자판 옆에 있는 빨간색 버튼을 누르면 코드 입력이 시작된다
- 빨간색 버튼을 누르면 **1~9** 숫자 자판 중 **4**개의 숫자에 불이 들어온다
- 숫자는 한 번에 하나씩 랜덤한 순서로 점등된다
- 플레이어는 숫자에 불이 들어오는 순서를 기억해야 한다
- **4**개의 숫자가 모두 점등되면 불이 꺼지고 입력 대기 상태가 된다
- 플레이어는 방금 불이 들어왔던 순서대로 해당 숫자 버튼을 차례대로 누른다
- 입력한 순서가 점등된 순서와 일치하면 다음 단계로 넘어간다
- 모든 숫자를 올바른 순서로 입력한 후 파란색 버튼을 누른다
- 파란색 버튼을 눌렀을 때 입력한 순서가 정확하면 미션이 완료된다

### 진행 순서

빨간색 버튼을 누름

↓

숫자 자판에서 랜덤하게 숫자가 점등

**3 → 8 → 1 → 6**

↓

모든 숫자의 불이 꺼짐

↓

플레이어가 기억한 순서대로 입력

**3 → 8 → 1 → 6**

↓

입력 성공

↓

파란색 버튼 누르기

↓

미션 클리어

## 완료 조건

빨간색 버튼 → 4개의 숫자 랜덤 점등 → 점등 순서 기억 → 동일한 순서로 숫자 입력 → 파란색 버튼

## 실패 조건

- 숫자를 잘못된 순서로 입력한 경우
- 점등되지 않은 숫자를 입력한 경우
- 입력 순서를 틀린 상태에서 파란색 버튼을 누른 경우

실패하면 입력 내용이 초기화되고 빨간색 버튼을 다시 눌러 새로운 4개의 숫자 순서를 생성해야함

## 고장난 장비 조립

### 목표

고장 난 장비의 부품을 찾아 원래 위치에 조립하여 장비를 복구

## 방법

- 고장 난 장비와 주변에 흩어져 있는 부품을 확인한다
- 장비를 수리하기 위해 필요한 부품을 각각 찾아낸다
- 필요한 부품은 총 4개이며, 맵 곳곳에 배치되어 있다
- 플레이어는 각 부품을 찾아 장비가 있는 조립 위치로 가져온다
- 조립 위치에 부품을 가져오면 해당 부품이 장비에 자동으로 배치된다
- 필요한 모든 부품이 조립 위치에 모이면 장비가 자동으로 조립된다
- 조립이 완료되면 장비가 정상적으로 작동하며 미션이 완료된다

## 진행 순서

부품이 필요한 고장 난 장비 발견

↓

필요한 부품 확인

↓

맵에서 부품 탐색

↓

부품 ① 획득

↓

부품 ② 획득

↓

부품 ③ 획득

↓

부품 ④ 획득

↓

조립 위치로 모든 부품 이동

↓

부품이 자동으로 장비에 조립

↓

장비 정상 작동

↓

미션 클리어

## 완료 조건

고장 난 장비 확인 → 필요한 부품 4개 탐색 → 모든 부품을 조립 위치로 가져오기 → 자동 조립 → 장비 정상 작동

## 실패 조건

- 필요한 부품을 모두 모으지 못한 상태에서는 조립이 완료되지 않음
- 잘못된 부품을 조립 위치에 가져와도 해당 장비의 부품으로 인정되지 않음
- 제한 시간이 있는 경우, 제한 시간 내 모든 부품을 모으지 못하면 미션이 실패
- 미션 실패 시 조립 진행 상태가 초기화되고 부품을 다시 찾아야함

## 생명 유지 장치 복구

산소 공급 시스템을 정상화한다.

방법

- 산소 밸브 4개를 찾는다.
- 제한 시간 안에 모두 열거나 닫는다.
- 성공 시 산소 공급이 정상화된다.

# 개인 미션



## 밸브 잠그기

가스 누출 또는 증기 누출을 막는다.

### 방법

- 새는 밸브를 찾는다.
- 상호작용한다.
- 표시되는 방향으로 밸브를 끝까지 돌린다.
- 성공 시 누출이 멈춘다.

## 차단기 올리기

꺼진 전기 회로를 복구한다.

### 방법

- 차단기 패널 발견
- 내려간 스위치를 모두 올린다.
- 완료 시 전력 복구

## 전선 연결하기

전선을 다시 연결한다.

### 방법

- 끊어진 선 확인
- 같은 색 선끼리 연결
- 모두 연결하면 완료

## 안테나 방향 맞추기

통신 신호 복구

### 방법

- 안테나 회전
- 신호 게이지가 최대가 되는 위치 찾기

- 고정

## 압력 수치 맞추기

기계 압력 안정화

방법

- 게이지가 움직인다.
- 초록색 구간에서 버튼을 눌러 멈춘다.
- 성공 시 완료

## 필터 청소하기

공기 정화 시스템 복구

방법

- 막힌 먼지나 이물질 제거
- 드래그하거나 클릭으로 제거
- 모두 제거 시 완료

## 개인 미션 - 예시

## 밸브 잠그기

(자동 | 추후에 성공,실패 경우의수 추가 | 우선은 성공만)

가스 누출 또는 증기 누출을 막는다.

### 방법

- 새는 밸브를 찾는다.
- 상호작용한다.
- 표시되는 방향으로 밸브를 끝까지 돌린다.
- 성공 시 누출이 멈춘다.



## 차단기 올리기

꺼진 전기 회로를 복구한다.

### 방법

- 차단기 패널 발견
- 내려간 스위치를 모두 올린다.
- 완료 시 전력 복구



빨간색 차단버튼 올림  
100% 차단 완료되면 미션 완료

## 전선 연결하기

전선을 다시 연결한다.

### 방법

- 끊어진 선 확인
- 같은 색 선끼리 연결
- 모두 연결하면 완료

<https://youtu.be/6rrP8baJ1cq>

영상에선 색이 다른 선끼리 이어지지만 우리는 같은 색끼리 연결함

## 안테나 방향 맞추기

통신 신호 복구

방법

- 안테나 회전
- 신호 게이지가 최대가 되는 위치 찾기
- 고정



## 압력 수치 맞추기

기계 압력 안정화

방법

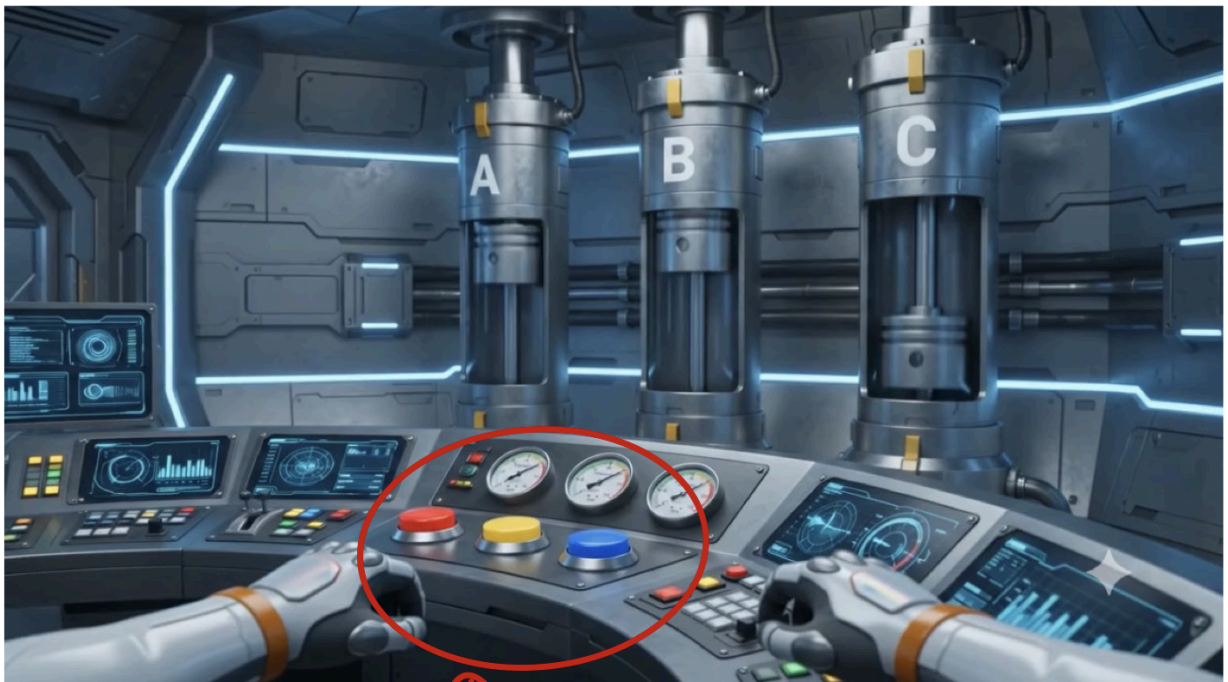
- 게이지가 움직인다.
- 초록색 구간에서 버튼을 눌러 멈춘다.
- 성공 시 완료

<https://youtu.be/KBMjswNhelk>

A, B, C 3개의 압력통이 있고 3가지의 색의 버튼이 있음

어떤 색의 버튼이 어떤 압력통이랑 연결되어있는지는 모름 (하나 하나 눌러서 찾아야함)

일정 높이에 압력추 가 도달하면 A, B, C에서 초록불이 들어옴 (영상에는 없음)



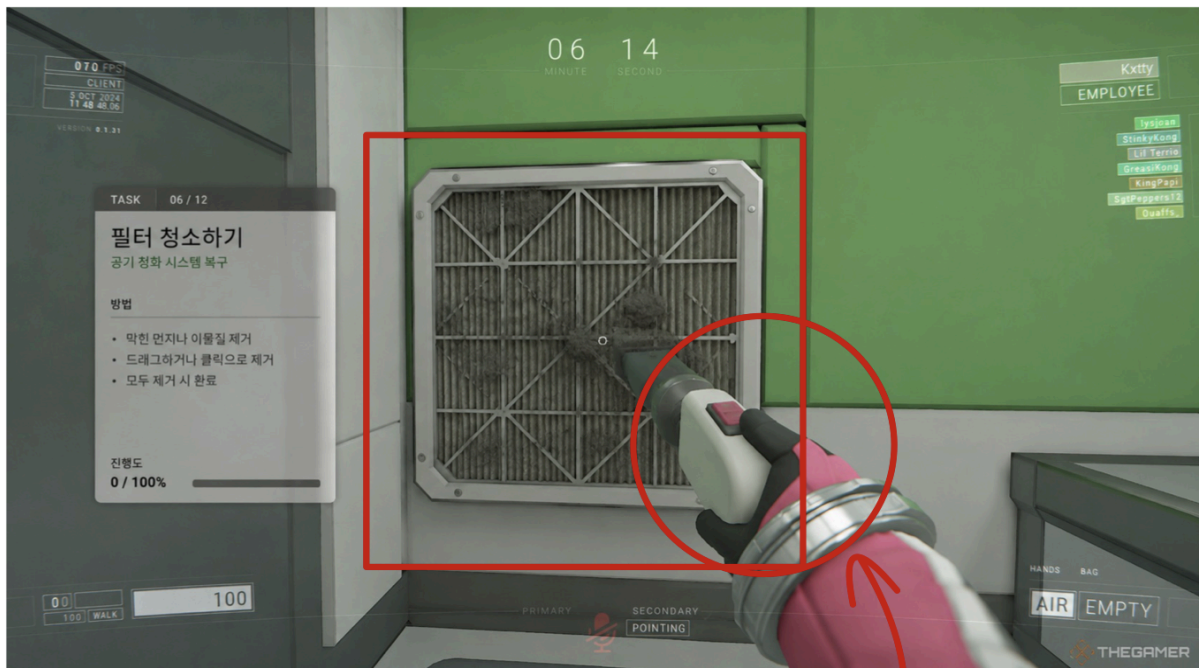
여기 버튼을 눌러서  
위에 압력 장치 높이를 맞춰야함

## 필터 청소하기

공기 정화 시스템 복구

방법

- 막힌 먼지나 이물질 제거
- 드래그하거나 클릭으로 제거
- 모두 제거 시 완료



청소기를 찾아서  
필터의 먼지를 치운다



# 개인 행동 목표

## 말하지 않는다

- 게임이 끝날 때까지 음성 채팅이나 마이크를 통한 발언을 하지 않는다.

## 뛰지 않는다

- 이동 시 달리기 기능을 사용하지 않고 천천히 이동해야 한다.

## 점프를 하지 않는다

- 게임이 종료될 때까지 점프 행동을 한 번도 하지 않아야 한다.

## 뒤를 돌아보지 않는다

- 플레이어는 뒤를 확인할 수 없으며 마우스 커서나 카메라 회전 각도를 제한하여 뒤를 보는 행동을 방지한다.

## 일정 데시벨 이하로 말한다

- 마이크 입력이 설정된 데시벨을 초과하면 즉시 탈락한다.

## N초 이상 제자리에 머물지 않는다

- 예를 들어 5초 이상 이동하지 않으면 즉시 탈락하며 지속적으로 위치를 변경해야 한다.

## 걸지 않는다

- 걷기 상태를 사용할 수 없으며 다른 이동 방식(예: 뛰기만 가능, 웅크리기만 가능 등)으로만 이동해야 한다.

## HP가 특정 수치 이하로 떨어지지 않는다

- 게임 시작 시 생존 기준 HP가 무작위로 결정되며 HP가 해당 수치 아래로 감소하는 순간 즉시 탈락한다.

## 화면 안에 N명 이상의 플레이어를 담지 않는다

- 자신의 화면에 설정된 인원 수 이상이 동시에 보이면 즉시 탈락한다. 항상 다른 플레이어와 적절한 거리를 유지해야 한다.

# 상호작용 시스템



추리, 의심 시스템



아이템



인벤토리 슬롯 몇칸으로 할지? 의견? 받아요?

+5칸? 4칸? 그정도가 적당하지않을까여

아이템 사용량을 걸어서 사용량을 다 사용하고 나면 아이템이 사라짐

최소 플레이어를 죽일 수 있는 횟수

허공에 헛스윙 하는 횟수는 차감하지 않음

피해량이 100을 넘지 않을때는 100을 넘었을 때 가장 최소 값으로 사용 횟수를 설정함

예) 플레이어 체력 : 100

드라이버

피해량 : 10

제한된 사용 횟수 : 10

총 피해량 : 100

나이프

피해량 : 23

제한된 사용 횟수 : 5

총 피해량 : 115

총

피해량 : 35

제한된 사용 횟수 : 3

총 피해량 : 105

# 설정창

